

CMSSDiracAux - Full Technical Report

Merged report from repository Markdown sources

Todor Ivanov

2026-03-16

Contents

1 Preface	15
2 CMSSDiracAux Technical Report	15
3 CMSSDiracAux Report	15
4 Table of Contents	16
4.1 Introduction	16
5 Background	16
5.1 CMS Computing Architecture	16
5.2 WMCore Architecture	16
5.3 DIRAC Computing Architecture	16
5.4 DIRACX Evolution	16
6 Architecture	17
6.1 Execution Models	17
7 Interoperability Analysis	17
7.1 Workflow Model Differences	17
7.2 Translation Layer	17
7.3 Runtime and Job Translation	17
7.4 Dataset Handling	17
8 Implementation	17
9 Evaluation	18
10 Development Checkpoints	18
10.1 Current major PoC limitations	18
10.2 Current request-scoped output layout	18
10.3 Active follow-up items	18
11 CMS Workflow Management System Architecture	19
11.1 Purpose of this section	19

12 Overview of CMS Workflow Architecture	19
13 Workflow Definition Layer	20
14 WMCore Framework	21
15 Workflow Task Structure	21
16 WMBS: Job Generation and Bookkeeping	22
17 Splitting and Workload Partitioning	23
18 Job Submission Infrastructure	24
19 CMS Workflow Abstraction Levels	24
20 Architectural Characteristics	25
21 Relationship to DIRAC Workflow Systems	25
22 Implications for CMSDiracAux	26
23 Detailed architecture	26
24 Summary	29
25 WMBS Splitting Model	29
25.1 Purpose of this section	29
26 CMS Data Granularity	30
27 WMBS Role in Workflow Execution	31
28 Splitting Algorithms	31
29 Architectural Motivation for WMBS	32
30 Implications for CMSDiracAux	32
31 Impact on Scheduling and Resource Utilization	33
32 Consequences for Interoperability with DIRAC	33
33 Summary	34
34 CMS data hierarchy	34
35 CMS Job Splitting Algorithms	38
35.1 Splitting dimension	39
35.21. File-level splitting	39
35.32. Lumi-based splitting	39
35.43. Event-based splitting	40

35.54. Event-aware lumi splitting	40
35.65. Run-based splitting	41
35.76. Lumi-run hybrid splitting	41
35.87. Dataset-level splitting	41
35.98. Block-based splitting	42
35.10. Production-style event generation splitting	42
35.110. Event-range splitting	42
35.11. The three <i>most important</i> in CMS production	43
35.12. CMS splitting visualized	43
36 1. FileBased splitting	43
37 2. LumiBased splitting	44
38 3. EventBased splitting	44
39 4. EventAwareLumiBased splitting	45
40 5. RunBased splitting (rare)	46
41 Importance for the CMS vs. DIRAC interoperability	46
42 CMS Workflow Splitting Model — Storage vs Content Hierarchy	46
43 DIRAC Workload Management System	51
43.1 Purpose of this section	51
44 Role of the Workload Management System	52
45 Core Scheduling Paradigm: Pilot Jobs	52
46 Job Lifecycle in DIRAC WMS	53
47 Internal Architecture of the WMS	54
48 Task Queues	55
49 Resource Integration	55
50 Monitoring and Bookkeeping	56
51 Comparison with CMS Workflow Scheduling	56
52 Implications for CMSDiracAux	56
53 Summary	57
54 Architectural Relationship of DIRAC Workflow Systems	57
54.1 Purpose of this section	57
55 DIRAC Architectural Context	58

56The Production System: Workflow Orchestration	59
57The Transformation System: Data-Driven Job Generation	59
58The Workload Management System: Job Scheduling	60
59Relationship Between the Systems	61
60Workflow Abstraction Levels	62
61Architectural Implications	62
62Implications for CMSDiracAux	63
63Detailed architecture	63
64Summary	66
65DIRAC Production System	66
65.1 Purpose of this section	66
66Role of the Production System in DIRAC	67
67Concept of Productions	67
68Production Workflow Structure	68
69Relationship Between Productions and Transformations	69
70Job Creation within a Production	69
71Monitoring and Bookkeeping	70
72Comparison with CMS Workflow Management	71
73Implications for CMSDiracAux	71
74Summary	72
75DIRAC Transformation System	72
75.1 Purpose of this section	72
76DIRAC Architecture Context	72
77Concept of Transformations	73
78Transformation Lifecycle	74
79Data-Driven Job Generation	74
80File-Level Processing Model	75

81 Pilot Job Execution Model	76
82 Job Execution Description	77
83 Transformation Plugins	77
84 Comparison with CMS Workflow Execution	78
85 Implications for CMSDiracAux	78
86 Summary	79
87 Changes with DiracX	79
88 1. Core Idea of DIRACX	79
89 2. Structural Change in the System	80
89.1 Classic DIRAC	80
89.2 DIRACX	80
90 3. Transformation System Evolution	81
91 4. Production System Evolution	81
92 5. Workload Management Changes	82
93 6. Control Plane vs Execution Plane	82
94 7. Impact on Workflow Abstractions	83
95 8. Why This Matters for CMSDiracAux	84
96 9. Key Architectural Differences	84
97 10. The Big Picture	84
98 Architectural Comparison: CMS Workflow System, DIRAC, and DIRACX	85
98.1 Purpose	85
99 Workflow Architecture Comparison	85
100 Workflow Abstraction Layers	87
101 Scheduling Model	88
102 Data Processing Granularity	89
103 Structural Differences	89
104 Relevance for CMSDiracAux	90
105 Summary	90

106 Knowledge Base	92
107. Abstraction layers	92
108. Translation pipeline	93
109. Major architectural mismatch discovered	93
109. Runtime code distribution	93
109. Job execution model	94
109. Job splitting model	94
110. DIRAC architecture relevant to the PoC	94
111. CMS workflow complexity (critical insight)	95
112. Workflow structure in CMS	96
113 Important insight discovered	96
114 WMCORE vs DIRAC Workflow Model Comparison	97
115 Workflow Architecture Comparison	97
116 Key Architectural Differences	98
116.1 Runtime environment	98
116.2 Workflow semantics	98
116.3 Job splitting model	98
116.4 Runtime entry point	98
117. Formal Translation IR Schema	99
118 Translation IR Layer	99
119 Translation IR Core Entities	99
119. Workflow	99
119. Task	99
119. RuntimeDefinition	100
119. DataReference	100
119. SplittingPolicy	100
119. JobTemplate	100
120 Translation IR Graph	101
121 IR → DIRAC Mapping	101
122 IR → CWL Mapping	101
123 Why Translation IR is Essential	102
124 Updated Architecture Diagram	102

125	Important Insight for the Report	103
126	System Architecture Diagram	103
127	System Architecture	106
127.0	Overview	106
128	Architectural Layers	106
129	CMS Workflow Management Layer	107
130	CMSDiracAux Translation Layer	109
131	Canonical Translation IR	109
132	DIRAC Execution Infrastructure	110
133	DIRAC Workload Management	111
134	Worker Node Runtime Environment	111
135	Runtime Environment Constraints	112
136	Runtime Distribution Strategy	113
137	Architectural Summary	113
138	Detailed view	114
139	Key Architectural Insight	116
140	CMSDiracAux Main Architecture	116
141	Key Architectural Message of the Diagram	120
142	Important Insight Highlighted	120
143	CMS-DIRAC hybrid system	121
144	Current CMS Workflow management system	121
145	Hybrid CMS-DIRAC Workflow System with CMSDiracAux as transitional layer/system	124
146	Interpretation of the Two Systems	126
146.0	First cut and new inter system communication line — workflow extraction	127
146.1	Second cut and new inter system communication line — execution backend	127
147	Key Architectural Insight	128
148	VMCore vs DIRAC Execution Model	128

14CMS vs DIRAC Workflow Execution Models	133
15Data Interaction Comparison	135
15Conceptual Interpretation	136
15Physics Analogy	136
15Implication for CMSDiracAux	136
15WMCore-DIRAC Workflow Model Mismatch	137
154.Purpose of this section	137
15CMS Workflow Execution Model	138
15Key Characteristics of the CMS Workflow Model	139
15DIRAC Workflow Execution Model	140
15Key Characteristics of the DIRAC Workflow Model	141
15Architectural Comparison	142
16Why a Translation Layer is Necessary	144
16Static vs Dynamic Workflow-Data Coupling	145
16CMS Workflow Model: Static Workflow Partitioning	145
16DIRAC Workflow Model: Dynamic Workload Expansion	146
16Comparison of the Two Models	147
16Analogy Between Quantum Pictures and Workflow Management Models	147
165.Context	147
16Schrödinger Picture	147
166.Analogy with CMS WMCore	148
16Heisenberg Picture	148
167.Analogy with DIRAC	148
16Conceptual Summary	149
16Validity of the Analogy	149
17Implications for Workflow Translation	150
17Static vs Dynamic Workflow-Data Interaction Models	150
17WMBS necessity argument	153

17	DIRACX outlook	153
17	Conceptual Interpretation	153
17	IR Layer placement	154
17	WMBS, DIRAC, and the Granularity Gap	155
176.	CMS workflow-management granularity	155
176.	Lower-level scheduling granularity	155
176.	Why WMBS exists	156
176.	Relevance for the current PoC	156
17	Translation IR Rationale	156
177.	Purpose	156
17	Motivation	157
17	Position of the Translation IR	157
18	Why Direct Field Mapping Is Insufficient	158
180.	Field-level schema with more concrete names	160
180.	What this table is really saying	161
180.	A more realistic mini-example	162
18	Translation IR Architecture	162
18	Canonical Translation Objects	163
18	Workflow Parameter Mapping	163
18	Dataset Representation Mapping	164
18	Task Definition Mapping	164
18	Job Definition Mapping	165
18	Splitting Policy Representation	165
18	DIRAC Materialization	166
18	Role of the Translation IR in CMSDiracAux	166
19	Summary	166
19	Translation IR Design	167
191.	Overview	167
19	Architectural Role of the Translation IR	168
19	Motivation for an Intermediate Representation	168
193.	Workflow definition philosophy	169

193.	Data-driven splitting differences	169
19	Design Goals	170
19	Translation IR Architecture	170
19	Conceptual Structure of the Translation IR	171
19	Canonical Workflow Object	171
19	Canonical Task Object	172
19	Canonical Splitting Description	172
20	Dataset Resolution	172
20	Translation Pipeline	173
20	Mapping Between WMCore and Translation IR	173
20	Mapping to DIRAC	174
20	Mapping to CWL	174
20	Translation IR and DIRAC Materialization	175
20	Relationship to WMBS	175
20	Role in the CMSDiracAux Architecture	176
20	Importance of the Translation IR	176
20	Design Advantages	176
21	Summary	177
21	Dataset Resolution Model	177
211.	Purpose	177
21	CMS Data Hierarchy	177
21	Dataset References in Workflow Definitions	178
21	Dataset Discovery Infrastructure	179
21	Data Representation in DIRAC	179
21	Dataset Resolution Pipeline	180
21	Example DAS Query	180
21	Dataset Resolution in CMSDiracAux	181

21	Handling Large Datasets	181
22	Integration with Splitting	182
22	Architectural Role in the CMSDiracAux System	182
22	Implications for DIRAC Execution	183
22	Importance for Workflow Interoperability	183
22	Summary	184
22	Job Description Translation	184
	225. Purpose of this section	184
22	DIRAC Job Execution Model	184
22	JDL Job Definition	185
22	Role of jobDescription.xml	186
22	Job Description Translation Pipeline	186
23	Mapping Translation IR to DIRAC Job Structures	187
23	Runtime Entry Point	187
23	Runtime Environment Considerations	187
23	Interaction with Job Splitting	188
23	Role in the CMSDiracAux Architecture	188
23	Limitations of the Prototype	189
23	Importance for Workflow Interoperability	189
23	Job Runtime Differences Between CMS and DIRAC	190
	237. Purpose	190
23	CMS Runtime Execution Model	190
23	Runtime Dependencies of CMS Jobs	191
24	Job Identity and PSet Tweaks	192
24	DIRAC Runtime Execution Model	192
24	Sandbox Distribution Constraints	193
24	DIRAC Workflow Construction	194

24	Bootstrap Execution Strategy	194
24	Role of __CMSJobParameter__	195
24	Translation IR as a Replacement Layer	196
24	Relationship Between IR and Runtime Parameters	196
24	How This Relates to CMSDiracAux	197
24	Why This Is Needed	197
25	The Resulting Runtime Strategy	198
25	How the __CMSJobParameter__ Fields Fit	199
25	Architectural Interpretation	199
25	Why This Matters for CMSDiracAux	200
25	Key Architectural Constraint	200
25	1. What WMBS Actually Does	201
25	2. DIRAC's Native Model	201
25	3. The Core Mismatch	202
25	4. What Happens If WMBS is Removed	203
25	5. What Happens in CMSDiracAux	203
26	6. Where the WMBS Logic Moves	204
26	7. Architectural Consequence	204
26	8. Impact on Scheduling	205
26	9. Final Architectural Insight	206
26	How things change with DiracX	206
26	Runtime constraints for CMS processes	208
	265. Where PSet tweaks fit	209
	265. How this interacts with DIRAC / DIRACX	209
26	Paths for change of the CMS runtime environment	210
	266.1. Distributing the Core Runtime Bundle via CVMFS	210
	266.2. Reconstructing Per-Job PSet Configurations at Runtime	210
26	Impact on the CMSDiracAux Architecture	211
	267. Direction 1: move the WMCore-dependent core bundle to CVMFS . . .	212

267.	Direction 2: rethink per-job PSet distribution	213
267.	How much do these two changes help?	213
267.	What does not change, even after both improvements	214
267.	DIRACX improvement	214
267.	Bottom line	215
26	MSDiracAux	216
268.	Relation to the IR layer of CMSDiracAux	217
26	Current architectural interpretation	219
27	On practice	219
27	In Conclusion	219
27	Summary	220
27	MS Runtime Construction Challenges	220
273.	Inputs to runtime behavior	220
273.	Runtime dependency layers	220
273.	Why this matters	221
273.	Current PoC approach	221
273.	Future direction	221
27	WMCore → DIRAC Parameter Mapping	221
27	Translator Design	226
275.	Overview	226
27	Translator Pipeline	227
27	Workflow Extraction	227
27	Workflow Normalization	228
27	Translation IR Construction	228
28	Translation Strategy	229
28	Translation Design Principles	229
28	Relationship to Execution Systems	230
28	Summary	230
28	DIRAC Materialization	230
284.	Overview	230
28	Materialization Pipeline	231
28	DIRAC Transformation System	231

28 CMS Splitting Plugin	231
28 Job Description Construction	232
28 Runtime Parameter Injection	233
29 Worker Node Execution	233
29 Summary	233
29 CWL Export	234
292. Motivation	234
29 Export Pipeline	234
29 Workflow Representation	234
29 Mapping IR Objects to CWL	234
29 Experimental Nature	235
29 Summary	235
29 Proof-of-Concept Evaluation	235
298. Overview	235
29 Translation Feasibility	235
30 Execution Model Compatibility	236
30 Preservation of Splitting Semantics	236
30 Runtime Reconstruction	236
30 Architectural Insight	237
30 Summary	237
30 Proof of Concept Evaluation	237
30 Limitations	241
30 Dataset Size Limitation	241
30 Incomplete Run/Lumi Support	242
30 Runtime Environment Distribution	242
31 DIRAC Server Integration	242
31 Experimental CWL Export	242

31	Summary	243
31	Current Stage Limitations and Checkpoints	243
313.	Server-side limitation	243
313.	Data visibility limitation	243
313.	Postponed topics	243
313.	Current milestone	243
31	Checkpoint — Local Transformation Materialization	244
314.	Achieved	244
314.	Local Pipeline	244
314.	Important Decision	244
314.	Known Gaps	245
314.	Next Step	245
314.	Future Branch	245
31	Checkpoint — DAS LFN Resolution and PoC File Cap	245
315.	Current milestone state	245
315.	Output directory structure	246
315.	DAS-based LFN resolution	246
315.	PoC scalability limitation	247
315.	CMS data hierarchy relevance	247
315.	DIRAC runtime observations	247
315.	Next technical directions	248
315.	Report workstream reminder	248

1 Preface

This document merges the report-oriented Markdown sources from the uploaded repository snapshot into a single PDF in report order.

\newpage

\newpage

2 CMSDiracAux Technical Report

This directory contains the evolving technical report for the WMCore → DIRAC interoperability proof of concept implemented in CMSDiracAux.

The report is organized as follows.

3 CMSDiracAux Report

This report documents the architecture, design, and proof-of-concept implementation of **CMSDiracAux**, a translation layer that enables interoperability between the

CMS workflow management system (WMCore) and the DIRAC distributed computing framework.

4 Table of Contents

4.1 Introduction

- Report Overview
 - Early Draft Notes
-

5 Background

5.1 CMS Computing Architecture

- CMS Workflow Management
- CMS Workflow Management System
- CMS Data Model
- CMS Splitting Model
- WMBS Splitting Model

5.2 WMCore Architecture

- WMCore Architecture
- WMCore-DIRAC High-Level Comparison

5.3 DIRAC Computing Architecture

- DIRAC Architecture
- DIRAC Production System
- DIRAC Transformation System
- DIRAC Workload Management System
- DIRAC Workflow Systems Architecture

5.4 DIRACX Evolution

- DIRAC → DIRACX Architectural Changes
 - DIRACX and CMS Integration
-

6 Architecture

- CMSDiracAux Main Architecture
- System Architecture
- Interoperability Architecture
- Architecture Diagram

6.1 Execution Models

- CMS vs DIRAC Execution Model
 - WMCore vs DIRAC Execution Model
-

7 Interoperability Analysis

7.1 Workflow Model Differences

- WMCore-DIRAC Workflow Mismatch
- WMBS-DIRAC Gap Analysis

7.2 Translation Layer

- Translation IR Rationale
- Translation IR Design

7.3 Runtime and Job Translation

- CMS Runtime Construction
- Job Runtime Differences
- Job Description Translation
- WMCore-DIRAC Parameter Mapping

7.4 Dataset Handling

- Dataset Resolution Model
-

8 Implementation

- Translator Design
 - DIRAC Materialization
 - CWL Export
-

9 Evaluation

- Proof-of-Concept Evaluation
 - Current Stage Limitations
 - System Limitations
-

10 Development Checkpoints

- Local Transformation Materialization
 - DAS LFN Resolution
-

10.1 Current major PoC limitations

- only the first 20 files per dataset are materialized
- no full run/lumi support
- no server-side DIRAC transformation agent integration yet

10.2 Current request-scoped output layout

```
REQUEST_ROOT
|
|-- WMCore.fetched.d
|
|-- DIRAC.transf.d
|
|-- DIRAC.cwl.d
```

10.3 Active follow-up items

- update the architecture diagram whenever the pipeline structure changes
- preserve the parameter mapping tables
- preserve the DIRAC InputSandbox vs jobDescription.xml analysis
- preserve the DAS/DBS data discovery branch notes

\newpage

\newpage

11 CMS Workflow Management System Architecture

11.1 Purpose of this section

The CMS Workflow Management System provides the infrastructure used to define, orchestrate, and execute large-scale data processing workflows across the distributed computing resources of the Worldwide LHC Computing Grid.

The CMS workflow system is designed to support complex multi-stage data processing campaigns, including:

- detector data reconstruction
- Monte Carlo simulation
- data reprocessing
- analysis data preparation

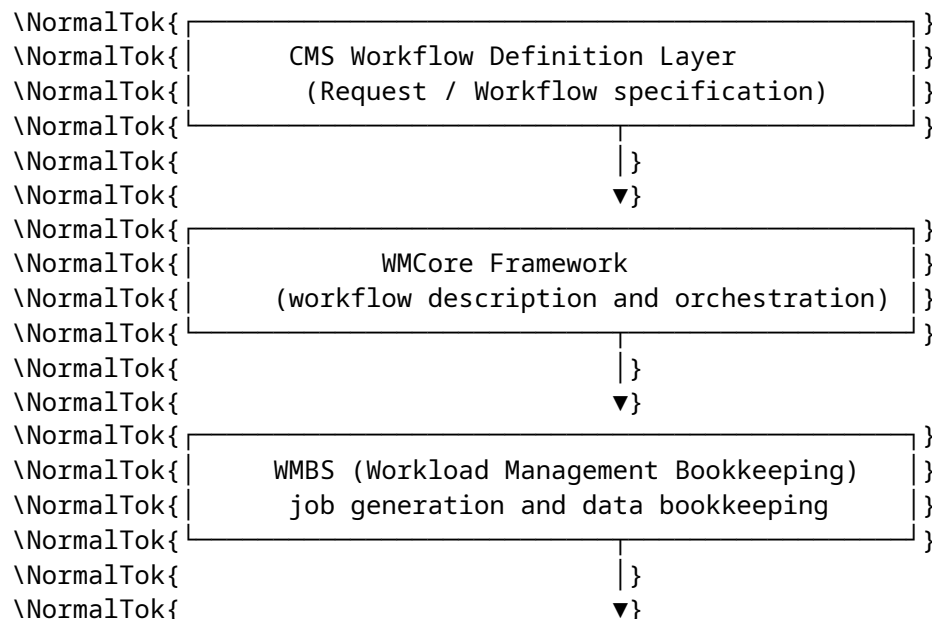
Unlike the DIRAC workflow architecture, which emphasizes dynamic job generation driven by available data, the CMS workflow management system relies on **explicit workflow descriptions and predefined job splitting strategies**.

Understanding the CMS workflow architecture is essential for evaluating how CMS workflows can be translated into DIRAC-compatible execution models.

12 Overview of CMS Workflow Architecture

The CMS workflow management architecture is composed of several subsystems that together transform workflow definitions into distributed jobs.

At a high level the system can be represented as:



```
\NormalTok{ }
\NormalTok{ Submission Infrastructure (SI) }
\NormalTok{ job submission to distributed resources }
\NormalTok{ }
\NormalTok{ |}
\NormalTok{ ▼}
\NormalTok{ }
\NormalTok{ Distributed Computing Resources }
\NormalTok{ (Grid / Cloud / HPC infrastructure) }
\NormalTok{ }
```

Each layer progressively converts the workflow definition into executable jobs.

13 Workflow Definition Layer

CMS workflows begin as **workflow requests** describing the processing to be performed.

These requests define:

- input datasets
- processing steps
- software configuration
- output datasets
- resource requirements

Conceptually:

```
\NormalTok{ }
\NormalTok{   Workflow Request }
\NormalTok{ }
\NormalTok{   input dataset }
\NormalTok{   processing steps }
\NormalTok{   CMSSW configuration }
\NormalTok{   output datasets }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{   }
\NormalTok{ }
\NormalTok{   Workflow Object }
\NormalTok{   internal WMCore structure }
\NormalTok{ }
```

These workflow definitions are typically created and managed through CMS request management systems.

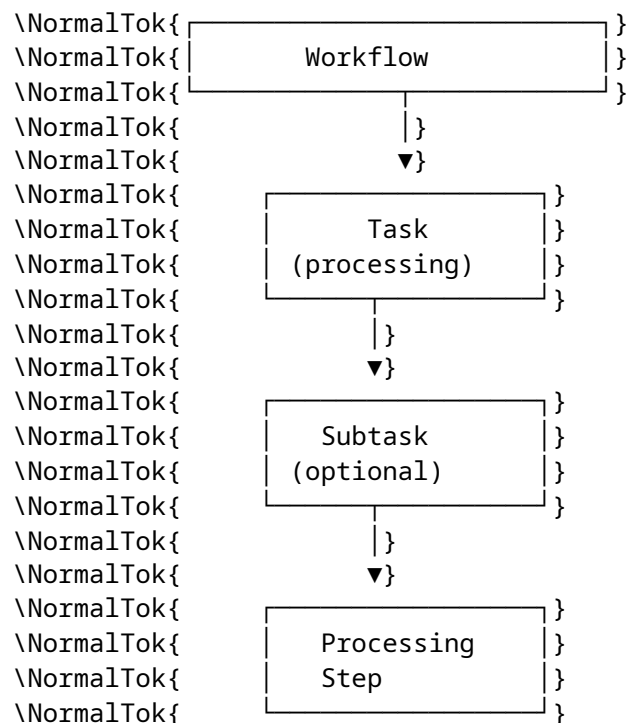
14 WMCore Framework

The **WMCore framework** provides the internal data structures and software components used to represent and manage CMS workflows.

WMCore defines:

- workflow objects
- task hierarchies
- data dependencies
- execution parameters

Conceptually a workflow is represented as a tree of tasks:



Tasks represent logical processing stages applied to input datasets.

15 Workflow Task Structure

Each task within a CMS workflow defines the processing applied to a subset of data.

Tasks specify:

- input dataset
- processing configuration
- splitting policy
- resource requirements

Example task structure:

```

\NormalTok{
\NormalTok{ Task
\NormalTok{
\NormalTok{ Input dataset
\NormalTok{ CMSSW configuration
\NormalTok{ Splitting policy
\NormalTok{ Output dataset
\NormalTok{
|
\NormalTok{ }
\NormalTok{ ▼}
\NormalTok{
\NormalTok{ Job definitions generated
\NormalTok{ by splitting algorithms
\NormalTok{

```

Tasks are therefore the primary abstraction used to represent workflow processing steps.

16 WMBS: Job Generation and Bookkeeping

The **Workload Management Bookkeeping System (WMBS)** is responsible for generating jobs and tracking the association between jobs and data.

WMBS performs several key functions:

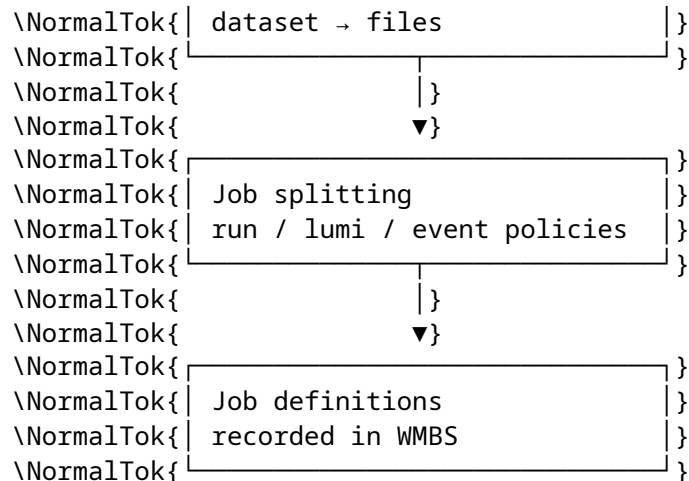
- dataset resolution
- file discovery
- job splitting
- job bookkeeping
- workflow progress tracking

Conceptually:

```

\NormalTok{
\NormalTok{ Workflow Task
\NormalTok{
|
\NormalTok{ }
\NormalTok{ ▼}
\NormalTok{
\NormalTok{ Dataset discovery
\NormalTok{ (DAS queries)
\NormalTok{
|
\NormalTok{ }
\NormalTok{ ▼}
\NormalTok{
\NormalTok{ File resolution
\NormalTok{

```



WMBS maintains the mapping between:

workflow tasks

↓

jobs

↓

data units

This bookkeeping is critical for large-scale workflow execution.

17 Splitting and Workload Partitioning

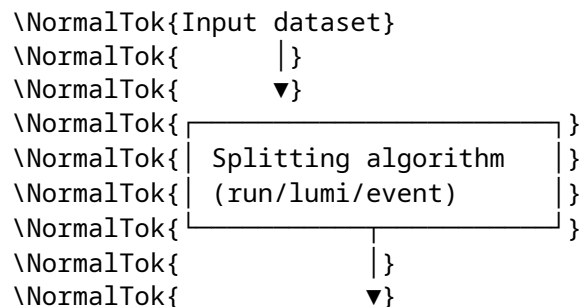
CMS workflows typically partition data using **splitting algorithms**.

These algorithms define how data are divided into jobs.

Typical splitting modes include:

- FileBased
- RunBased
- LumiBased
- EventAware

Conceptually:



```

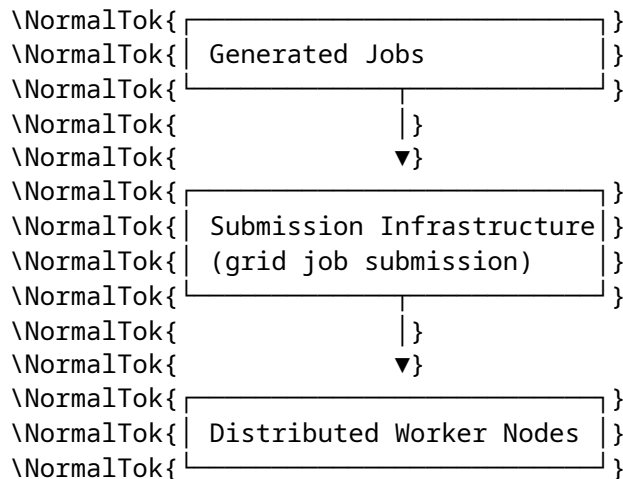
\NormalTok{ }
\NormalTok{ Job definitions }
\NormalTok{ each processes data }
\NormalTok{ }

```

These splitting strategies allow workflows to maintain predictable job runtimes despite heterogeneous data content.

18 Job Submission Infrastructure

Once jobs are generated by WMBS, they are submitted to distributed computing resources.



Jobs execute within the CMS runtime environment, typically provided through CVMFS.

19 CMS Workflow Abstraction Levels

The CMS workflow management system operates across several abstraction levels.

Layer	Workflow abstraction
Workflow request	full processing campaign
Workflow	structured set of tasks
Task	processing step
Job	executable workload unit

Conceptually:

```

\NormalTok{Processing campaign}
\NormalTok{    ↓}
\NormalTok{Workflow}
\NormalTok{    ↓}
\NormalTok{Tasks}
\NormalTok{    ↓}
\NormalTok{Jobs}

```

These abstractions allow CMS workflows to manage large processing campaigns consisting of millions of jobs.

20 Architectural Characteristics

Several characteristics distinguish the CMS workflow architecture:

- workflows are **explicitly defined before execution**
- job boundaries are **determined by splitting algorithms**
- bookkeeping tracks job-data relationships
- workflow state is maintained centrally

This architecture provides strong control over workflow execution and data processing.

21 Relationship to DIRAC Workflow Systems

The CMS workflow architecture differs from the DIRAC workflow stack.

Conceptually:

```

\NormalTok{CMS workflow architecture}

\NormalTok{Workflow}
\NormalTok{    ↓}
\NormalTok{Task}
\NormalTok{    ↓}
\NormalTok{WMBS splitting}
\NormalTok{    ↓}
\NormalTok{Jobs}

```

versus

```

\NormalTok{DIRAC workflow architecture}

\NormalTok{Production}
\NormalTok{    ↓}
\NormalTok{Transformation}
\NormalTok{    ↓}

```

```

\NormalTok{WMS scheduling}
\NormalTok{  ↓}
\NormalTok{Jobs}

```

These differences explain why CMS workflows cannot be directly expressed using DIRAC workflow constructs.

22 Implications for CMSDiracAux

The CMSDiracAux project introduces an intermediate abstraction layer that bridges the CMS workflow system and the DIRAC execution model.

Conceptually:

```

\NormalTok{CMS workflow (WMCORE)}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{Translation IR}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{DIRAC workflow systems}
\NormalTok{(production / transformation / WMS)}

```

This translation layer preserves workflow semantics while enabling execution within the DIRAC infrastructure.

23 Detailed architecture

```

\NormalTok{CMS Workflow Management System}
\NormalTok{  |}
↪  |}
\NormalTok{|}
↪  |}
\NormalTok{| CENTRAL SERVICES}
↪  |}
\NormalTok{|}
↪  |}
\NormalTok{|}
↪  |}
\NormalTok{|}
↪  |}
\NormalTok{|}
↪  |}
\NormalTok{| Request / ReqMgr | DBS / DAS | CouchDB /}
↪  | |}
\NormalTok{| workflow definition | data discovery | workflow}
↪  state | |}
\NormalTok{|}
↪  |}

```




```

\NormalTok{|           | Worker Nodes           |
↪ |}
\NormalTok{|           | CMS jobs execute         |
↪ |}
\NormalTok{|           |_____|
↪ |}
\NormalTok{|
↪ |}
\NormalTok{|_____|
↪ _____}

```

24 Summary

The CMS Workflow Management System provides a structured architecture for defining and executing large distributed workflows across the CMS computing infrastructure.

Key architectural components include:

- workflow definitions
- WMCore workflow objects
- WMBS job generation and bookkeeping
- distributed job submission

These components collectively transform high-level workflow requests into millions of distributed jobs executed across grid resources.

Understanding this architecture is essential for designing interoperability mechanisms such as the Translation IR used in the CMSDiracAux project.

\newpage

\newpage

25 WMBS Splitting Model

25.1 Purpose of this section

The CMS workflow management system includes a dedicated subsystem called **WMBS (Workload Management Bookkeeping System)** responsible for job generation and bookkeeping.

WMBS exists primarily because CMS workflows require **extremely fine-grained control over how workloads are distributed over data**.

Unlike many distributed computing systems that schedule work at the level of files or datasets, CMS workflows may split workloads at the level of:

- files
- runs
- luminosity sections
- events

This high-granularity splitting requires an explicit bookkeeping layer capable of associating **individual jobs with specific portions of the data content**.

26 CMS Data Granularity

CMS data management separates **storage organization** from **physics content structure**.

26.0.1 Storage hierarchy

```

Dataset
├─ Block
│   ├── File
│   └── File

```

26.0.2 Physics content hierarchy

```

Run
├─ Lumi Section
│   └─ Event

```

These hierarchies are orthogonal.

Files represent storage containers, while runs and luminosity sections represent the logical structure of recorded physics data.

Important structural properties:

Run $\not\subset$ File
 File $\not\subset$ Run

Lumi $\subset$ File
 Event $\subset$ Lumi

Consequently:

- runs may span multiple files
- files may contain events from multiple runs
- lumisections remain contained within files

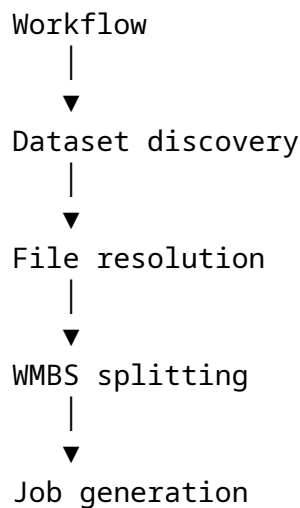
This structural mismatch between storage units and physics units is the **primary reason CMS requires sophisticated splitting algorithms**.

27 WMBS Role in Workflow Execution

WMBS is responsible for:

- associating workflows with datasets
- resolving datasets into file lists
- applying splitting algorithms
- generating job definitions
- tracking job-data associations

The workflow execution process in CMS can therefore be summarized as:



The splitting stage determines the **runtime boundaries of each job**.

28 Splitting Algorithms

CMS supports multiple splitting algorithms depending on the type of workload and the desired level of granularity.

Typical splitting modes include:

Splitting mode	Description
FileBased	jobs process full files
RunBased	jobs process runs
LumiBased	jobs process luminosity sections
EventAware	jobs process fixed event counts

These algorithms exist because **runs, lumisections and files are not aligned structures**.

For example:

- a run may span multiple files
- a file may contain events from multiple runs

Therefore splitting at the level of runs or lumisections requires **metadata inspection and bookkeeping beyond simple file lists**.

29 Architectural Motivation for WMBS

The existence of WMBS is not merely an implementation choice.

It reflects a fundamental architectural property of CMS workflows: **workload scheduling must be tied to physics data granularity rather than storage containers**.

The following observation is therefore critical for understanding CMS workflow design.

One of the reasons for WMBS existence at the first place is the fact that we have high granularity job splitting in CMS reaching as deep as data content (lumis/events) rather than only staying at data storage containers level (datasets/blocks/files), which means whatever central scheduling system we take we will still need to implement additional bookkeeping for associating jobs to data contents while distributing the workload over the data.

This in the concept of CMSDiracAux would mean re-implementing WMBS functionalities in the plugin mechanisms for job splitting. WMBS itself, good or bad implemented as is currently would have to be reborn as long as we keep this level of granularity.

30 Implications for CMSDiracAux

This architectural fact has direct implications for the CMSDiracAux project.

Even if CMS workflows are executed on top of DIRAC infrastructure, the system must still provide functionality equivalent to WMBS.

In particular:

- jobs must be associated with **specific lumisections or event ranges**
- splitting logic must preserve **deterministic mapping between jobs and data**

- bookkeeping must track which parts of a dataset have already been processed

Therefore the CMSDiracAux architecture effectively requires **re-implementing WMBS functionality inside DIRAC splitting plugins**.

31 Impact on Scheduling and Resource Utilization

Fine-grained splitting is not only a bookkeeping concern.

It directly affects how distributed resources are utilized.

How this impacts the lower levels of the system, such as Submission Infrastructure (SI) etc. It affects workload scheduling and resource pool utilization, because it delivers predictability of work entity at runtime, so that one may schedule based on projected runtime and fine-grained control over these parameters.

In CMS workflows:

- jobs process relatively uniform workloads
- runtime predictions are therefore reliable
- scheduling systems can distribute jobs efficiently

If splitting were performed only at the file level, this predictability would be lost.

While if we lose this low level of granularity splitting and live only with file-based splitting, due to the extremely non-consistent event contents of different files, the effect would be a randomization of the projected runtime parameter, loss of predictability of the resource utilization in terms of time, loss of control over job lengths and resource pool fragmentation.

Thus the WMBS splitting model is also a mechanism for **maintaining stable resource scheduling behavior across heterogeneous datasets**.

32 Consequences for Interoperability with DIRAC

DIRAC transformations typically operate at the **file level**.

However CMS workflows frequently require splitting at the **run or lumisection level**.

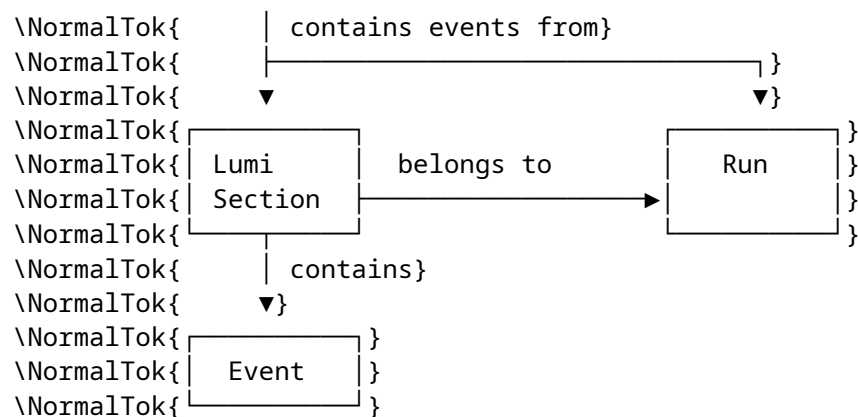
Therefore the CMSDiracAux translation layer must:

1. resolve datasets into files
2. inspect file metadata
3. map runs and lumisections to files

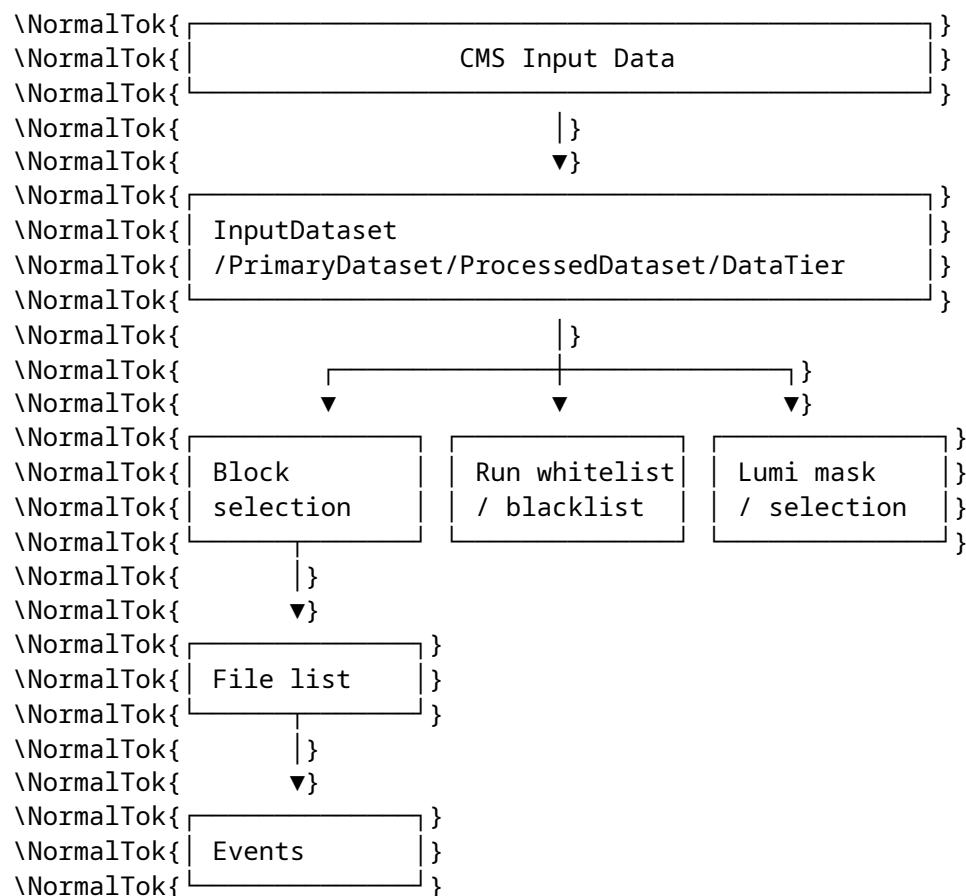
- Instead it requires **extended splitting plugins capable of CMS-style bookkeeping**.

The CMSDiracAux project addresses this requirement by embedding WMBS-like functionality inside the workflow translation layer and DIRAC splitting plugins.

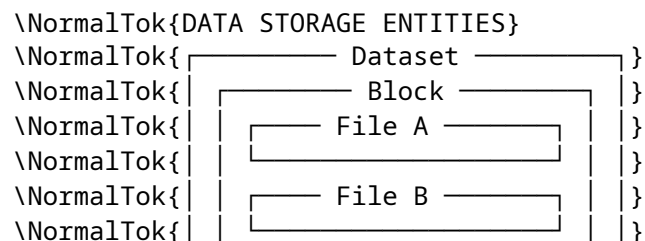
34



- **Workflow/input semantics:**



- **Orthogonal relation between storage structure and physics content.**



```

\NormalTok{ {
\NormalTok{ {
\NormalTok{ {

```

```

\NormalTok{DATA CONTENT OBJECTS}
\NormalTok{+ {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} +}
\NormalTok{. Run .}
\NormalTok{. Lumi → Event → Event .}
\NormalTok{. Lumi → Event → Event .}
\NormalTok{. ... .}
\NormalTok{+ {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} {-} +}

```

```
\NormalTok{Run spans multiple files}
\NormalTok{Files contain lumisections and events}
```

1. Storage hierarchy

Dataset → Block → File

2. Content hierarchy

Run → Lumi Section → Event

3. Key CMS property

- Data contents relation
 - **Runs cross file boundaries**
 - **Files contain lumisections**
 - **Events belong to exactly one lumisection**
 - **Luminosity sections do not cross file boundaries.**

34.0.1 Correct compact relation

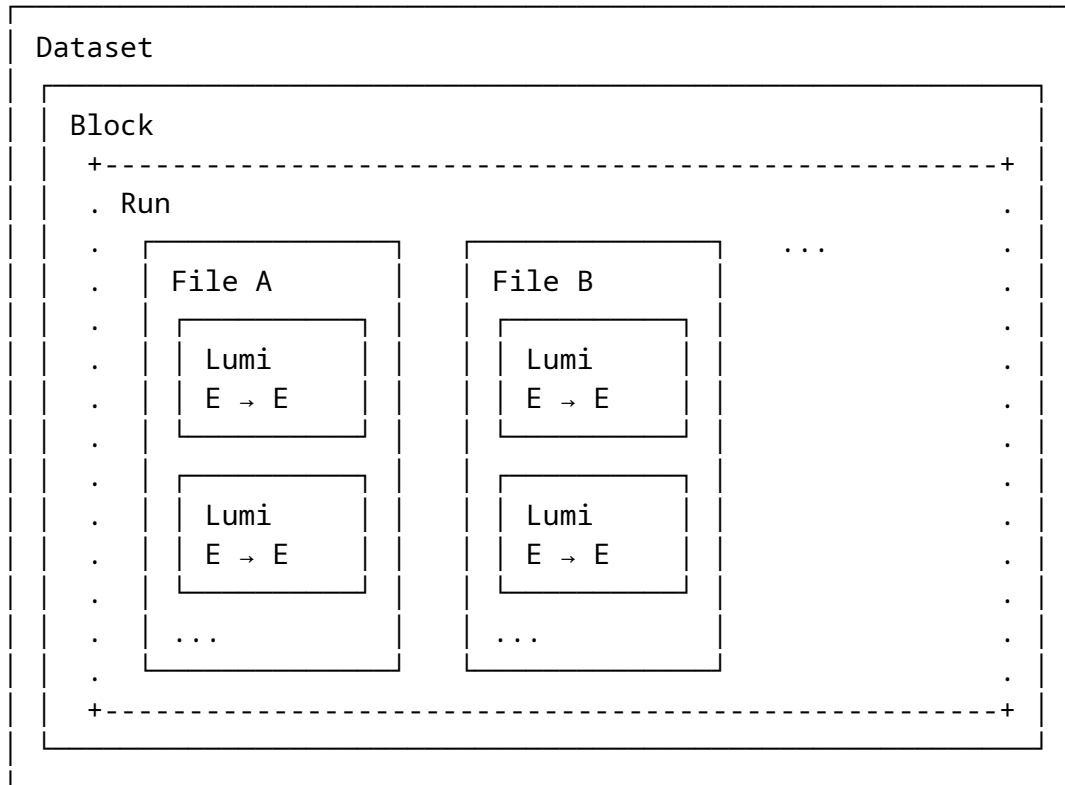
```
\NormalTok{Lumi C File}
\NormalTok{File \not C Lumi}
\NormalTok{Event C Lumi}
\NormalTok{Event C File}
```

Meaning:

- **Lumi $\subset$ File** $\rightarrow$ a lumi is fully contained in one file
- **File $\not\subset$ Lumi** $\rightarrow$ a file may contain multiple lumis
- **Event $\subset$ Lumi** $\rightarrow$ events belong to a lumi
- **Event $\subset$ File** $\rightarrow$ events are stored in files
- Run-Files relation
 - **A run can span multiple files**

- **Run boundaries are not required to coincide with file boundaries**
- **Therefore a file may contain events from multiple runs**

Run $\not\subset$ File
File $\not\subset$ Run
Event $\subset$ Run
Event $\subset$ File



35 CMS Job Splitting Algorithms

- **Complete list of CMS workflow job-splitting algorithms** used in the **WM-Core / WMAgent** splitting framework:

Algorithm	Unit	Typical use
FileBased	File	most production
LumiBased	Lumi	data reconstruction
EventBased	Event	MC
EventAwareLumiBased	Lumi	modern CMS production
RunBased	Run	rare workflows
BlockBased	Block	dataset management
DatasetBased	Dataset	special workflows

Algorithm	Unit	Typical use
ProductionEventBased	Event	MC generation
EventRangeBased	EventRange	event service

35.1 Splitting dimension

1. **canonical splitting algorithms implemented in WMCore**, and
2. **variants / configuration-driven hybrids** that appear in production workflows.

35.2 1. File-level splitting

These operate purely on **storage entities** (Files).

35.2.1 1. FileBased

Most common.

Splits jobs by **number of input files**.

Typical parameters:

files_per_job
max_files_per_job

Used for:

- MC processing
- reprocessing
- workflows where file boundaries are safe units

Properties:

job_unit = File

35.3 2. Lumi-based splitting

Operates on **lumisections**, which are **content objects inside files**.

35.3.1 2. LumiBased

Splits jobs by **number of lumisections**.

Parameters:

lumis_per_job

Used for:

- prompt reconstruction
- skims
- workflows that must preserve lumi boundaries

Properties:

`job_unit = LumiSection`

Important constraint:

`Lumi ∈ File`

35.4 3. Event-based splitting

Splits purely by **number of events processed**.

35.4.1 3. EventBased

Splits jobs by **event count**.

Parameters:

`events_per_job`

Used for:

- Monte Carlo generation
- some simulation steps

Properties:

`job_unit = Event`

35.5 4. Event-aware lumi splitting

Hybrid splitting.

35.5.1 4. EventAwareLumiBased

Splits by **lumisections**, but uses **event counts to size jobs**.

Parameters:

`events_per_job`

`max_events_per_lumi`

Purpose:

- avoid large lumis producing oversized jobs
- maintain lumi integrity

Properties:

```
job_unit = LumiSection
size_metric = EventCount
```

This is very common in CMS production.

35.6 5. Run-based splitting

Rare but supported.

35.6.1 5. RunBased

Splits by **run number**.

Used when workflows must **preserve run boundaries**.

Properties:

```
job_unit = Run
```

35.7 6. Lumi-run hybrid splitting

35.7.1 6. LumiBased with run whitelist

A configuration-driven variant.

Splits by lumis but **restricted by run masks**.

Used for:

- re-reconstruction
- partial dataset reprocessing

Properties:

```
job_unit = LumiSection
constraint = Run
```

35.8 7. Dataset-level splitting

35.8.1 7. DatasetBased

One job processes **entire dataset partitions**.

Rare.

Mostly used for:

- merge workflows
 - cleanup steps
-

35.9 8. Block-based splitting

Splits by **dataset block**.

35.9.1 8. BlockBased

Units:

`job_unit = Block`

Use cases:

- data placement
 - replication workflows
 - some large merge steps
-

35.10 9. Production-style event generation splitting

35.10.1 9. ProductionEventBased

Special variant used for MC generation.

Properties:

`events_per_job`

`events_per_lumi`

Important because **MC generation must synthesize lumisections**.

35.11 10. Event-range splitting

Used internally in some generation workflows.

35.11.1 10. EventRangeBased

Units:

`event_range`

Used in:

- event service
 - opportunistic computing
-

35.12 The three *most important* in CMS production

In practice ~95% of CMS workflows use only three:

FileBased

LumiBased

EventAwareLumiBased

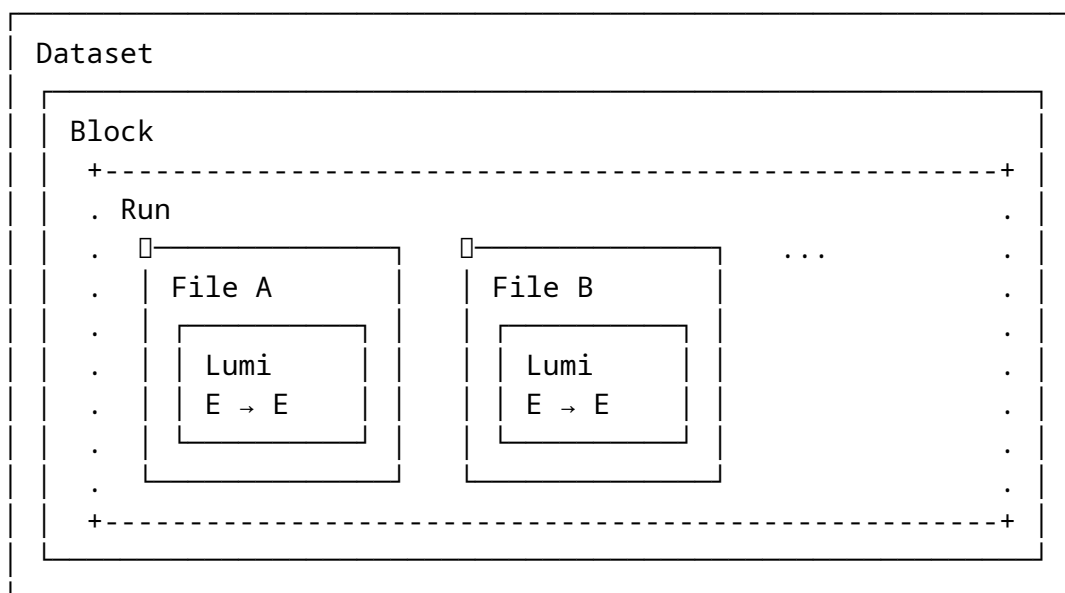
Everything else is niche.

35.13 CMS splitting visualized

□ = job split boundary

36 1. FileBased splitting

Jobs are groups of files

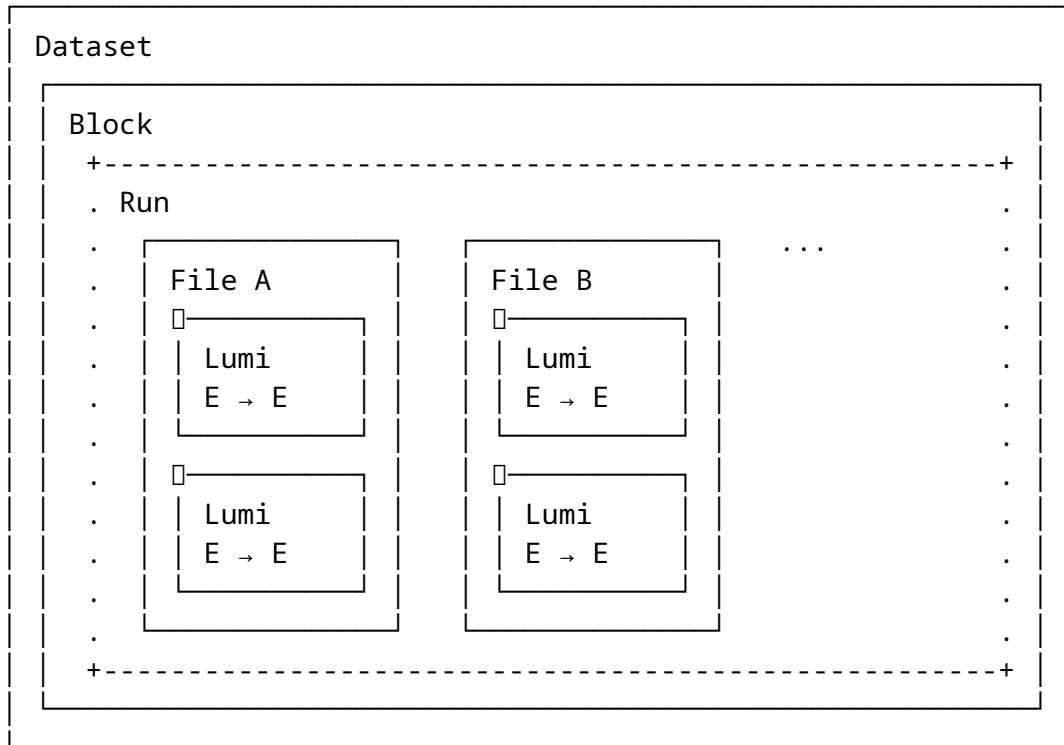


Meaning

job = N files

37 2. LumiBased splitting

Jobs split at lumisection boundaries

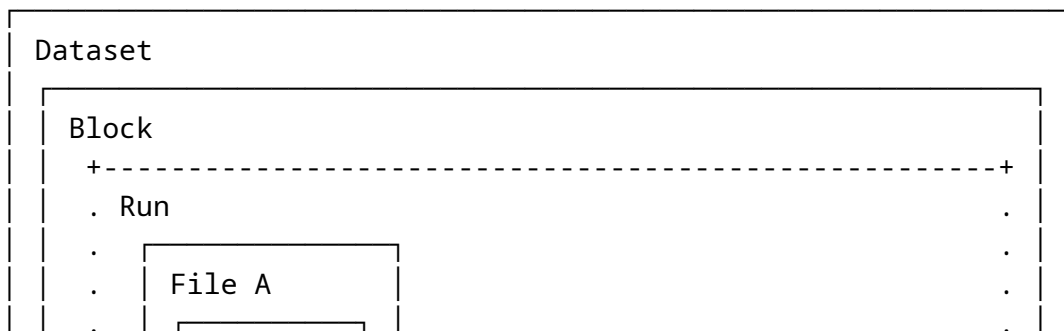


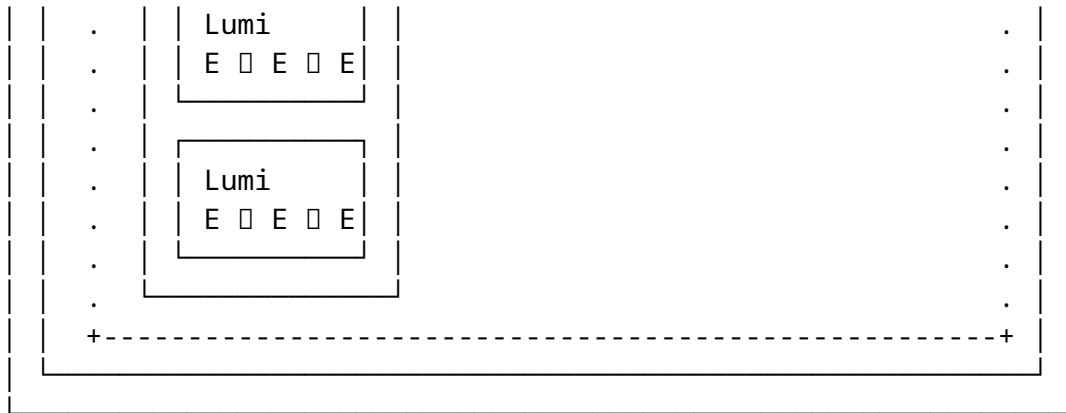
Meaning

job = N lumisections

38 3. EventBased splitting

Events themselves define the boundary



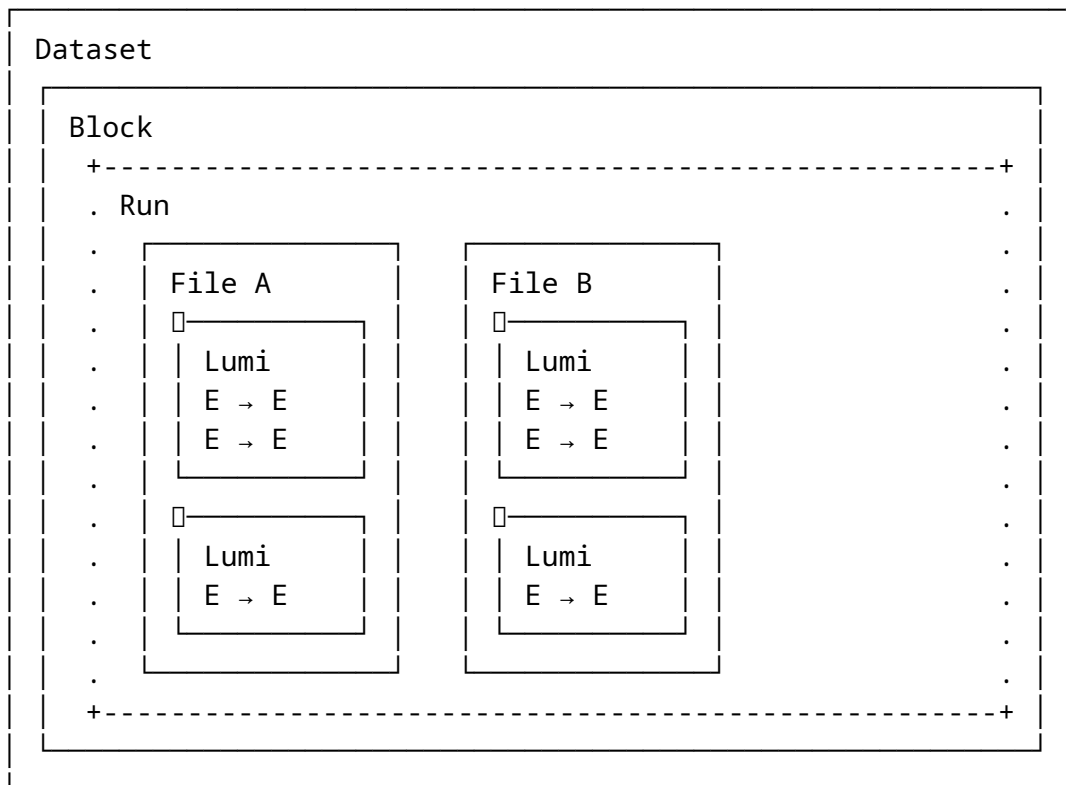


Meaning

job = N events

39 4. EventAwareLumiBased splitting

Jobs split by lumis but sized by events



Meaning

split_unit = lumi

job_size $\approx$ events_per_job

40 5. RunBased splitting (rare)

Cuts along **Run boundary**.

```
+-----+
. Run 1  .
.       .
+-----+

+-----+
. Run 2  .
.       .
+-----+
```

41 Importance for the CMS vs. DIRAC interoperability

This immediately exposes the **core mismatch** between CMS and DIRAC:

CMS splitting units:

File
Lumi
Event

But **Run spans files**.

So when translating CMS workflows to DIRAC transformations:

run constraints
must be projected onto file-level job units

—which is exactly the architectural tension your **translation layer must resolve**.

\newpage

42 CMS Workflow Splitting Model — Storage vs Content Hierarchy

42.0.1 1. CMS Data Hierarchy

CMS workflows operate on a data model that separates **storage entities** from **physics/content entities**.

Storage hierarchy

Dataset
└ Block
└ File

Datasets are logical collections of data. Files within datasets are grouped into **blocks** to support scalable data management and placement across distributed storage systems.

Content hierarchy

Run
└ Lumi Section
└ Event

- **Run** – period of detector operation with consistent configuration
- **Luminosity section (lumi)** – small time slice within a run (~23 seconds)
- **Event** – individual collision record

The two hierarchies intersect but are not identical.

Key containment relations:

File $\supset$ Lumi $\supset$ Event

However:

Run groups lumisections but is not a storage container

Runs therefore **span multiple files**, while lumisections and events remain contained within a single file.

42.0.2 2. Workflow Splitting as Partitioning

CMS workflow splitting algorithms determine how input data are partitioned into independent processing jobs.

Conceptually, splitting can be modeled as a **partition operator**:

Split(level, size)

where

Parameter	Meaning
level	hierarchy level at which partitioning occurs
size	job sizing metric

The splitting algorithm determines which objects are grouped into each job.

42.0.3 3. Confirmed CMS Splitting Algorithms

The commonly used CMS workflow splitting modes are:

Algorithm	Partition Level	Job Size Metric
FileBased	File	files per job
LumiBased	Lumisection	lumisections per job
EventAwareLumiBased	Lumisection	events per job
EventBased	Event	events per job (primarily PrivateMC)

The first three are the dominant modes used in CMS production and analysis workflows.

42.0.4 4. Example Partitions

42.0.4.1 FileBased

```
Jobs = partition(Files, files_per_job)
```

Example:

```
[FileA FileB] → Job1  
[FileC FileD] → Job2
```

Each job processes a fixed number of input files.

42.0.4.2 LumiBased

```
Jobs = partition(Lumis, lumis_per_job)
```

Example:

```
[L1 L2 L3] → Job1  
[L4 L5 L6] → Job2
```

Because lumisections are contained within files, file boundaries must still be respected.

42.0.4.3 EventBased

```
Jobs = partition(Events, events_per_job)
```

Jobs process a specified number of events regardless of lumisection boundaries. This mode is primarily used in Monte-Carlo generation workflows.

42.0.4.4 EventAwareLumiBased This hybrid algorithm partitions by lumisection but sizes jobs according to the total number of events.

```
Jobs = partition(Lumis)
subject to
```

```
 $\sum \text{events}(\text{lumi\_i}) \approx \text{events\_per\_job}$ 
```

Thus:

```
split_unit = lumi
job_size_metric = events
```

Each job still processes **complete lumisections**, but the grouping is determined by event counts.

42.0.5 5. Projection onto File-Based Execution

Although splitting may be defined at the **lumi or event level**, actual execution requires **file inputs**.

Therefore each job description must be expressed as a set of files containing the required content.

Conceptually this is a **projection problem**:

Projection:

Split(Level X) → File-based job description

Examples:

LumiBased

job files = files containing the selected lumisections

EventBased

job files = files containing the selected events

EventAwareLumiBased

job files = files containing the selected lumisections
job size determined by summed event counts

42.0.6 6. Translation IR Representation

A useful intermediate representation for workflow translation can therefore be expressed as:

```
Job = {  
  files,  
  lumis,  
  event_range  
}
```

Where:

- files represent the physical execution input
- lumis and event_range act as selection constraints

This representation cleanly separates **storage objects** from **content filters**.

42.0.7 7. Minimal Formal Model

CMS splitting can be normalized into a three-parameter abstraction:

Split(level, weight, boundary)

Parameter	Meaning
level	hierarchy level of the split unit
weight	metric used to size jobs
boundary	containment constraint

Example mappings:

Algorithm	level	weight	boundary
FileBased	File	files	file
LumiBased	Lumi	lumis	file
EventAwareLumiBased	Lumi	events	file
EventBased	Event	events	lumi

42.0.8 8. Structural Insight

The hierarchy implies the following relationships:

File $\supset$ Lumi $\supset$ Event

Run $\supset$ Lumi

Consequences:

- File-level splitting is always compatible with storage organization.
 - Lumi and event splitting require projection onto files.
 - Run boundaries do not align with storage units.
-

42.0.9 9. Implications for Workflow Translation

Any system translating CMS workflows to another execution model must therefore handle the mapping:

content-level splitting

↓

file-level execution units

The translation layer must project run/lumi/event constraints onto the files that contain the corresponding data.

This separation between **storage hierarchy** and **content hierarchy** is the key architectural constraint governing CMS workflow splitting.

\newpage

\newpage

43 DIRAC Workload Management System

43.1 Purpose of this section

The **DIRAC Workload Management System (WMS)** is responsible for scheduling and executing computational workloads across distributed computing resources.

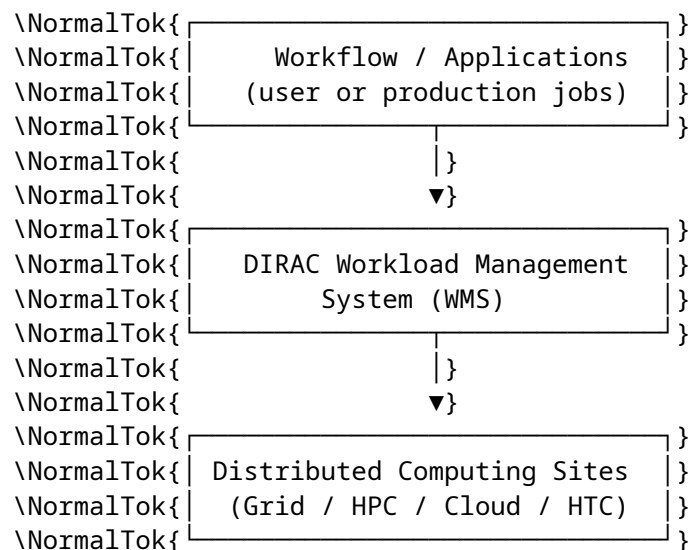
The DIRAC WMS implements a **pilot-job based scheduling model**, where resources are acquired first and the actual workload is assigned later. This approach improves reliability and efficiency when operating on heterogeneous and unreliable distributed resources such as grid infrastructures. (dirac.diracgrid.org)

Understanding the WMS architecture is essential for analyzing how workflows defined outside DIRAC (such as CMS workflows) can be executed within a DIRAC-based infrastructure.

44 Role of the Workload Management System

Within the DIRAC architecture, the Workload Management System orchestrates the lifecycle of distributed jobs.

Conceptually the system sits between the user workflow layer and the computing infrastructure.

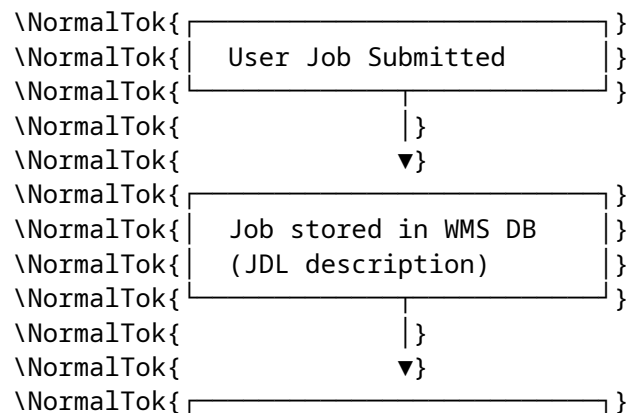


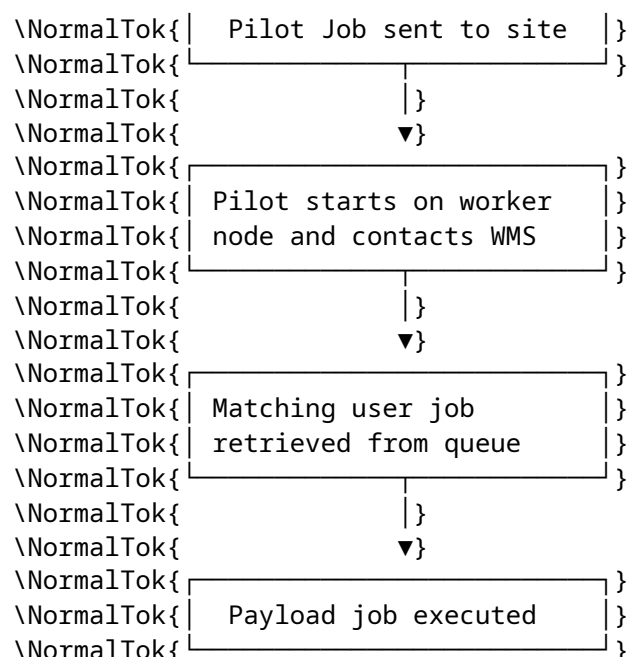
The WMS coordinates job submission, scheduling, monitoring, and execution across multiple computing environments.

45 Core Scheduling Paradigm: Pilot Jobs

A defining feature of DIRAC WMS is the **pilot job model**.

Instead of directly submitting user jobs to computing sites, the system first deploys pilot jobs that reserve resources. The actual workload is then matched to these resources dynamically. (ResearchGate)

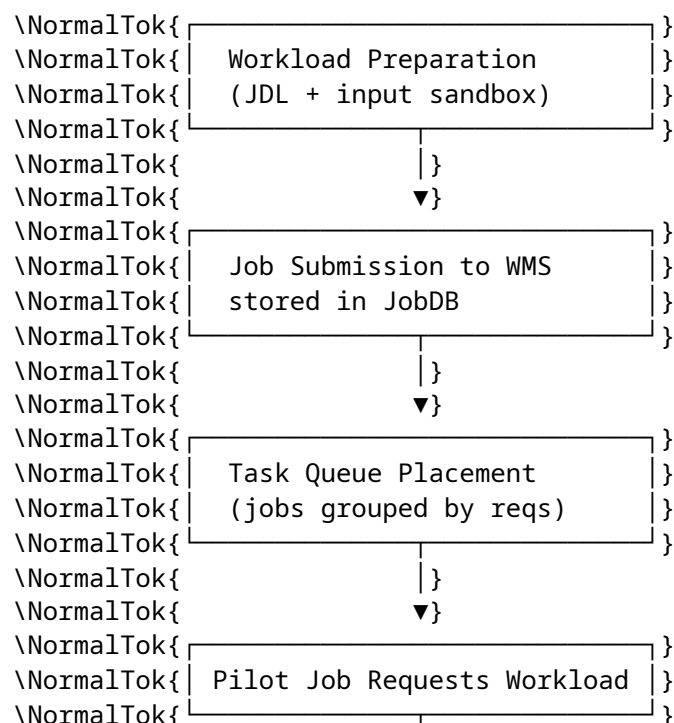


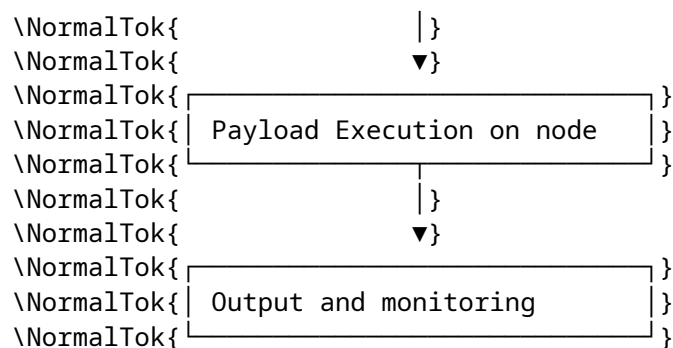


This model implements a **pull scheduling mechanism**, where compute resources pull workloads when they are ready to execute them.

46 Job Lifecycle in DIRAC WMS

The typical lifecycle of a workload in the DIRAC WMS includes several stages.

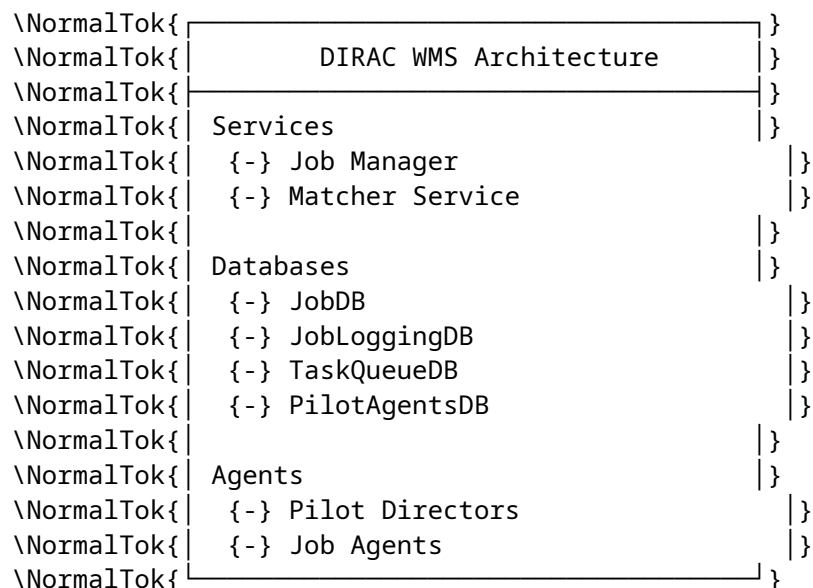




This lifecycle ensures that jobs are scheduled only when suitable resources are available.

47 Internal Architecture of the WMS

The WMS is implemented as a distributed system composed of several cooperating components.



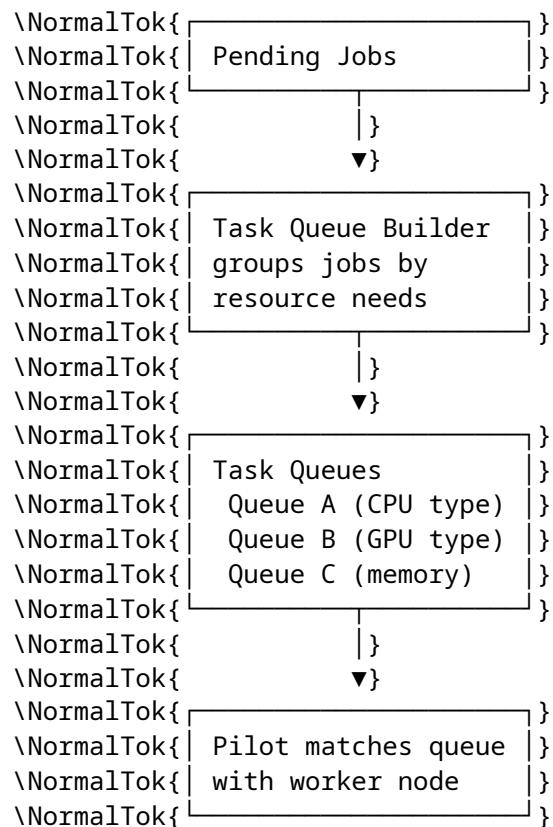
DIRAC systems typically include several component types:

- **Services** that respond to requests and provide system functionality
- **Agents** that perform background tasks
- **Databases** storing job state and metadata (dirac.diracgrid.org)

48 Task Queues

A central concept in the WMS scheduling mechanism is the **task queue**.

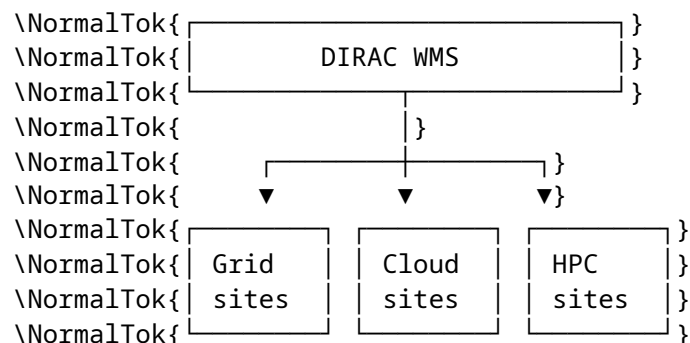
Jobs with similar requirements are grouped together, allowing efficient matching between jobs and available resources.



This mechanism enables efficient scheduling across heterogeneous infrastructures.

49 Resource Integration

The DIRAC WMS can integrate multiple resource types.



This abstraction allows the same workflow to run across different computing environments.

50 Monitoring and Bookkeeping

The WMS maintains detailed information about job execution.

Important data stored by the system include:

- job definitions
- job states
- execution logs
- resource usage

Key databases include:

```
\NormalTok{JobDB}  
\NormalTok{JobLoggingDB}  
\NormalTok{TaskQueueDB}  
\NormalTok{PilotAgentsDB}
```

These databases maintain the persistent state of the system. (dirac.diracgrid.org)

51 Comparison with CMS Workflow Scheduling

The DIRAC Workload Management System differs significantly from the CMS workflow management approach.

Property	CMS WMCORE	DIRAC WMS
Job generation	predefined by splitting	dynamic via pilots
Scheduling	push model	pull model
Data abstraction	dataset / run / lumi	file-oriented
Bookkeeping	WMBS	WMS task queues

The **pilot-based pull scheduling model** used in DIRAC allows resources to request jobs dynamically rather than being assigned workloads in advance.

52 Implications for CMSDiracAux

The CMSDiracAux project aims to translate CMS workflows into structures compatible with DIRAC execution.

```

\NormalTok{ }
\NormalTok{ CMS Workflow (WMCore) }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ Translation IR }
\NormalTok{ workflow abstraction }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ }
\NormalTok{ DIRAC WMS execution }
\NormalTok{ via pilots and queues }
\NormalTok{ }

```

The DIRAC Workload Management System provides a scalable infrastructure for scheduling distributed workloads using a pilot-job based architecture.

- late binding of jobs to resources
- dynamic job scheduling
- efficient use of heterogeneous infrastructures

\newpage

54.1 Purpose of this section

The three subsystems most relevant for workflow execution are:

- **Production System**
- **Transformation System**
- **Workload Management System (WMS)**

These systems operate at **different abstraction layers** within the DIRAC architecture and collectively provide the functionality required to execute large-scale distributed workflows.

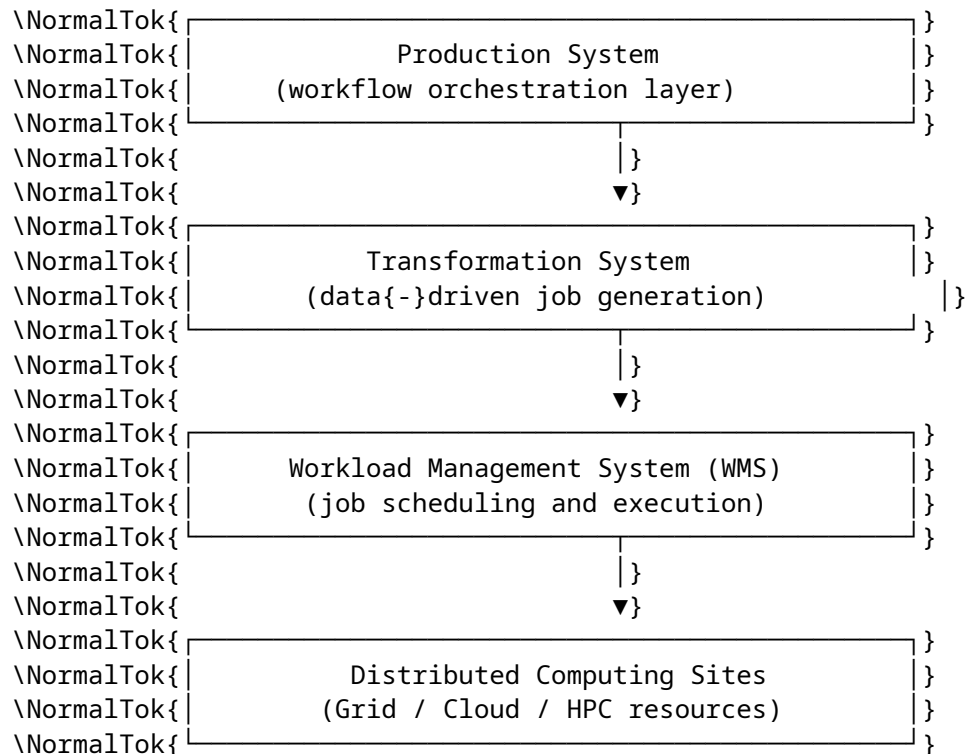
Understanding their architectural relationship is essential for evaluating how external workflow systems—such as CMS WMCore—can be expressed and executed within the DIRAC environment.

55 DIRAC Architectural Context

DIRAC is designed as a **service-oriented distributed computing framework** composed of loosely coupled services and agents that manage workload and data operations. (dirac.diracgrid.org)

At a high level, the system integrates computing and storage resources into a unified infrastructure for executing distributed workloads. (Proceedings of Science)

The workflow-related components can be organized into layered abstractions:



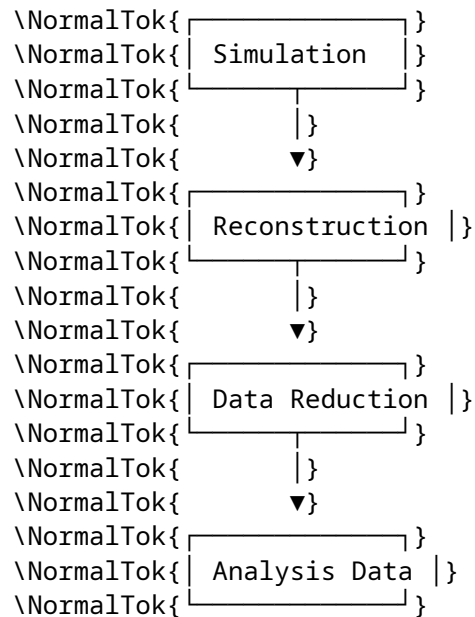
Each subsystem performs a different role in transforming a **workflow description into executed jobs**.

56 The Production System: Workflow Orchestration

The **DIRAC Production System** operates at the highest level of abstraction.

It manages **production campaigns**, which typically represent large multi-step processing workflows.

A production workflow usually contains multiple stages:



The Production System therefore provides:

- workflow orchestration
- production bookkeeping
- coordination of multi-step processing chains

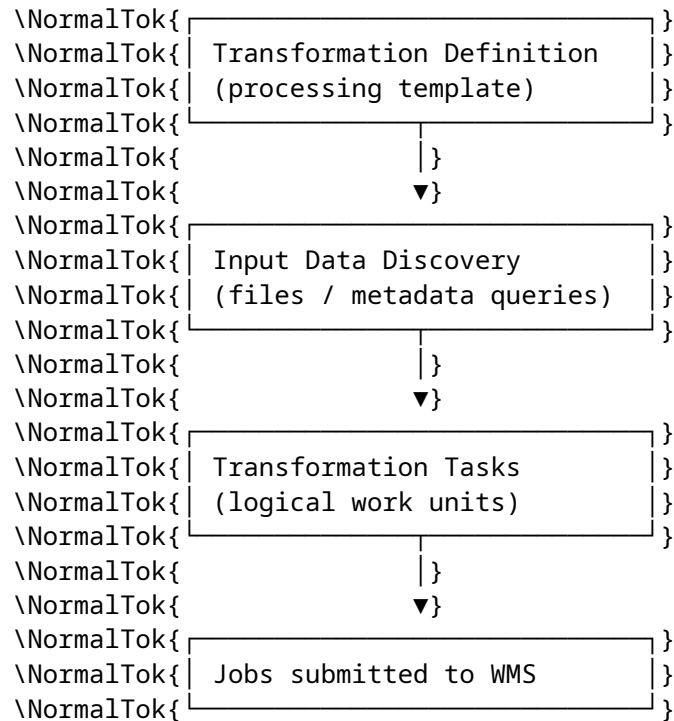
In many DIRAC deployments, production workflows internally use the **Transformation System** to generate the jobs required for each processing step. (Indico)

57 The Transformation System: Data-Driven Job Generation

The **Transformation System** acts as the bridge between workflow descriptions and job execution.

Its main responsibility is to generate **large numbers of tasks and jobs based on input data.**

Typical workflow:



A transformation therefore represents **repetitive work applied to many input data elements**, typically files.

Plugins determine how tasks are generated, for example:

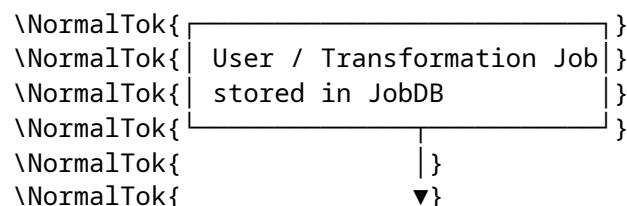
- how many files are processed per job
- how jobs are partitioned
- where jobs should run. (Indico)

This layer converts a **logical workflow step** into a set of executable jobs.

58 The Workload Management System: Job Scheduling

The **Workload Management System (WMS)** is responsible for executing jobs generated by the Transformation System.

DIRAC WMS uses a **pilot-job scheduling paradigm**, where computing resources pull workloads from the central system. (dirac.diracgrid.org)




```

\NormalTok{          |}
\NormalTok{          ▼}
\NormalTok{          }
\NormalTok{ Distributed computing sites }
\NormalTok{          }

```

Each layer transforms the workflow description into a more concrete execution form.

60 Workflow Abstraction Levels

These systems correspond to **three levels of workflow abstraction**.

Layer	Workflow abstraction	Example concept
Production System	high-level workflow	processing campaign
Transformation System	data processing task	transformation
Workload Management System	executable job	pilot-executed job

Thus the pipeline can be interpreted as a sequence of **abstraction reductions**:

```

\NormalTok{Workflow description}
\NormalTok{      ↓}
\NormalTok{Processing campaign}
\NormalTok{      ↓}
\NormalTok{Transformation tasks}
\NormalTok{      ↓}
\NormalTok{Executable jobs}

```

61 Architectural Implications

The DIRAC architecture separates:

- **workflow description**
- **job generation**
- **job execution**

This separation allows the system to scale efficiently across large distributed infrastructures.

However, this layered abstraction also means that DIRAC workflows typically operate on **file-level processing units**, whereas some external workflow systems may require finer-grained control over data.

62 Implications for CMSDiracAux

The CMSTDiracAux project must bridge two workflow paradigms:

CMS workflow model

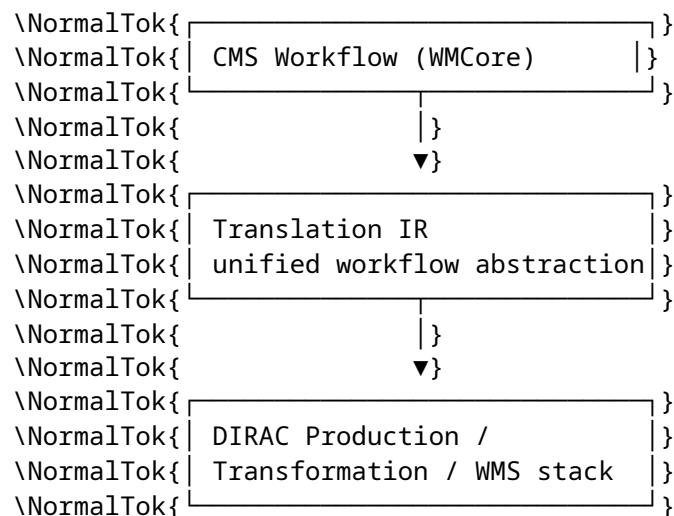
dataset → run → lumi → event

DIRAC workflow model

production → transformation → job

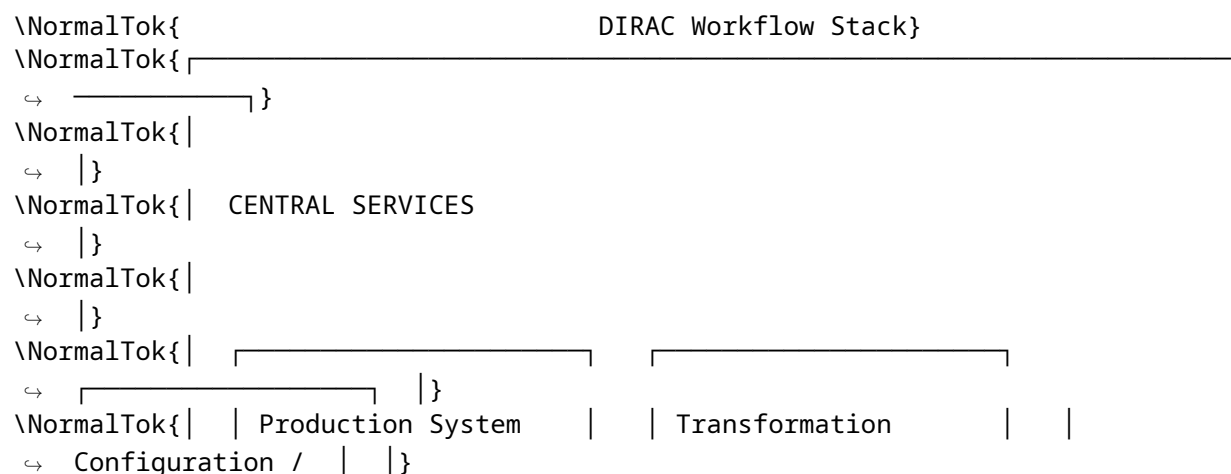
The **Translation IR** therefore acts as a compatibility layer between these models.

Conceptually:



This translation enables CMS workflows to be executed within the DIRAC infrastructure while preserving workflow semantics.

63 Detailed architecture



[illegible]

workflows are structured in DIRAC and how they compare with CMS workflow management systems.

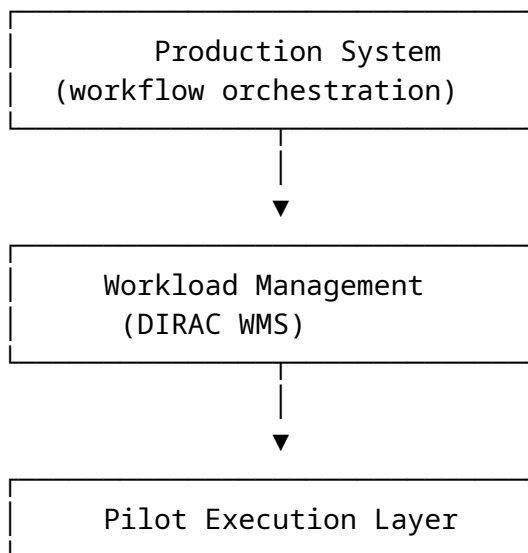
66 Role of the Production System in DIRAC

The Production System was originally designed to support the large production campaigns of the LHCb experiment.

It provides functionality for:

- defining production workflows
- orchestrating multi-step processing chains
- tracking production status
- managing large sets of related jobs

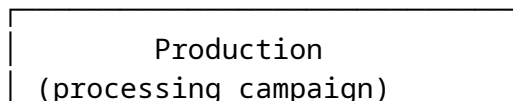
Conceptually, the Production System sits above the workload management layer.

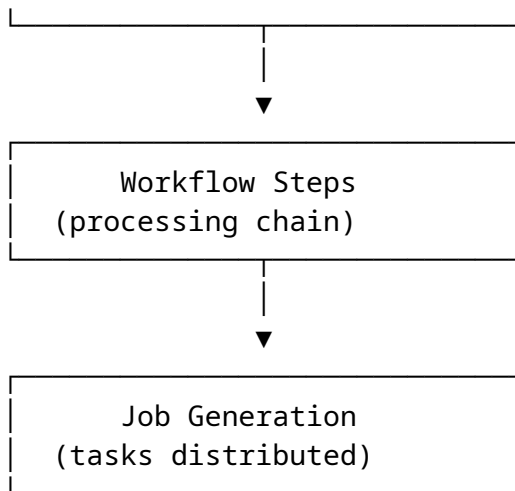


67 Concept of Productions

A **production** represents a coordinated set of jobs that process large volumes of data according to a defined workflow.

Conceptually:





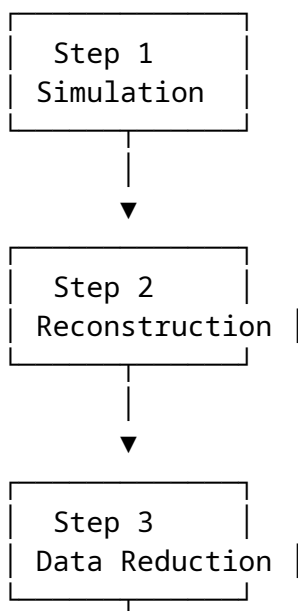
A production may involve multiple stages such as:

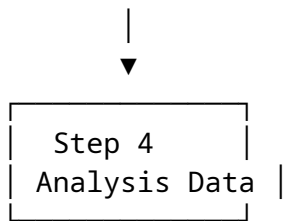
- simulation
- reconstruction
- data reduction
- analysis preparation

Each stage produces output data used by the next stage.

68 Production Workflow Structure

Production workflows in DIRAC are typically represented as **sequences of processing steps**.



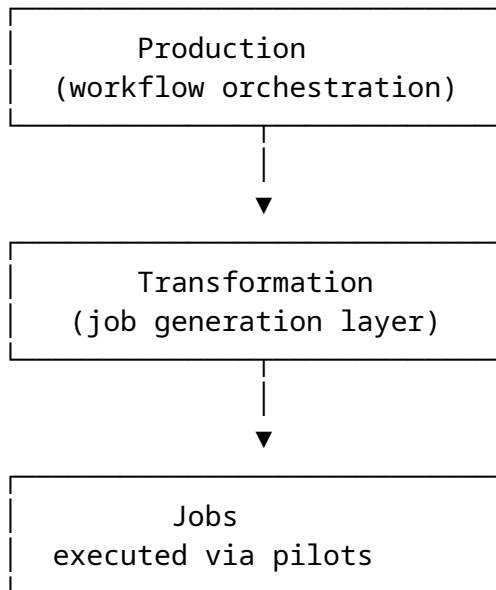


Each step may generate thousands of distributed jobs.

69 Relationship Between Productions and Transformations

In many DIRAC deployments, productions are implemented internally using **transformations**.

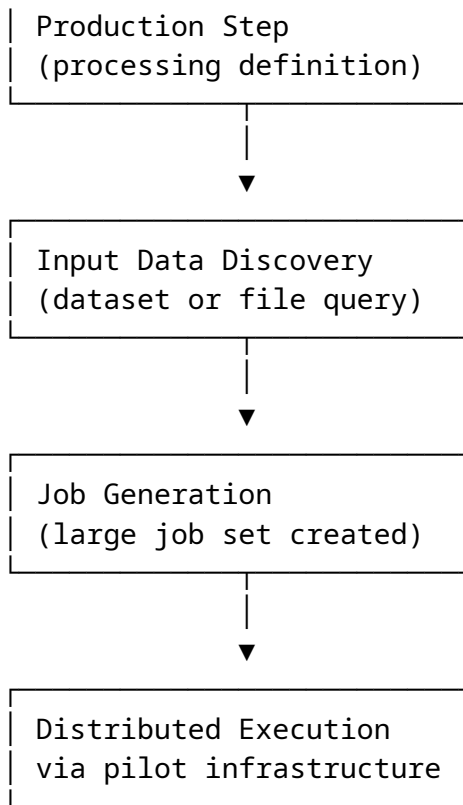
Conceptually:



Thus the Transformation System often acts as the **execution engine** for production workflows.

70 Job Creation within a Production

Within a production step, the system generates jobs that process input data.



Each production may generate **tens or hundreds of thousands of jobs**.

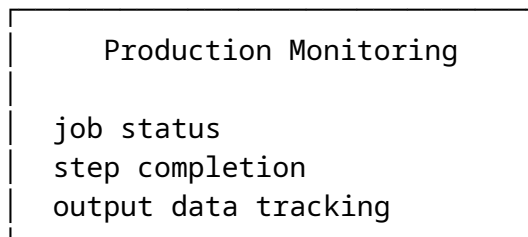
71 Monitoring and Bookkeeping

The Production System includes services for tracking workflow progress.

These services maintain information about:

- job status
- data produced
- workflow step completion
- production progress

Conceptually:



This bookkeeping functionality is critical for large production campaigns.

72 Comparison with CMS Workflow Management

The DIRAC Production System differs from CMS workflow management in several important ways.

Property	CMS Workflow System	DIRAC Production System
Workflow definition	explicit multi-step workflow	production steps
Job generation	predefined during splitting	generated dynamically
Data abstraction	dataset + run/lumi splitting	file-level processing
Bookkeeping	WMBS	production monitoring

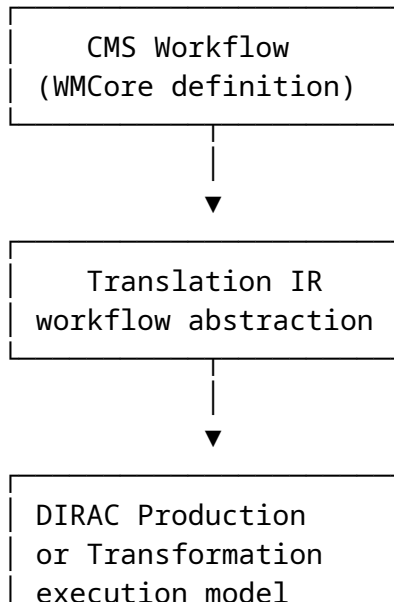
These differences highlight the conceptual gap between CMS workflows and DIRAC execution models.

73 Implications for CMSDiracAux

The CMSDiracAux project aims to express CMS workflows within DIRAC infrastructure.

This requires mapping CMS workflow concepts onto DIRAC constructs.

Conceptually:



The Translation IR acts as the layer that reconciles the different workflow abstractions used by the two systems.

74 Summary

The DIRAC Production System provides high-level orchestration for large-scale distributed processing campaigns.

It manages multi-step workflows and coordinates the generation and monitoring of large numbers of distributed jobs.

In modern DIRAC deployments, the Production System often relies on the Transformation System to generate and execute individual jobs.

Understanding the relationship between these systems is essential for integrating CMS workflows within DIRAC infrastructures, as explored by the CMSDiracAux project.

\newpage

75 DIRAC Transformation System

75.1 Purpose of this section

The DIRAC Transformation System provides a framework for executing large-scale distributed workloads across heterogeneous computing resources.

Unlike the CMS workflow system, which constructs jobs based on predefined splitting of datasets, the DIRAC Transformation System operates using **dynamic job generation driven by available input data**.

Understanding the DIRAC Transformation System is essential for evaluating how CMS workflows can be executed within a DIRAC-based infrastructure.

76 DIRAC Architecture Context

DIRAC (Distributed Infrastructure with Remote Agent Control) is a distributed computing framework originally developed by the LHCb experiment and later adopted by other experiments and communities.

DIRAC provides services for:

- workload management

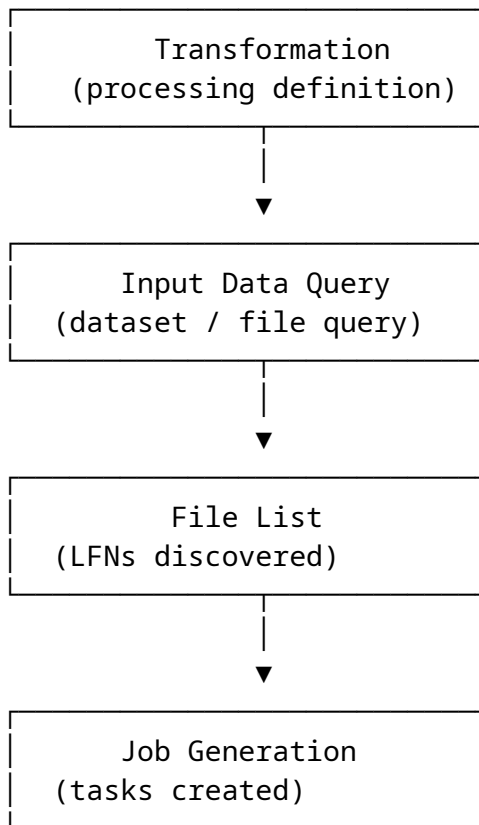
- data management
- distributed resource integration
- pilot-based job execution

Within the workload management layer, the **Transformation System** is responsible for orchestrating large production campaigns.

77 Concept of Transformations

A **transformation** in DIRAC represents a high-level description of a processing task that must be executed over a set of input data.

Conceptually:



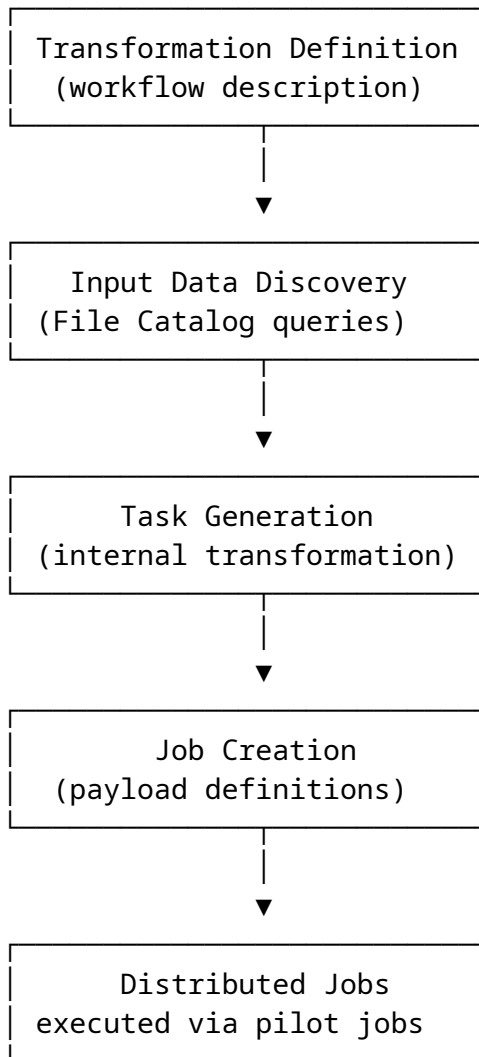
The transformation defines:

- the executable application
- input data selection
- job parameters
- output data handling

Jobs are then generated dynamically based on the discovered input data.

78 Transformation Lifecycle

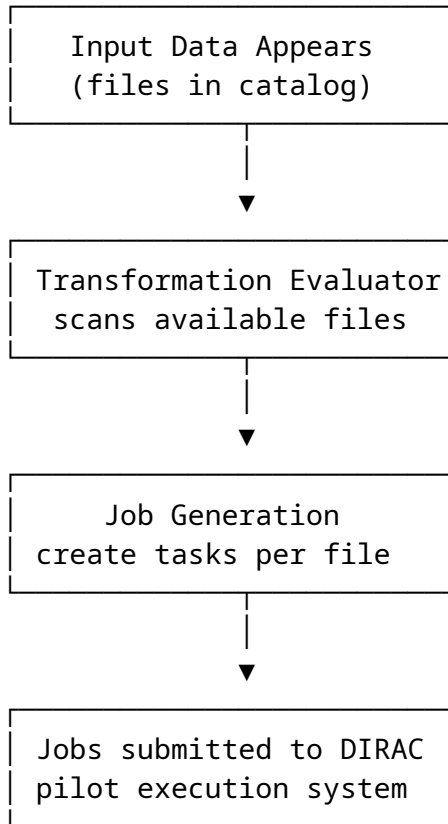
The lifecycle of a DIRAC transformation typically follows the sequence below.



Each transformation therefore results in **large numbers of jobs** executed across distributed computing sites.

79 Data-Driven Job Generation

A defining property of the DIRAC Transformation System is that job generation is **driven by available input data**.

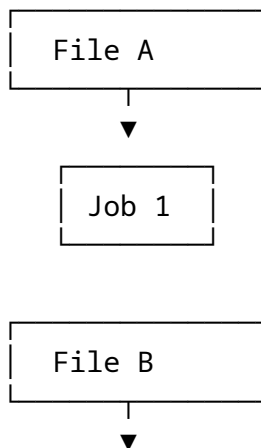


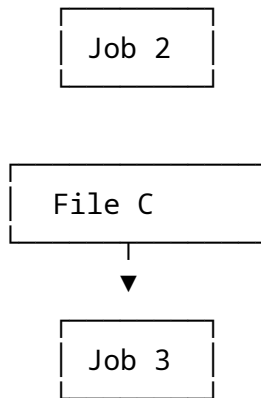
This dynamic behaviour contrasts with workflow systems that precompute all jobs before execution.

80 File-Level Processing Model

DIRAC transformations generally operate on **file-level input units**.

Typical mapping between files and jobs:



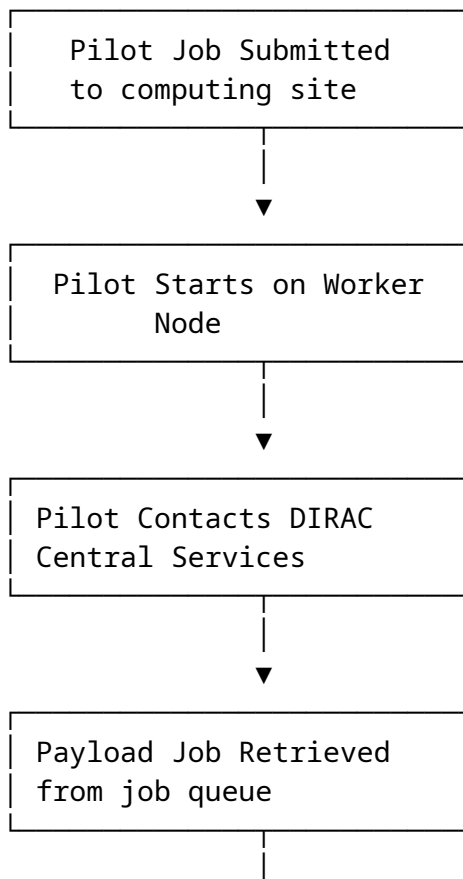


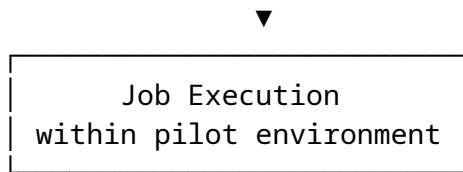
This model assumes that files represent reasonable units of work.

However, it implicitly assumes **uniform file workloads**, which is not always true for CMS data.

81 Pilot Job Execution Model

DIRAC employs a **pilot job model** for workload execution.





This mechanism improves resource utilization by separating **resource acquisition** from **job scheduling**.

82 Job Execution Description

Jobs executed by DIRAC are typically described using the **Job Description Language (JDL)**.

The JDL specifies:

- executable
- runtime arguments
- input sandbox
- output sandbox
- environment configuration

The actual execution workflow may also be described using:

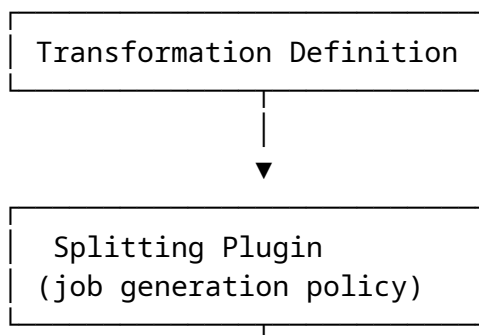
`jobDescription.xml`

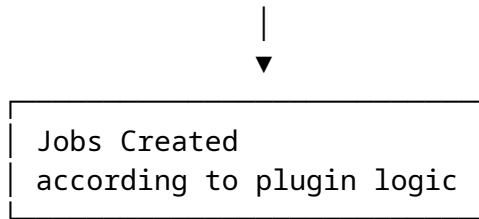
which defines the internal job workflow executed by the worker node.

83 Transformation Plugins

The Transformation System supports **plugins** responsible for defining how input data is partitioned into jobs.

Conceptually:





These plugins determine:

- job granularity
- data partitioning
- runtime parameterization

The plugin architecture is particularly relevant for implementing CMS-specific splitting logic.

84 Comparison with CMS Workflow Execution

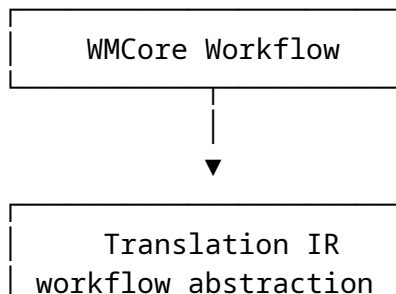
The DIRAC Transformation System differs fundamentally from the CMS workflow model.

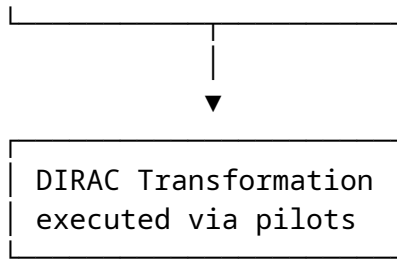
Property	CMS WMCore	DIRAC
Job generation	predefined during workflow preparation	dynamic based on data availability
Data abstraction	dataset oriented	file oriented
Splitting granularity	run / lumi / event possible	typically file level
Bookkeeping	WMBS	transformation tasks

85 Implications for CMSDiracAux

The CMSDiracAux project introduces a **Translation IR** that bridges the conceptual gap between the CMS workflow model and the DIRAC Transformation System.

The translation process can be summarized as:





This architecture allows CMS workflows to be executed within DIRAC while preserving workflow semantics.

86 Summary

The DIRAC Transformation System provides a flexible infrastructure for executing large-scale distributed workloads through **dynamic job generation and pilot-based execution**.

However, its **file-oriented processing model** differs from the fine-grained data abstractions used in CMS workflows.

Bridging this difference requires an intermediate abstraction layer, implemented in CMSDiracAux as the **Translation IR**.

\newpage

87 Changes with DiracX

This is a **concise architectural explanation of what changes with DIRACX**, focusing on **how the workflow stack evolves** relative to the classic DIRAC architecture.

Production
Transformation
Workload Management

88 1. Core Idea of DIRACX

DIRACX is **not just a new version of DIRAC**.

It is a **re-architecture of the control plane** intended to:

- modernize the system
- simplify deployment
- support cloud-native infrastructures

microservice-based control plane

workflow descriptions

↓

workflow services

Workflows become easier to represent as:

- DAGs
- CWL workflows
- declarative task graphs

This is why DIRACX aligns well with **CWL-style workflows**.

92 5. Workload Management Changes

The classic DIRAC WMS tightly integrates:

- job queues
- matching
- pilot scheduling
- job states

DIRACX separates these responsibilities.

Conceptually:

task scheduling service

|

▼

resource matching service

|

▼

pilot services

This allows:

- better scaling
 - more flexible scheduling policies
 - cloud-native resource integration.
-

93 6. Control Plane vs Execution Plane

DIRACX clearly separates:

CONTROL PLANE

(workflow / scheduling)

EXECUTION PLANE

(pilots / jobs)

```
\NormalTok{CONTROL PLANE}
\NormalTok{
\NormalTok{
\NormalTok{ Workflow Services }
\NormalTok{ Transformation APIs }
\NormalTok{ Scheduling Services }
\NormalTok{
\NormalTok{ | }
\NormalTok{ ▼ }
\NormalTok{EXECUTION PLANE}
\NormalTok{
\NormalTok{ Pilot Infrastructure }
\NormalTok{ Worker nodes }
\NormalTok{ }
```

Classic DIRAC mixes these concerns more tightly.

94 7. Impact on Workflow Abstractions

DIRACX introduces **clearer workflow abstractions**.

Classic DIRAC:

```
production
  ↓
transformation
  ↓
jobs
```

DIRACX moves toward:

```
workflow
  ↓
tasks
  ↓
jobs
```

This model maps more naturally to:

- workflow DAGs
 - CWL
 - modern workflow engines.
-

95 8. Why This Matters for CMSDiracAux

DIRACX architecture is actually **much closer to the CMS workflow model** than classic DIRAC.

CMS:

workflow

↓

tasks

↓

jobs

DIRACX:

workflow

↓

tasks

↓

jobs

Classic DIRAC:

production

↓

transformation

↓

jobs

So the **Translation IR layer you designed aligns extremely well with DIRACX**.

96 9. Key Architectural Differences

Feature	Classic DIRAC	DIRACX
Architecture	monolithic services	microservices
Interfaces	RPC services	REST APIs
Workflow abstraction	production / transformation	workflow / tasks
Scheduling	integrated WMS	modular services
Infrastructure	grid-centric	cloud / grid / hybrid

97 10. The Big Picture

The transition looks like this:

```
\NormalTok{Classic DIRAC}
```

```
\NormalTok{Production}  
\NormalTok{\downarrow}  
\NormalTok{Transformation}  
\NormalTok{\downarrow}  
\NormalTok{WMS}  
\NormalTok{\downarrow}  
\NormalTok{Pilots}
```

becomes

```
\NormalTok{DIRACX}  
  
\NormalTok{Workflow}  
\NormalTok{\downarrow}  
\NormalTok{Task services}  
\NormalTok{\downarrow}  
\NormalTok{Scheduling services}  
\NormalTok{\downarrow}  
\NormalTok{Pilots}
```

\newpage

98 Architectural Comparison: CMS Workflow System, DIRAC, and DIRACX

98.1 Purpose

This section compares the architectural organization of three workflow execution models:

- the **CMS Workflow Management System (WMCore)**
- the **classic DIRAC workflow stack**
- the **DIRACX architecture**

The comparison focuses on how each system transforms **workflow descriptions into executable jobs** and how the abstraction layers differ.

Understanding these differences is important for evaluating the design of interoperability layers such as the **Translation IR introduced in the CMSDiracAux project**.

99 Workflow Architecture Comparison

The diagram below illustrates the conceptual workflow pipeline implemented by each system.

\NormalTok{		
↪	}	
\NormalTok{	CMS WMCore	DIRAC (classic)
↪	DIRACX	}
\NormalTok{		
↪	}	
\NormalTok{		
↪	}	
\NormalTok{	WORKFLOW DEFINITION	PRODUCTION SYSTEM
↪	WORKFLOW SERVICES	}
\NormalTok{	(request / workflow object)	(processing campaigns)
↪	(workflow APIs)	}
\NormalTok{		
↪	}	
\NormalTok{		
↪	}	
\NormalTok{	▼	▼
↪	▼	}
\NormalTok{	TASK STRUCTURE	TRANSFORMATION SYSTEM
↪	TASK SERVICES	}
\NormalTok{	(task tree / processing steps)	(data{-}driven job
↪	generation) (workflow task graph)	}
\NormalTok{		
↪	}	
\NormalTok{		
↪	}	
\NormalTok{	▼	▼
↪	▼	}
\NormalTok{	WMBS	WORKLOAD MANAGEMENT SYSTEM
↪	SCHEDULING SERVICES	}
\NormalTok{	(dataset discovery	(job queues / matching)
↪	(resource matchmaking)	}
\NormalTok{	+ splitting + bookkeeping)	
↪	}	
\NormalTok{		
↪	}	
\NormalTok{		
↪	}	
\NormalTok{	▼	▼
↪	▼	}
\NormalTok{	JOB GENERATION	PILOT DIRECTORS
↪	PILOT SERVICES	}
\NormalTok{	(run/lumi/event jobs)	(pilot submission)
↪	(pilot orchestration)	}
\NormalTok{		
↪	}	

\NormalTok{			
↪		}	
\NormalTok{	▼		▼
↪	▼	}	
\NormalTok{	SUBMISSION INFRASTRUCTURE		PILOT INFRASTRUCTURE
↪	EXECUTION PLANE	}	
\NormalTok{	(grid / batch systems)		(grid / batch)
↪	(grid / cloud / HPC)	}	
\NormalTok{			
↪		}	
\NormalTok{			
↪		}	
\NormalTok{	▼		▼
↪	▼	}	
\NormalTok{	WORKER NODES		WORKER NODES
↪	WORKER NODES	}	
\NormalTok{	(CMS payload jobs)		(pilot executes payload)
↪	(pilot executes payload)	}	
\NormalTok{			
↪		}	
\NormalTok{			
↪	}		

Each column represents a complete workflow processing stack from workflow definition to execution.

100 Workflow Abstraction Layers

Abstraction Layers

CMS WMCore	: Workflow → Tasks → Jobs
DIRAC (classic)	: Production → Transformation → Jobs
DIRACX	: Workflow → Tasks → Jobs

Scheduling Model

CMS WMCore	: push scheduling
DIRAC (classic)	: pilot pull scheduling
DIRACX	: pilot pull scheduling (decoupled services)

Data Granularity

CMS WMCore	: dataset → run → lumi → event
DIRAC (classic)	: file-oriented processing
DIRACX	: file-oriented (but workflow abstractions available)

Key Structural Difference

CMS

workflow semantics defined BEFORE execution

DIRAC

jobs generated dynamically from data

DIRACX

workflow services define tasks,
but execution remains pilot-driven

The three systems organize workflow abstractions differently.

```
\NormalTok{CMS WMCore}  
\NormalTok{Workflow}  
\NormalTok{    ↓}  
\NormalTok{Tasks}  
\NormalTok{    ↓}  
\NormalTok{Jobs}
```

```
\NormalTok{DIRAC (classic)}  
\NormalTok{Production}  
\NormalTok{    ↓}  
\NormalTok{Transformation}  
\NormalTok{    ↓}  
\NormalTok{Jobs}
```

```
\NormalTok{DIRACX}  
\NormalTok{Workflow}  
\NormalTok{    ↓}  
\NormalTok{Tasks}  
\NormalTok{    ↓}  
\NormalTok{Jobs}
```

The CMS and DIRACX models expose workflow tasks explicitly, whereas classic DIRAC expresses workflows through production and transformation constructs.

101 Scheduling Model

The systems also differ in how jobs are scheduled and dispatched.

```
\NormalTok{CMS WMCore          : push scheduling}  
\NormalTok{DIRAC (classic)       : pilot pull scheduling}  
\NormalTok{DIRACX                : pilot pull scheduling with decoupled scheduling  
↪ services}
```

In the CMS system, jobs are generated and submitted explicitly to computing resources. In DIRAC systems, resources obtain work dynamically through pilot jobs that pull workloads from the central system.

102 Data Processing Granularity

Another important difference concerns the granularity of workload partitioning.

```
\NormalTok{CMS WMCORE      : dataset → run → lumi → event}
\NormalTok{DIRAC (classic)  : file{-}oriented processing}
\NormalTok{DIRACX             : file{-}oriented execution with higher{-}level
  ⇨ workflow abstractions}
```

CMS workflows frequently operate on data partitions defined by **run, luminosity section, or event counts**, whereas DIRAC typically partitions workloads at the **file level**.

103 Structural Differences

The key architectural distinction between the systems lies in how workflows are defined relative to execution.

```
\NormalTok{CMS workflow system}
\NormalTok{workflow semantics defined before execution}
```

```
\NormalTok{DIRAC (classic)}
```

```
\NormalTok{jobs generated dynamically from input data}
```

```
\NormalTok{DIRACX}
\NormalTok{workflow services define tasks,}
\NormalTok{while execution remains pilot{-}driven}
```

This difference reflects two different workflow philosophies:

- **CMS:** workflows are explicitly defined and decomposed into jobs before execution.
 - **DIRAC:** jobs are generated dynamically as data becomes available.
-

104 Relevance for CMSDiracAux

The CMSDiracAux project introduces a **Translation IR** that bridges the conceptual gap between these architectures.

Conceptually:

```
\NormalTok{CMS Workflow (WMCore)}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{Translation IR}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{DIRAC workflow stack}
\NormalTok{(production / transformation / WMS)}
```

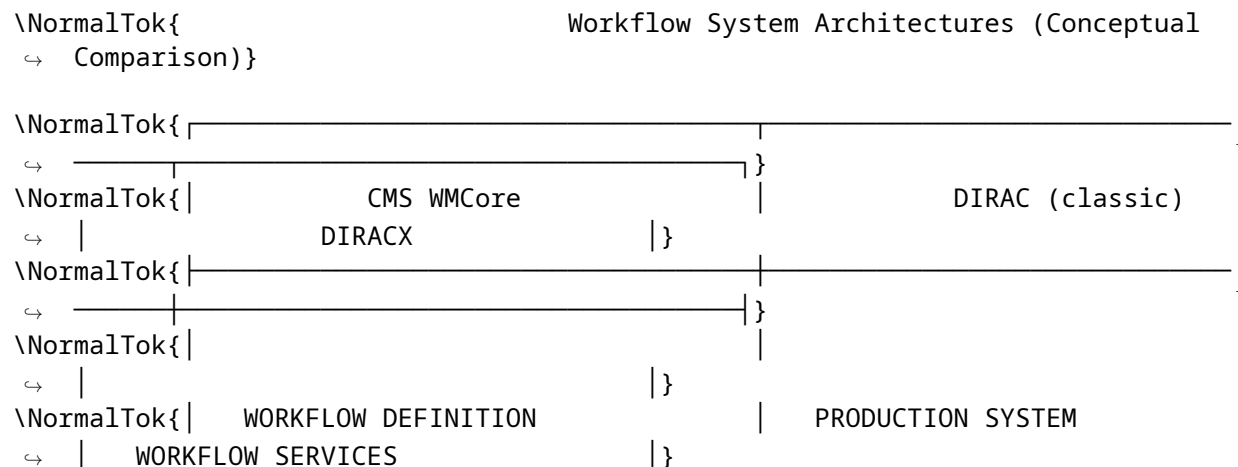
This intermediate abstraction allows workflows defined in the CMS system to be expressed in terms compatible with the DIRAC execution infrastructure.

105 Summary

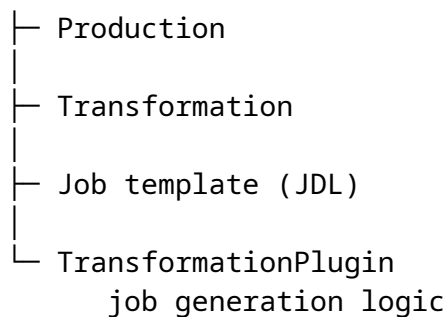
The CMS workflow system, classic DIRAC, and DIRACX represent three distinct approaches to distributed workflow execution:

- CMS emphasizes **explicit workflow definition and fine-grained data partitioning**.
- Classic DIRAC focuses on **dynamic job generation and pilot-based scheduling**.
- DIRACX introduces **modern workflow abstractions and service-oriented architecture**, while retaining pilot-based execution.

Understanding these architectural differences is essential for designing interoperability mechanisms between the two systems.



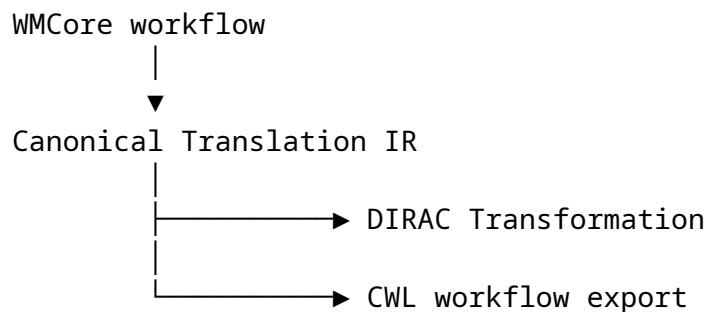
\NormalTok{	(Request / Workflow object)		(processing campaigns)
↪	(workflow APIs)	}	
\NormalTok{			
↪		}	
\NormalTok{			
↪		}	
\NormalTok{		▼	▼
↪		}	
\NormalTok{	TASK STRUCTURE		TRANSFORMATION SYSTEM
↪	TASK SERVICES	}	
\NormalTok{	(task tree / processing steps)		(data{-}driven job
↪	generation) (workflow task graph)	}	}
\NormalTok{			
↪		}	
\NormalTok{			
↪		}	
\NormalTok{		▼	▼
↪		}	
\NormalTok{	WMBS		WORKLOAD MANAGEMENT SYSTEM
↪	SCHEDULING SERVICES	}	
\NormalTok{	(dataset discovery		(job queues / matching)
↪	(resource matchmaking)	}	
\NormalTok{	+ splitting + bookkeeping)		
↪		}	
\NormalTok{			
↪		}	
\NormalTok{			
↪		}	
\NormalTok{		▼	▼
↪		}	
\NormalTok{	JOB GENERATION		PILOT DIRECTORS
↪	PILOT SERVICES	}	
\NormalTok{	(run/lumi/event jobs)		(pilot submission)
↪	(pilot orchestration)	}	
\NormalTok{			
↪		}	
\NormalTok{			
↪		}	
\NormalTok{		▼	▼
↪		}	
\NormalTok{	SUBMISSION INFRASTRUCTURE		PILOT INFRASTRUCTURE
↪	EXECUTION PLANE	}	
\NormalTok{	(grid / batch systems)		(grid / batch)
↪	(grid / cloud / HPC)	}	
\NormalTok{			
↪		}	



This mapping was already reconstructed in the restored conversation PDF.

108 2. Translation pipeline

The PoC architecture is therefore:



Important:

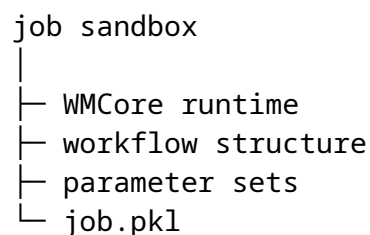
The **Translation IR is not optional**. It is the **architectural bridge** between the two systems.

109 3. Major architectural mismatch discovered

The discussions with DIRAC developers confirmed several **structural mismatches**.

109.1 Runtime code distribution

CMS:



DIRAC:

runtime code → CVMFS

job description → jobDescription.xml

DIRAC transformations **normally do not distribute runtime sandboxes**.

This is a major incompatibility with CMS workflows.

109.2 Job execution model

DIRAC runtime:

Executable: dirac-jobexec

Arguments: jobDescription.xml

All workflow steps are interpreted **inside the XML description**, not the JDL.

This means:

JDL → bootstrap

XML → workflow logic

109.3 Job splitting model

DIRAC splitting constraints:

file = lowest granularity

Sub-file splitting is not supported in vanilla DIRAC.

Workarounds suggested:

1) split dataset into multiple transformations

2) pre-split files externally

This limitation directly affects CMS **event-level splitting models**.

110 4. DIRAC architecture relevant to the PoC

From the developer discussions we learned the following key details.

110.0.1 Transformation plugins

Plugins generate tasks based on input data.

Examples:

Standard
BySize
ByShare

But **real experiments implement custom plugins.**

Example:

LHCbDIRAC TransformationSystem

This is important because CMS will likely need its own plugin.

110.0.2 Agent/plugin roles

DIRAC uses different agents for different operations:

RequestTaskAgent
 data movement

Transformation agents
 job generation

TaskManager plugins
 job placement

These are separate concerns.

110.0.3 Extension architecture

DIRAC is extended experiment-side.

Example:

DIRAC
└─ LHCbDIRAC
 plugin systems
 bookkeeping
 production system

This confirms the feasibility of a **CMSSDIRAC-like extension.**

111 5. CMS workflow complexity (critical insight)

The conversations revealed an important property of CMS workflows.

They are **not purely data-driven.**

Job splitting depends on:

- data volume
- +
- CMSSW runtime requirements
- +
- workflow structure
- +
- data tiers
- +
- processing chains

Meaning:

splitting = multi-dimensional

This explains why simple data-based splitting cannot replicate CMS workflows.

112 6. Workflow structure in CMS

CMS workflows contain chains:

StepChain

TaskChain

Difference:

StepChain

- single pilot

- multiple sequential steps

- no intermediate transfers

TaskChain

- multiple pilots

- intermediate data movement

This concept does **not exist directly in DIRAC**.

Therefore it must be represented in the Translation IR.

113 Important insight discovered

The PoC is not merely a **translator**.

It is effectively creating a **meta-workflow representation**.

Which means your project is really building:

```
WMCore
  ↓
Workflow IR
  ↓
DIRAC
  ↓
CWL
```

This is **very close to a workflow interoperability layer**.

That makes the project conceptually much stronger.

114 WMCore vs DIRAC Workflow Model Comparison

This table will become the **foundation of the report section**:

`docs/reports/interoperability/wmcore-dirac-mismatch.md`

115 Workflow Architecture Comparison

Concept	WMCore (CMS)	DIRAC
Workflow container	WMWorkload	Production / Transformation
Processing stage	WMTask	Transformation
Execution step	WMStep	Workflow Step
Job definition	WMBSJob	Job (JDL)
Job runtime definition	CMSSW configuration (pset)	jobDescription.xml
Job bootstrap	CMSRun	dirac-jobexec
Job parameters	JobPackage.pkl	JobParameters
Job splitting	WMBS splitting plugins	Transformation plugins
Splitting granularity	file / event / runtime constraints	file only
Dataset abstraction	dataset → block → file	LFN list
Data discovery	DAS	DIRAC File Catalog
Workflow orchestration	WMAgent	DIRAC Agents
Runtime code distribution	job sandbox	CVMFS
Workflow DAG	implicit in TaskChain/StepChain	implicit workflow XML
Future workflow representation	none standardized	CWL

116 Key Architectural Differences

116.1 1 Runtime environment

WMCore
payload aware
deeply coupled to CMSSW

DIRAC
payload agnostic
execution wrapper

116.2 2 Workflow semantics

WMCore workflow
= physics workflow

DIRAC workflow
= workload orchestration

116.3 3 Job splitting model

WMCore:
multi-dimensional
data
+
runtime requirements
+
data tiers

DIRAC:
primarily data driven

116.4 4 Runtime entry point

DIRAC jobs:
Executable: dirac-jobexec
Arguments: jobDescription.xml

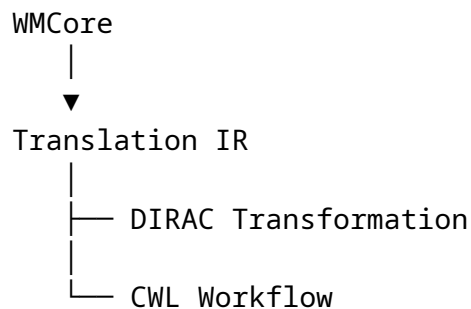
Everything else is interpreted **inside the workflow XML**.

117 3. Formal Translation IR Schema

This is the **most important architectural artifact** in the project.

The Translation IR must represent **workflow semantics independent of both systems**.

118 Translation IR Layer



119 Translation IR Core Entities

119.1 Workflow

Top-level container.

```
Workflow
  id
  name
  tasks[]
  datasets[]
  metadata
```

119.2 Task

Represents a **processing stage**.

```
Task
  id
  name
  input_dataset
  output_dataset
  runtime
```

```
splitting
dependencies[]
```

119.3 RuntimeDefinition

Abstract runtime description.

```
RuntimeDefinition
  executable
  environment
  parameter_sets
  software_stack
```

Examples:

```
CMSSW runtime
container runtime
script runtime
```

119.4 DataReference

Dataset abstraction.

```
DataReference
  dataset
  blocks[]
  files[]
```

119.5 SplittingPolicy

Job generation model.

```
SplittingPolicy
  algorithm
  granularity
  events_per_job
  files_per_job
  runtime_constraints
```

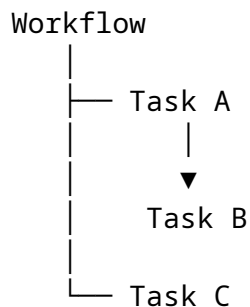
119.6 JobTemplate

Abstract job model.

JobTemplate
runtime
input_data
parameters
resources

120 Translation IR Graph

The IR is naturally a **DAG**.



Dependencies come from **data flow**.

121 IR → DIRAC Mapping

IR element	DIRAC
Workflow	Production
Task	Transformation
RuntimeDefinition	WorkflowStep
JobTemplate	JDL
SplittingPolicy	TransformationPlugin
DataReference.files	LFN list

122 IR → CWL Mapping

IR element	CWL
Workflow	Workflow
Task	CommandLineTool

IR element	CWL
RuntimeDefinition	baseCommand
DataReference	File inputs
SplittingPolicy	Scatter

123 Why Translation IR is Essential

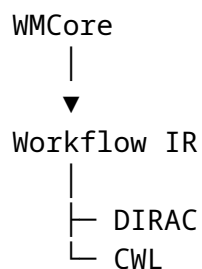
Without the IR the system becomes:

WMCore → DIRAC

Which fails because:

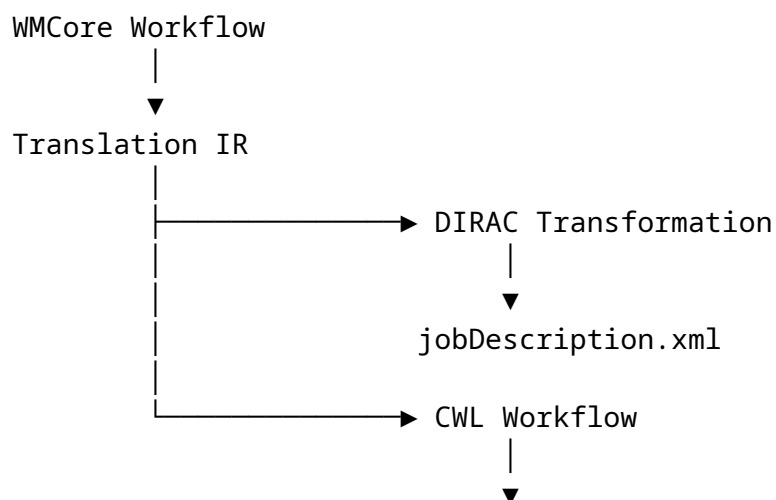
workflow semantics ≠ compatible

With IR:



Now both targets can evolve independently.

124 Updated Architecture Diagram



125 Important Insight for the Report

The PoC is **not simply a translator**.

It is effectively building:

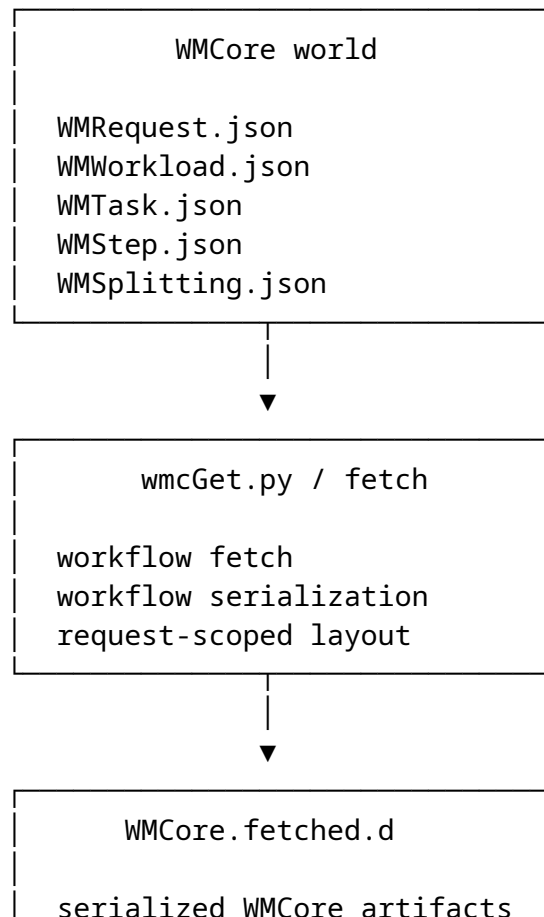
Workflow Interoperability Layer

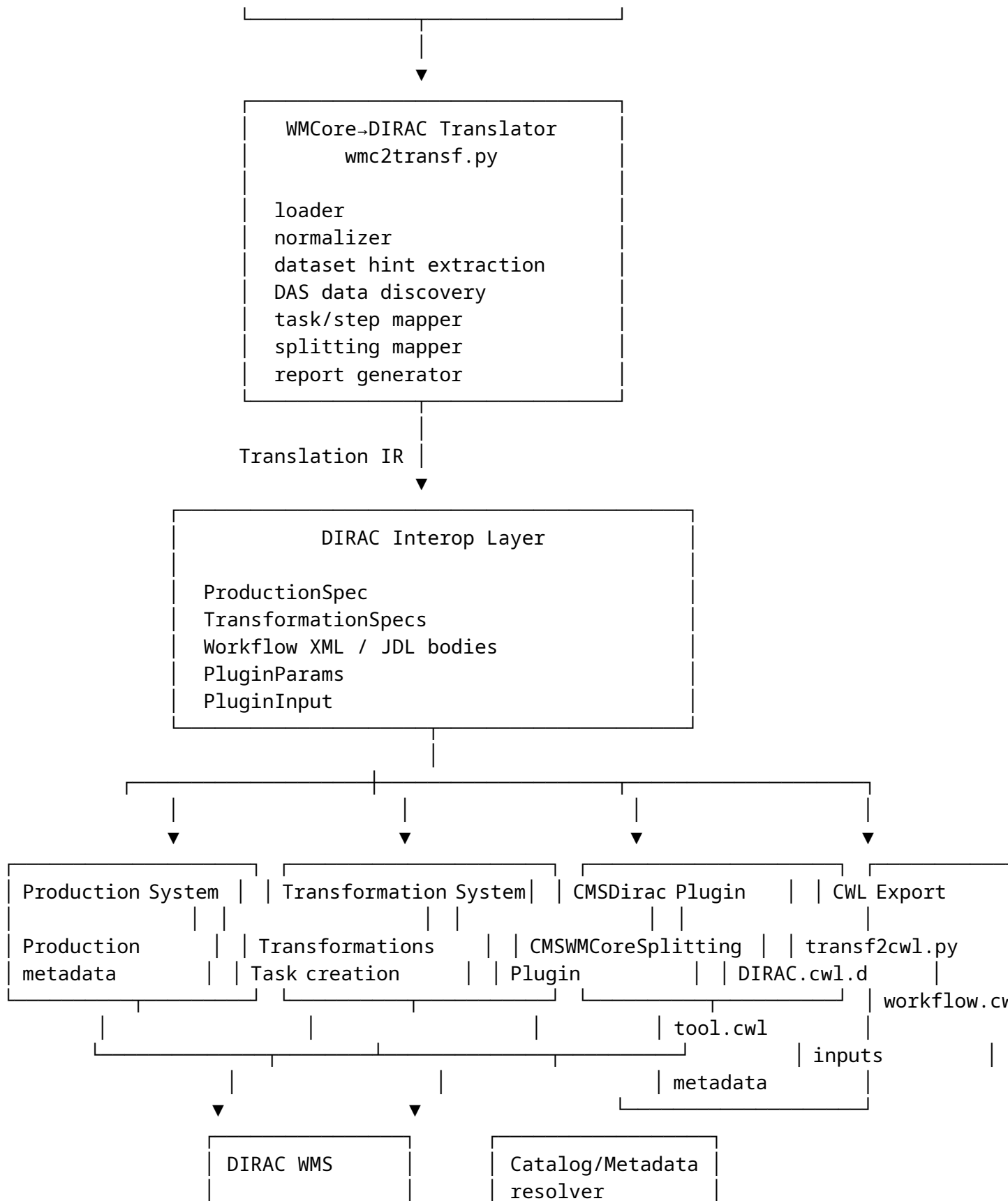
Between two experiment computing systems.

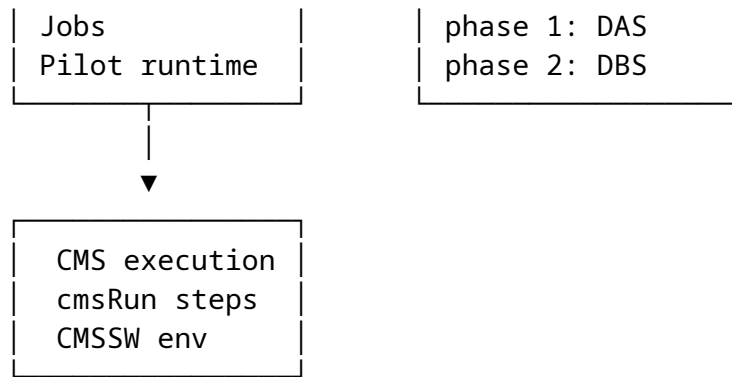
\newpage

126 System Architecture Diagram

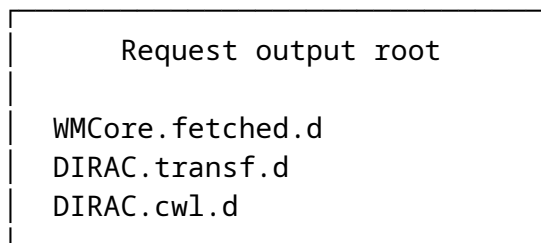
The following diagram shows the current architecture of the WMCore → DIRAC interoperability proof of concept.



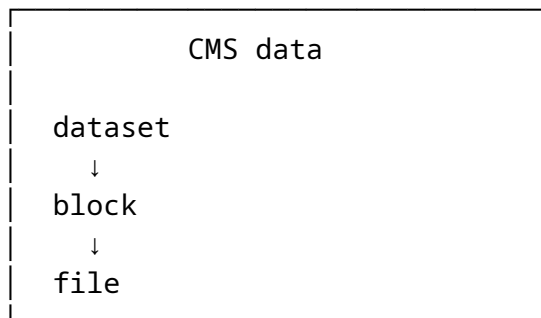




Additional context:



CMS data hierarchy:



PoC scalability limit:

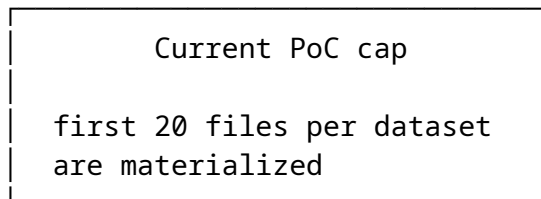


Figure: Detailed CMSDiracAux architecture overview with components. The system extracts workflows from the CMS WMCore infrastructure and translates them into a canonical intermediate representation (Translation IR). The IR is then materialized into DIRAC transformations that reproduce CMS splitting semantics through dedicated plugins. Jobs are executed through the DIRAC workload management system while preserving CMS

runtime behavior.

\newpage

127 System Architecture

127.1 Overview

This document describes the overall architecture of the WMCore → DIRAC interoperability proof of concept implemented in CMSDiracAux.

The architecture bridges two different workflow systems:

WMCore (CMS workflow management)

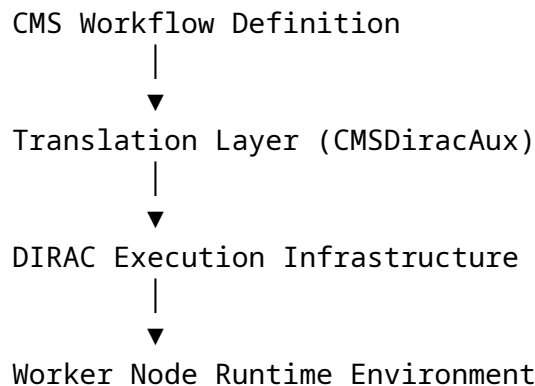
and

DIRAC (distributed workload management).

The CMSDiracAux project demonstrates how workflows defined in the CMS workflow management system (WMCore) can be translated into execution structures compatible with the DIRAC distributed computing framework.

The architecture introduced by CMSDiracAux separates the workflow description from the execution infrastructure by introducing a **canonical Translation Intermediate Representation (IR)**. This allows CMS workflow semantics to be preserved while enabling execution within a different distributed workload management system.

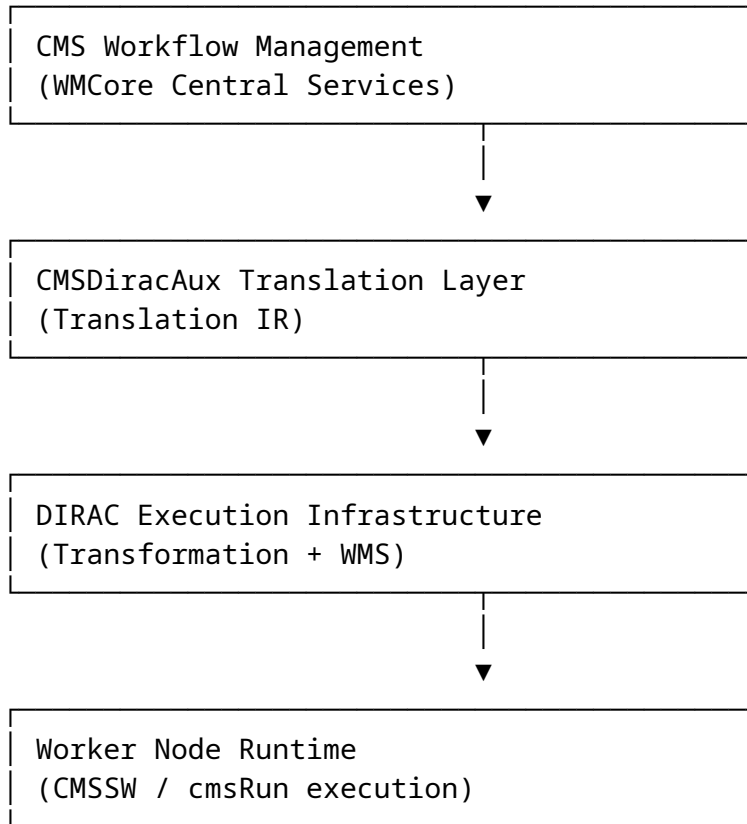
The overall architecture therefore consists of several layers:



Each layer serves a different purpose and represents a different abstraction level in the workflow execution pipeline.

128 Architectural Layers

The CMSDiracAux architecture can be divided into four major layers.



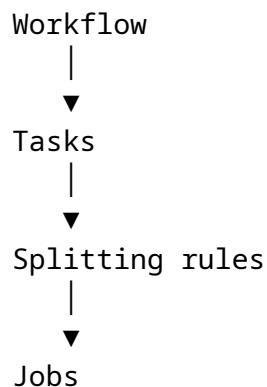
Each layer is described in detail below.

129 CMS Workflow Management Layer

The first layer of the architecture is the CMS workflow management system.

In CMS, workflows are defined and managed by **WMCore**, which orchestrates the entire data processing pipeline for CMS computing tasks.

A CMS workflow typically consists of:



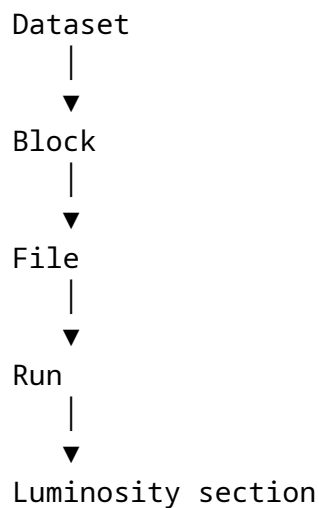
This model emphasizes **explicit workflow definition** before execution.

A CMS workflow definition includes:

- processing steps
- input datasets
- splitting policies
- runtime parameters
- job configuration templates.

The splitting policies determine how a dataset is divided into units of work that can be processed independently.

The CMS data hierarchy underlying this process is:

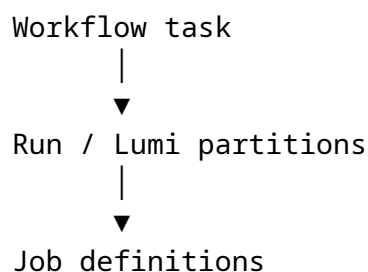


The important aspect of this hierarchy is that CMS jobs often operate on **run or luminosity section ranges**, rather than directly on files.

This means that the storage hierarchy alone does not uniquely determine job boundaries.

To maintain the relationship between workflow tasks, data partitions, and jobs, CMS relies on the **Workload Management Bookkeeping System (WMBS)**.

WMBS stores the association:



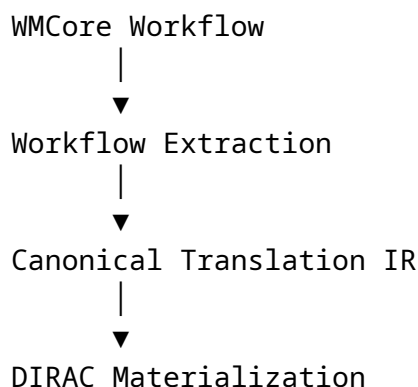
This bookkeeping layer ensures that each job processes the correct portion of the dataset.

130 CMSDiracAux Translation Layer

The central component of the CMSDiracAux architecture is the **translation layer**.

This layer extracts workflow information from WMCore and converts it into a canonical representation that can later be materialized into DIRAC execution structures.

The translation pipeline can be summarized as:



The canonical Translation IR acts as the semantic bridge between the two systems.

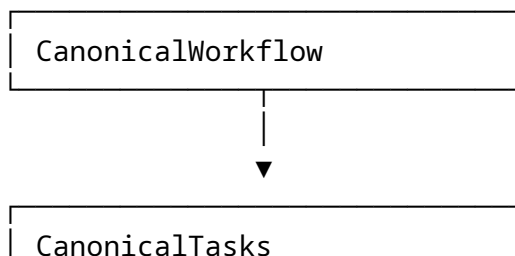
The purpose of the IR is to decouple:

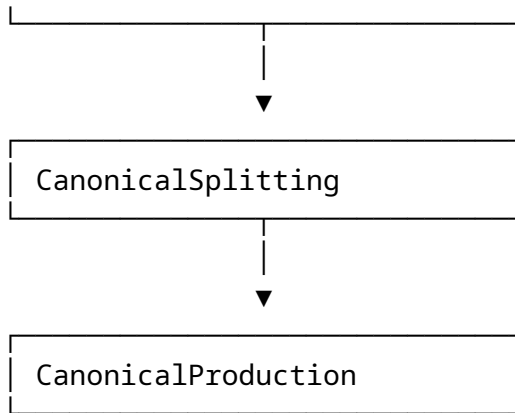
CMS workflow semantics
from
DIRAC execution infrastructure

Without this layer, the translation would require a direct one-to-one mapping between WMCore objects and DIRAC objects, which is difficult because the two systems follow different architectural philosophies.

131 Canonical Translation IR

The Translation IR provides structured objects representing the essential components of a workflow.





These objects capture the essential workflow semantics:

- task hierarchy
- splitting rules
- dataset relationships
- resource hints
- executable definitions.

The IR therefore becomes the **canonical description of the workflow independent of execution infrastructure**.

132 DIRAC Execution Infrastructure

Once the workflow has been translated into the IR, it can be materialized into structures understood by DIRAC.

The primary mechanism for large-scale processing in DIRAC is the **Transformation System**.

A DIRAC transformation is responsible for generating jobs based on data inputs and defined processing rules.

Transformation



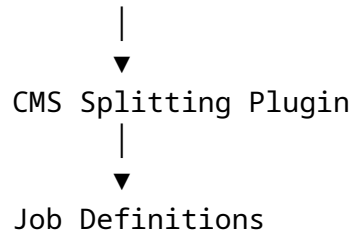
Task queues



Jobs

In the CMSDiracAux architecture, CMS-specific splitting behavior must be implemented inside a transformation plugin.

Transformation



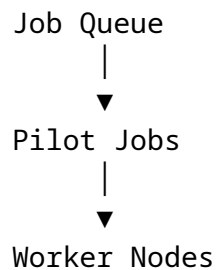
This plugin effectively reproduces WMBS functionality within the DIRAC environment.

This is necessary because CMS workflows require splitting at a granularity that is finer than the storage abstraction used by DIRAC.

133 DIRAC Workload Management

Once jobs have been generated, they are handled by the DIRAC Workload Management System (WMS).

The WMS uses a **pilot-based scheduling model**.



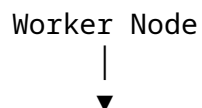
Pilots are generic worker agents that start on computing resources and request work from the central system.

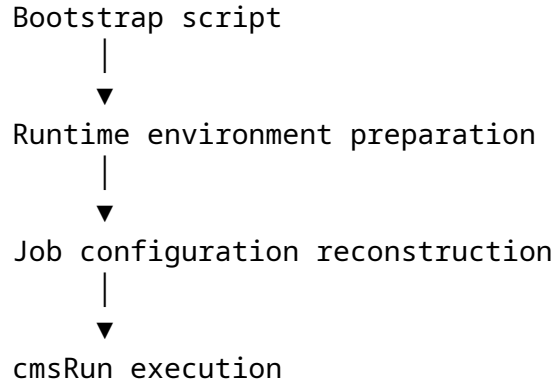
This model improves reliability and resource utilization on heterogeneous distributed computing infrastructures.

134 Worker Node Runtime Environment

When a job reaches a worker node, the runtime environment must be reconstructed before the CMS job can execute.

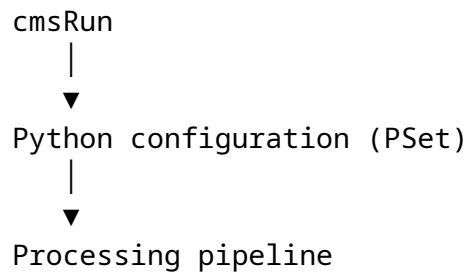
The runtime process typically follows these steps:





The final processing is performed by the **CMSSW framework executable cmsRun**.

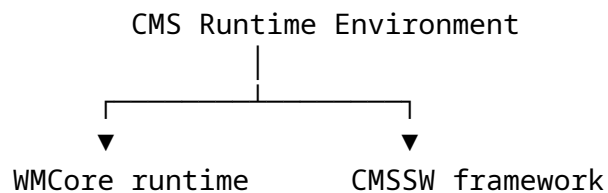
The behavior of cmsRun is determined entirely by its Python configuration (PSet).



Each job therefore requires a configuration that reflects the dataset partition assigned to it.

135 Runtime Environment Constraints

A CMS job depends on two different runtime domains.



WMCORE provides:

- workflow context
- job packages
- runtime parameters
- PSet tweaks.

CMSSW provides:

- physics algorithms
- framework libraries

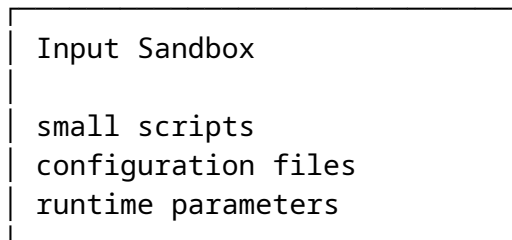
- module execution environment.

Both layers must be available for successful job execution.

136 Runtime Distribution Strategy

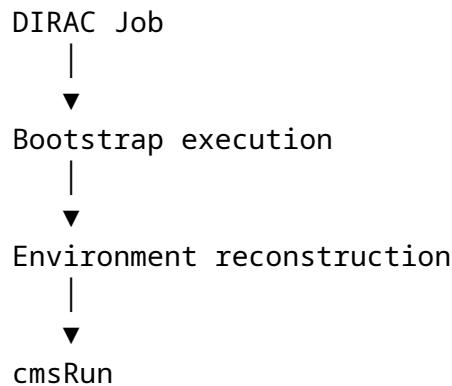
The architecture must also account for constraints in the DIRAC job sandbox model.

The input sandbox is intended to carry small files required for execution.



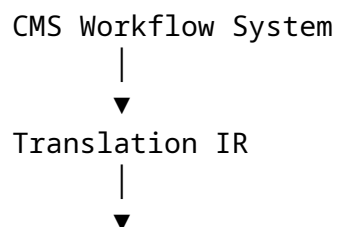
Large runtime bundles are not suitable for sandbox transport.

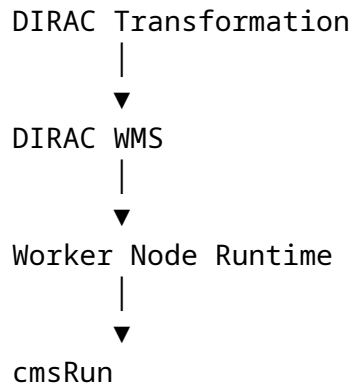
CMSDiracAux therefore uses a bootstrap approach where the worker node reconstructs the runtime environment.



137 Architectural Summary

The CMSDiracAux architecture provides a layered interoperability model.



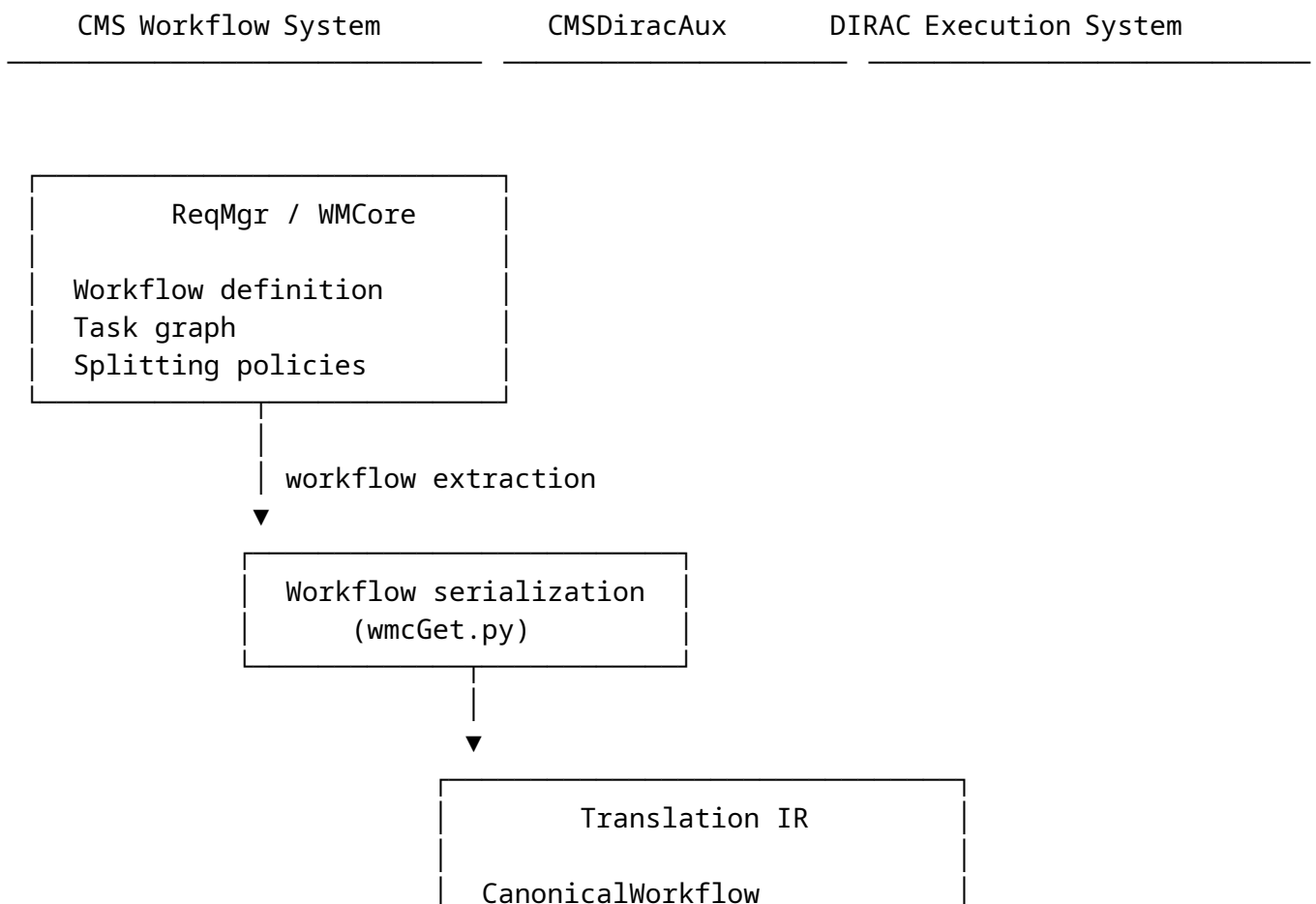


This architecture preserves CMS workflow semantics while allowing execution within the DIRAC distributed computing framework.

The Translation IR serves as the key abstraction layer enabling this interoperability.

138 Detailed view

CMSTDiracAux Architecture Overview



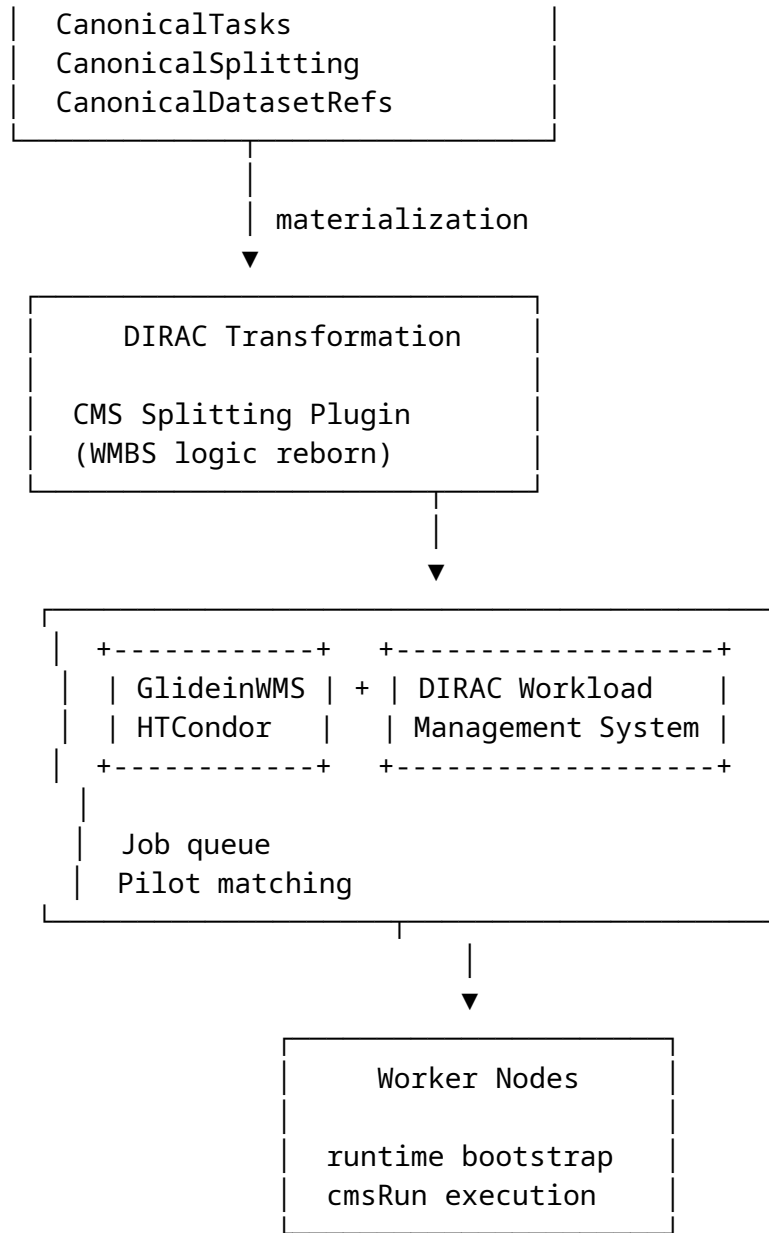
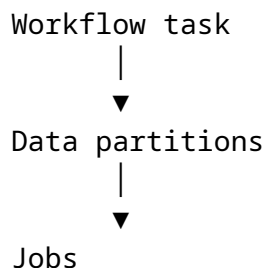


Figure: Detailed CMSDiracAux Functional overview. The system extracts workflows from the CMS WMCore infrastructure and translates them into a canonical intermediate representation (Translation IR). The IR is then materialized into DIRAC transformations that reproduce CMS splitting semantics through dedicated plugins. Jobs are executed through the DIRAC workload management system while preserving CMS runtime behavior.

139 Key Architectural Insight

One of the central conclusions of the CMSDiracAux architecture is that CMS-specific workload bookkeeping cannot be eliminated.

Even when workflows are executed through DIRAC, the system must maintain a mapping between jobs and the data content they process.



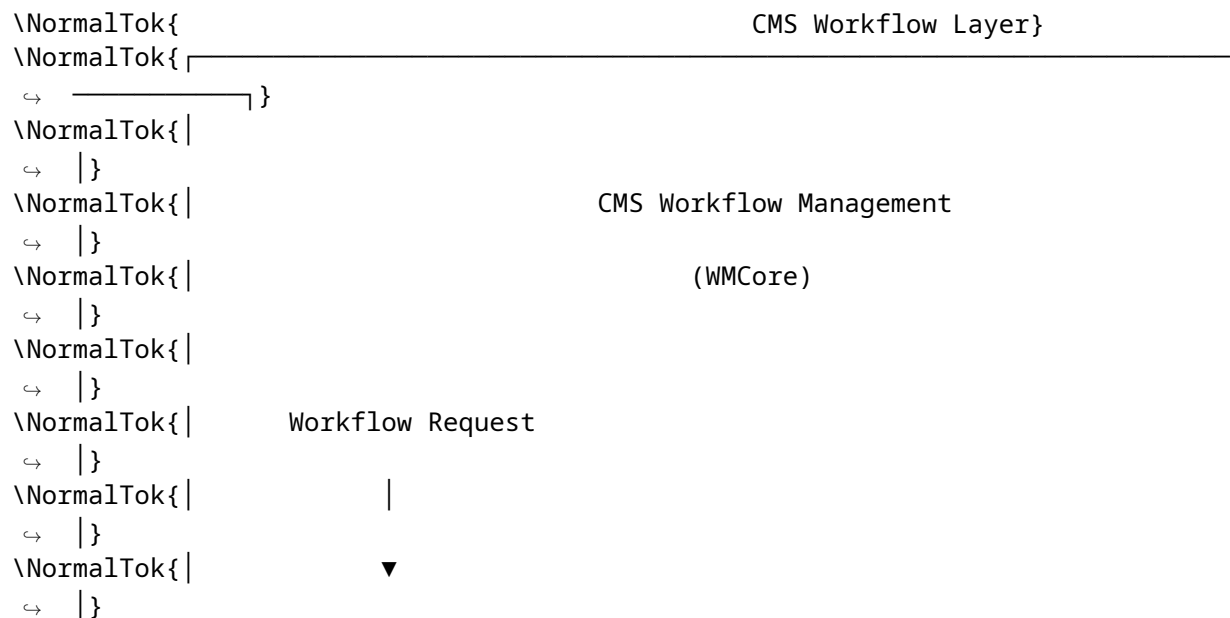
This functionality, originally provided by WMBS, must therefore be implemented inside the DIRAC execution layer through CMS-aware transformation plugins.

This document forms the **central architectural description of the CMSDiracAux system** and should be read together with:

architecture-diagram.md
translation-ir-design.md
wmcore-dirac-mismatch.md

\newpage

140 CMSDiracAux Main Architecture



```

\NormalTok{|           Tasks
  ↪ |}
\NormalTok{|           |
  ↪ |}
\NormalTok{|           ▼
  ↪ |}
\NormalTok{|           Splitting Policies
  ↪ |}
\NormalTok{|
  ↪ |}
\NormalTok{|}
  ↪ }
\NormalTok{|           |}
\NormalTok{|           | workflow extraction}
\NormalTok{|           ▼}

\NormalTok{|           CMSDiracAux Translation Layer}
\NormalTok{|}
  ↪ }
\NormalTok{|}
  ↪ |}
\NormalTok{|           Canonical Translation IR
  ↪ |}
\NormalTok{|}
  ↪ |}
\NormalTok{|           CanonicalWorkflow
  ↪ |}
\NormalTok{|           |
  ↪ |}
\NormalTok{|           ▼
  ↪ |}
\NormalTok{|           CanonicalTasks
  ↪ |}
\NormalTok{|           |
  ↪ |}
\NormalTok{|           ▼
  ↪ |}
\NormalTok{|           CanonicalSplitting
  ↪ |}
\NormalTok{|           |
  ↪ |}
\NormalTok{|           ▼
  ↪ |}
\NormalTok{|           CanonicalProduction
  ↪ |}

```

```

\NormalTok{|
  ↪ |}
\NormalTok{|}
  ↪ }
\NormalTok{|}
\NormalTok{| transformation materialization}
\NormalTok{|}
  ↪ }

```

```

\NormalTok{|}
\NormalTok{| DIRAC Execution Layer}
  ↪ }
\NormalTok{|}
\NormalTok{|}
\NormalTok{| DIRAC Transformation System}
  ↪ |}
\NormalTok{|}
\NormalTok{| Transformation}
  ↪ |}
\NormalTok{|}
  ↪ |}
\NormalTok{|}
  ↪ }
\NormalTok{|}
\NormalTok{| CMS Splitting Plugin}
  ↪ |}
\NormalTok{|}
  ↪ |}
\NormalTok{|}
  ↪ }
\NormalTok{|}
\NormalTok{| Jobs}
  ↪ |}
\NormalTok{|}
  ↪ |}
\NormalTok{|}
  ↪ }
\NormalTok{|}
  ↪ }
\NormalTok{|}
  ↪ |}
\NormalTok{|}
  ↪ | pilot scheduling}
\NormalTok{|}
  ↪ }

```

```

\NormalTok{|}
\NormalTok{| DIRAC Workload Management System}
  ↪ }
\NormalTok{|}
  ↪ |}
\NormalTok{|}
\NormalTok{| Job Queue}
  ↪ |}

```



```
\NormalTok{CMS WMBS}
\NormalTok{      ↓}
\NormalTok{CMSDiracAux splitting plugin}

\newpage
```

This section provides a view of which pieces would be substituted from the current CMS Workflow management system once a **CMS-Dirac** hybrid system is implemented with **CMSDiracAux** serving as an interoperable layer.

1. **Workflow extraction boundary** - where WMCore workflows are intercepted.
2. **Execution backend boundary** - where the CMS Submission Infrastructure would normally generate and dispatch jobs.

This approach is consistent with how large workflow systems are typically layered, where workflow definition and execution infrastructure can be decoupled through intermediate abstractions. (arXiv)

```

\NormalTok{CMS Workflow Management System}
\NormalTok{|}
↪ |}
\NormalTok{|}
↪ |}
\NormalTok{|} CENTRAL SERVICES
↪ |}
\NormalTok{|}
↪ |}
\NormalTok{|} |}
↪ |}
\NormalTok{|} |}
↪ |}
\NormalTok{|} | Request / ReqMgr | | DBS / DAS | | CouchDB /
↪ | |}
\NormalTok{|} | workflow definition | | data discovery | | workflow
↪ state | |}

```



- Not break communication APIs with stakeholders systems
- Not imply or require conceptual changes to external systems
- Not break operational patterns
- Be backwards compatible

This is the reason why the initial Workflow construction from the current WMCore system should remain as is. It serves as a bridge between other stakeholder systems and the main Workflow management system. It may by itself evolve in the future or in the process of creating the new system, but nevertheless, should still follow the same principles as above.

This is the layer which provides the full hierarchical assembly of a CMS workflow.

146.1 First cut and new inter system communication line — workflow extraction

CMSDiracAux intercepts the system **after workflow definition but before WMBS job materialization**.

Meaning:

WMCore workflows

↓

Translation IR

Instead of letting WMBS produce job definitions directly, the workflow is translated into a **canonical representation**.

This is the **semantic decoupling point**.

146.2 Second cut and new inter system communication line — execution backend

CMSDiracAux replaces the **CMS job construction and submission infrastructure**:

WMBS → SI → HTCondor → GlideinWMS

with

Translation IR → DIRAC Transformation → DIRAC WMS + GlideinWMS/HTCondor

This means the **execution backend changes**, but the **workflow semantics remain CMS-native**.

147 Key Architectural Insight

`\NormalTok{CMSDiracAux does not eliminate WMBS logic.}`

Instead:

WMBS functionality

↓

reappears in the DIRAC splitting plugin

This is necessary because CMS splitting operates **below the storage abstraction level** (run/lumi/event).

1. CMS workflow semantics remain intact.
2. Execution infrastructure becomes interchangeable.
3. The system can evolve toward DIRAC or DIRACX without rewriting the CMS workflow model.

Conceptually:

```
\NormalTok{CMS workflow semantics}
\NormalTok{      ↓}
\NormalTok{Translation IR}
\NormalTok{      ↓}
\NormalTok{execution backend (DIRAC / DIRACX)}
```

`\newpage`

148 WMCore vs DIRAC Execution Model

This document explains the conceptual differences between the CMS workflow management system (WMCore / WMAgent / WMBS) and the DIRAC workload management framework.

Understanding these differences is essential for explaining why the translation layer implemented in CMSDiracAux is necessary.

-
1. System roles
-

Both systems operate in the domain of distributed computing, but they focus on different layers of the workload lifecycle.

WMCore

workflow management system

DIRAC

workload execution and scheduling framework

WMCore defines physics workflows and dataset processing logic.

DIRAC focuses on scheduling jobs on distributed resources.

2. Execution unit definition

One of the most important differences is the definition of the atomic processing unit.

WMCore atomic unit

luminosity section

DIRAC atomic unit

file

This difference originates from the needs of CMS data processing.

CMS workflows must guarantee that each luminosity section is processed exactly once.

The CMS workflow management system therefore operates on a finer level than most workload schedulers.

3. WMCore workflow structure

A simplified WMCore workflow structure is:

Request | v Task | v Step | v WMBS job

Where:

Request

global workflow description

Task

dataset processing unit

Step

runtime configuration stage

WMBS job

execution unit assigned to the batch system

The WMBS component performs bookkeeping and static splitting.

4. WMBS splitting model

WMBS exists because the CMS workflow system operates on data units that are finer than the scheduling capabilities of typical batch systems.

Typical scheduling systems operate at:

file granularity

WMCore requires processing guarantees at:

luminosity section granularity

To reconcile these layers, WMBS performs static job splitting and tracks the relationship between:

dataset file run luminosity section

This allows CMS workflows to enforce strict data processing guarantees.

5. DIRAC execution structure

DIRAC organizes workload execution differently.

A simplified DIRAC structure is:

Transformation | v Task | v Job

Transformation

logical workload definition

Task

unit of work derived from transformation input data

Job

execution unit submitted to worker nodes

DIRAC transformations dynamically generate jobs based on input data.

6. Static vs dynamic splitting

Another important difference is splitting strategy.

WMCore

static splitting

DIRAC

dynamic splitting

WMCore splitting is usually performed before jobs are submitted.

DIRAC splitting typically occurs inside the Transformation Agent during workload execution.

7. Data discovery

CMS workflows rely heavily on dataset discovery.

Typical data discovery path:

dataset | v DBS / DAS query | v block list | v file list

DIRAC usually receives explicit input file lists.

Therefore the translation layer must perform dataset resolution.

8. Runtime environment

CMS jobs require a complex runtime environment.

Important components include:

CMSSW software framework

WMCore runtime modules

runtime configuration artifacts

Examples of runtime artifacts:

step_cfg.py WMWorkload.pkl JobPackage.pkl

These artifacts describe the physics workflow logic executed by the job.

DIRAC jobs normally execute simpler command-line payloads.

Integrating CMS runtime expectations with the DIRAC execution model requires additional translation logic.

9. Why a translation layer is required

Because WMCore and DIRAC operate at different abstraction levels.

WMCore focuses on:

physics workflow semantics

dataset processing logic

luminosity-level bookkeeping

DIRAC focuses on:

job scheduling

resource matchmaking

distributed execution

The translation layer converts workflow semantics into execution semantics.

10. Role of the Translation IR

The canonical Translation IR provides an intermediate abstraction layer.

WMCore | $\vee$ Translation IR | $\vee$ DIRAC transformation

The IR:

decouples source and target systems

preserves workflow semantics

enables export to workflow languages such as CWL

11. Limitations of the current PoC

The current proof of concept does not yet reproduce the full CMS workflow execution model.

Key limitations include:

file-level processing only

no run/lumi mask support

limited dataset materialization

Current file materialization cap:

20 files per dataset

This limitation exists to keep the PoC manageable during development.

12. Future integration challenges

Several challenges remain for a full integration.

Handling run/lumi masks

intra-file splitting

integration with CMS data management systems

integration with DIRAC server-side transformation agents

mapping CMS runtime sandbox semantics

13. Conceptual summary

WMCore and DIRAC operate at different conceptual layers.

WMCore

workflow management

DIRAC

workload execution

The translation layer implemented in CMSDiracAux bridges these layers through a canonical intermediate representation.

This architecture allows CMS workflows to be expressed in terms that can eventually be executed through DIRAC-based infrastructures or modern workflow languages.

\newpage

149 CMS vs DIRAC Workflow Execution Models

Comparison diagram that visualizes the **conceptual difference between CMS and DIRAC workflow execution models** and supports the **Schrödinger vs Heisenberg analogy**

```
\NormalTok{                                CMS Workflow Model}
\NormalTok{                                (Explicit Workflow Evolution)}

\NormalTok{    Workflow Definition}
\NormalTok{    |}
\NormalTok{    ▼}
\NormalTok{    Task Graph}
```

```

\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Splitting Rules}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Job Definitions Created}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Jobs Submitted to Grid}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Worker Node Execution}

```

```

\NormalTok{      Interpretation:}

```

```

\NormalTok{      Workflow evolution is defined}
\NormalTok{      before runtime execution.}

```

```

\NormalTok{-----}
↪  ———}

```

```

\NormalTok{      DIRAC Execution Model}
\NormalTok{      (Dynamic Data{-}Driven Execution)}

```

```

\NormalTok{      Production / Transformation}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Task Queues Created}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Pilot Jobs Start}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Worker Node Requests Job}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Job Assigned Dynamically}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Payload Execution}

```

```

\NormalTok{      Interpretation:}

```

```
\NormalTok{      Workload is assigned dynamically}
\NormalTok{      during runtime execution.}
```

150 Data Interaction Comparison

Another useful view highlights how workflows interact with data.

```
\NormalTok{      CMS Workflow Interaction with Data}
```

```
\NormalTok{      Workflow}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Dataset Definition}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Files Discovered}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Run / Lumi Partitioning}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Jobs Created}
```

```
\NormalTok{-----}
```

```
\NormalTok{      DIRAC Workflow Interaction with Data}
```

```
\NormalTok{      Transformation}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Files Discovered}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      Job Created}
\NormalTok{      |}
\NormalTok{      ▼}
\NormalTok{      File Processed}
```

Figure X: Conceptual comparison between CMS and DIRAC workflow execution models. CMS workflows explicitly define the job structure before execution, while DIRAC assigns work dynamically through a pilot-based execution model. The CMSDiracAux Translation IR bridges these two execution philosophies.

151 Conceptual Interpretation

The difference can be summarized as follows.

151.0.1 CMS

```
\NormalTok{Workflow → Data → Jobs}
```

The workflow evolves explicitly and determines the job set before execution.

151.0.2 DIRAC

```
\NormalTok{Data → Jobs during runtime}
```

The system dynamically creates and schedules jobs based on available resources and data.

152 Physics Analogy

This conceptual difference is similar to two equivalent formulations of quantum mechanics.

```
\NormalTok{CMS workflow model}
```

```
\NormalTok{↔}
```

```
\NormalTok{Schrödinger picture}
```

```
\NormalTok{The system state evolves explicitly.}
```

```
\NormalTok{DIRAC execution model}
```

```
\NormalTok{↔}
```

```
\NormalTok{Heisenberg picture}
```

```
\NormalTok{Operators evolve while the state}
```

```
\NormalTok{remains implicitly defined.}
```

This analogy is not exact but provides an intuitive mental model for understanding the architectural distinction.

153 Implication for CMSDiracAux

The interoperability layer must bridge these two philosophies.

```

\NormalTok{CMS explicit workflow model}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{Translation IR}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{DIRAC dynamic execution model}

```

The Translation IR allows CMS workflow semantics to be preserved while adapting them to the DIRAC execution infrastructure.

\newpage

154 WMCore-DIRAC Workflow Model Mismatch

154.1 Purpose of this section

This section explains the **architectural mismatch between the CMS workflow management system (WMCore/WMBS) and the DIRAC workload management framework**.

Understanding this mismatch is critical for the CMSDiracAux project because the goal of the project is **not only to translate workflow descriptions**, but also to provide a **transition layer of workflow-level abstractions between the two systems**.

The following sections address specific aspects of this interoperability challenge:

- dataset-resolution-model.md
- translation-ir-rationale.md
- job-description-translation.md

As stated in the project objectives:

The ultimate goal of the project is not only to create a deterministic translation PoC between the CMS workflow management system and DIRAC, but also to express the complexity of the workflows description and make and explain a transition layer of workflow level abstractions between the two systems, which is inevitably connected with the two system's internals. (reflecting the fact that Non of the two systems' workflows descriptions are agnostic to their own architectures)

The mismatch originates from a fundamental difference:

```

WMCore workflows = experiment execution model
DIRAC workflows  = distributed workload orchestration

```

Therefore, **a direct translation between the two systems is not possible without introducing an intermediate abstraction layer**.

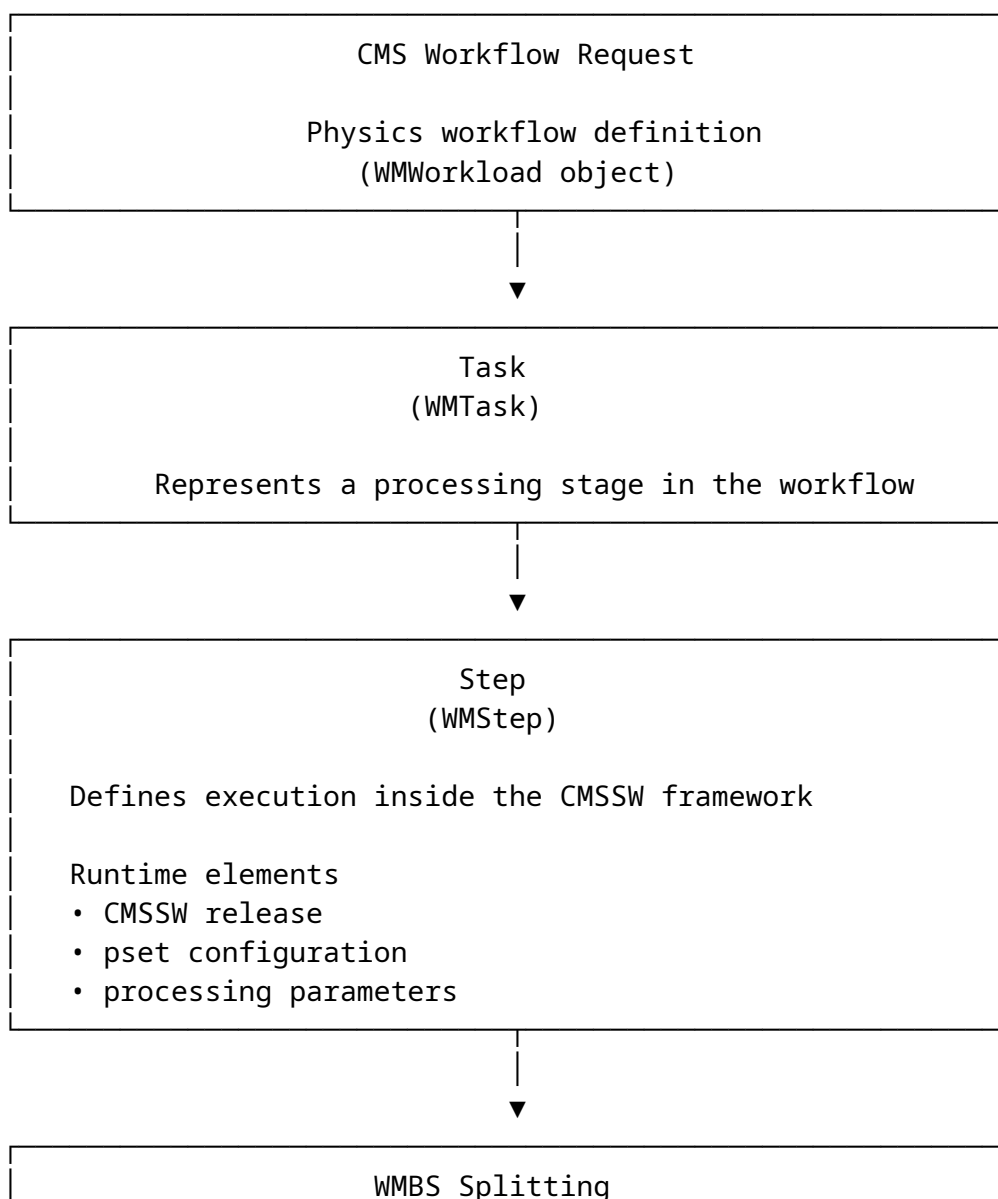
155 CMS Workflow Execution Model

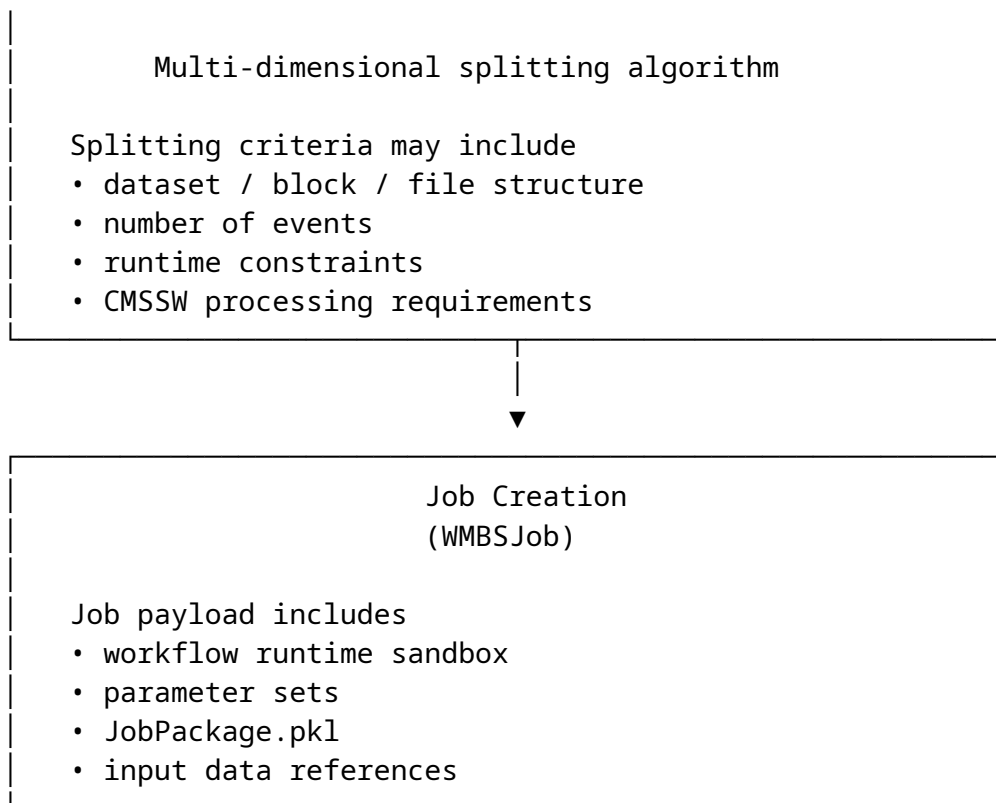
The CMS workflow system is implemented in **WMCore** and **WMBS** and is tightly coupled to the **CMSSW experiment framework**.

The system operates at multiple hierarchical levels:

- workflow definition
- task decomposition
- step execution
- dataset-aware splitting
- job creation

The execution model is shown below.





156 Key Characteristics of the CMS Workflow Model

156.0.1 Workflow definitions are experiment-aware

CMS workflows are tightly integrated with the **CMSSW runtime environment**.

A workflow does not only describe scheduling logic. It also contains:

- experiment software configuration
- physics processing parameters
- runtime environment assumptions

This makes CMS workflows **payload-aware**.

156.0.2 Multi-dimensional job splitting

Job splitting in CMS is not purely data driven.

Splitting decisions may depend on:

data hierarchy

+

runtime configuration
+
processing chains
+
resource constraints

This leads to **multi-dimensional splitting**.

156.0.3 Dataset hierarchy drives workflow execution

CMS data is structured hierarchically:

```
dataset
|
▼
block
|
▼
file
```

Workflow execution often depends on this hierarchy.

157 DIRAC Workflow Execution Model

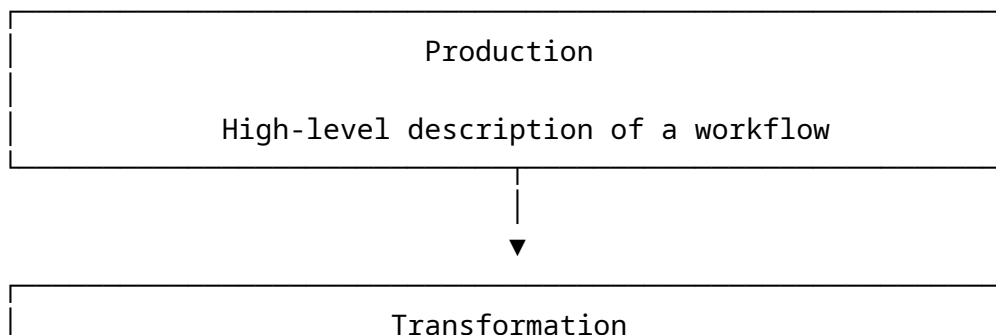
DIRAC is designed as a **general distributed workload management system**.

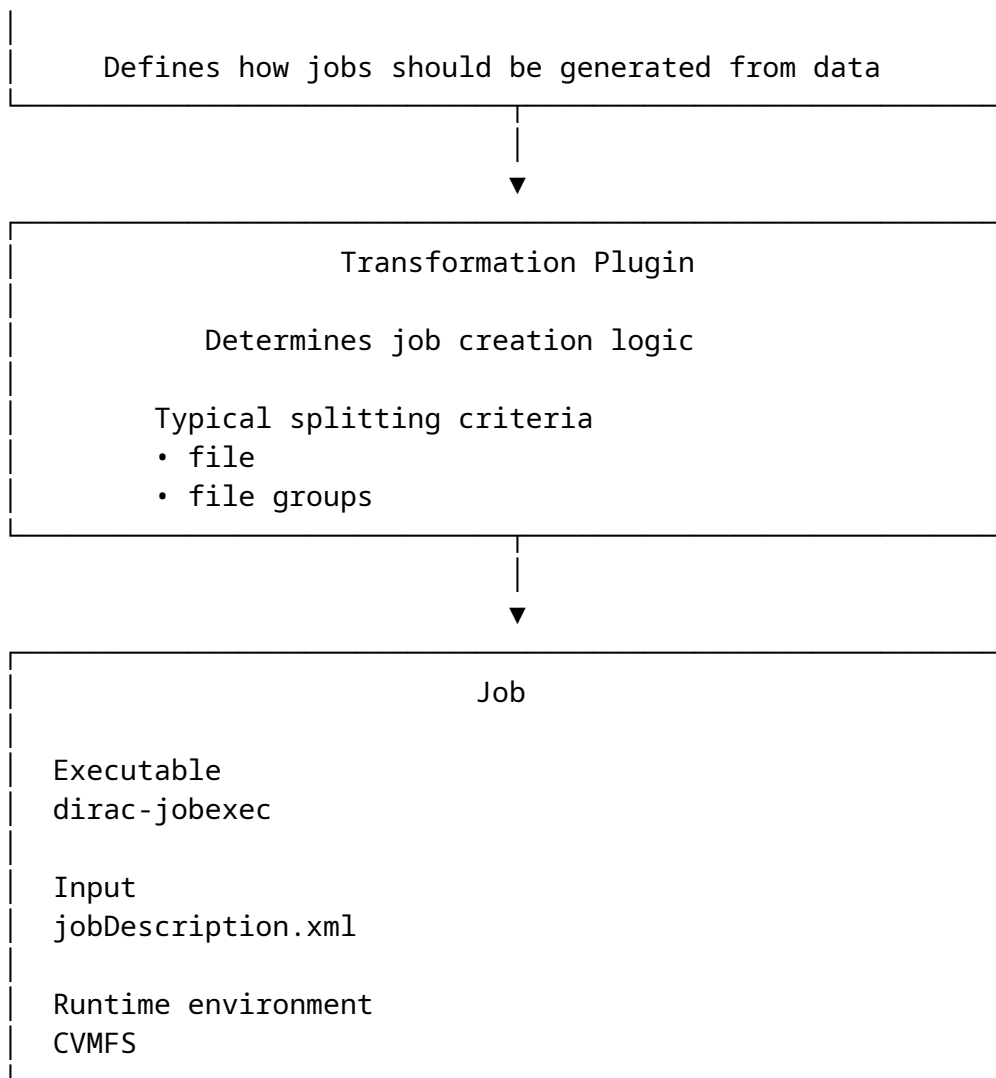
Unlike WMCore, DIRAC workflows are **payload-agnostic**.

The system focuses on:

- job orchestration
- resource scheduling
- distributed execution

The execution model is shown below.





158 Key Characteristics of the DIRAC Workflow Model

158.0.1 Payload-agnostic workflow definition

DIRAC does not assume knowledge of experiment-specific software.

Jobs are executed through a generic entry point:

dirac-jobexec

The job runtime logic is described in:

jobDescription.xml

158.0.2 Runtime environment externalization

Unlike CMS workflows, DIRAC assumes runtime software is **already available on the worker node**.

Typically via:

CVMFS

Therefore jobs do not normally carry runtime code in a sandbox.

158.0.3 Data-driven job splitting

DIRAC transformations typically split work based on **input data granularity**.

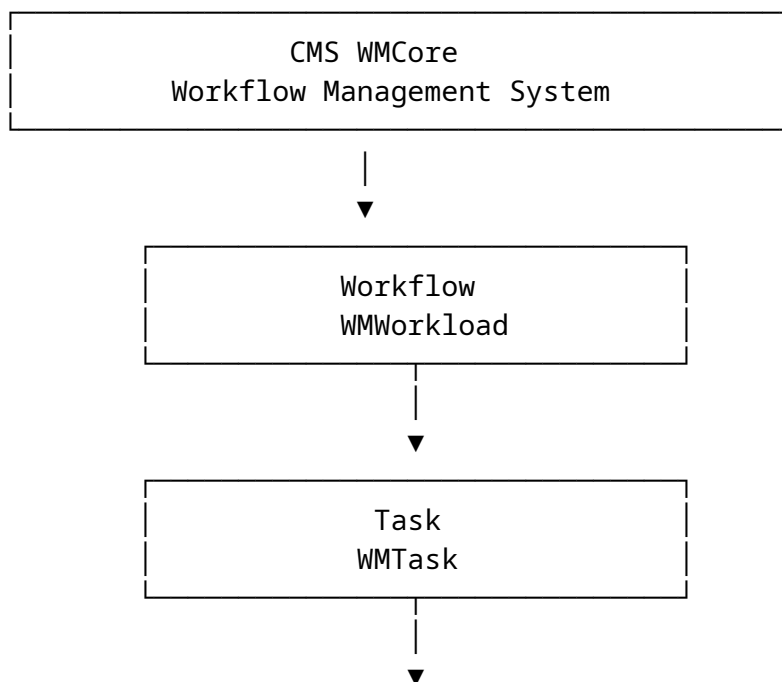
The most common granularity is:

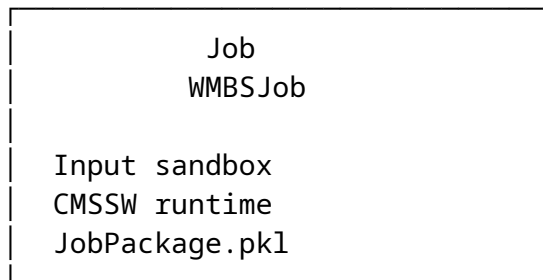
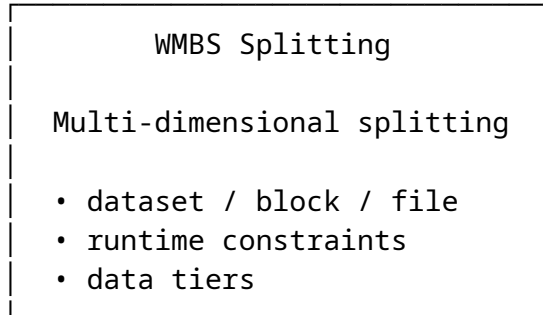
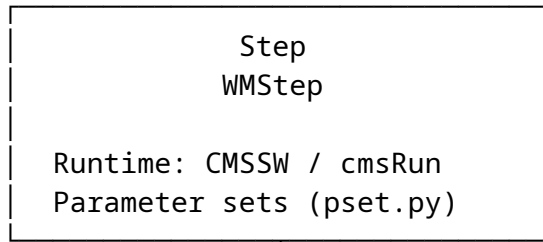
file

This differs significantly from CMS workflows where splitting may depend on **processing parameters and runtime logic**.

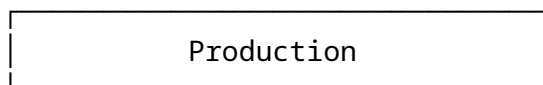
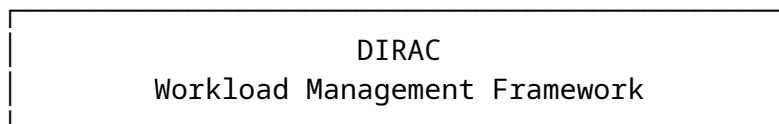
159 Architectural Comparison

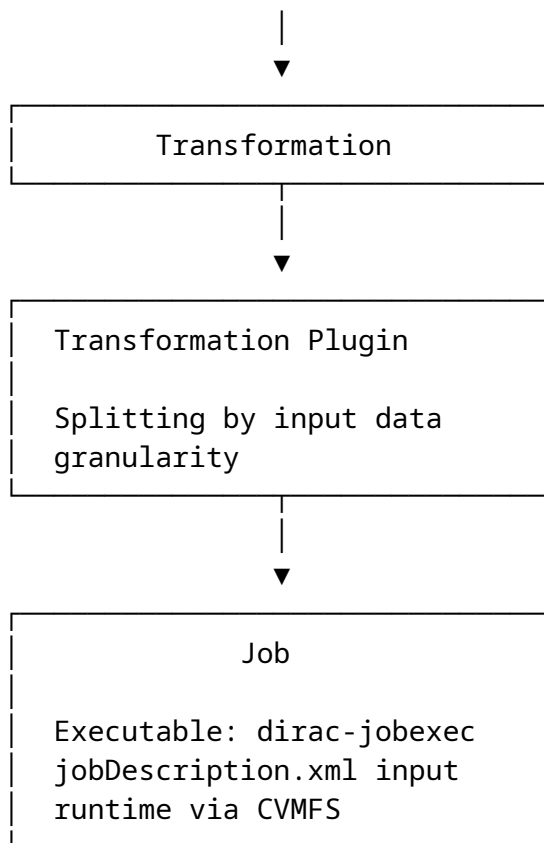
The following diagram highlights the structural mismatch between the two workflow models.





ARCHITECTURAL GAP





160 Why a Translation Layer is Necessary

The comparison shows that **the two systems operate at different abstraction levels**.

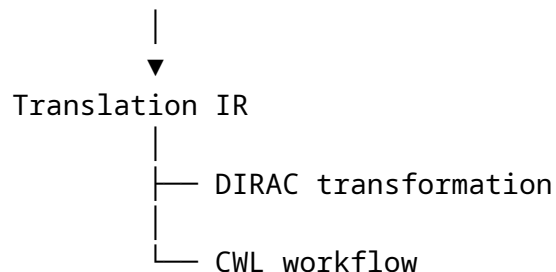
WMCore → experiment workflow system

DIRAC → distributed execution framework

Direct translation would therefore lose essential information.

To resolve this problem, CMSDiracAux introduces a **canonical Translation IR**.

WMCore Workflow



The Translation IR allows workflow semantics to be represented in a **system-independent form**.

This makes it possible to:

```
translate WMCore workflows
generate DIRAC transformations
export workflows to CWL
```

without coupling the workflow description to either system.

161 Static vs Dynamic Workflow-Data Coupling

An important conceptual difference between the CMS workflow system (WM-Core/WMBS) and DIRAC concerns **how workflows interact with data and how workload is distributed over the data space**.

Although both systems ultimately execute jobs on distributed computing resources, they approach the relationship between **workflow definition and data distribution** in fundamentally different ways.

This difference can be characterized as a contrast between **static workflow partitioning** and **dynamic workload expansion**.

162 CMS Workflow Model: Static Workflow Partitioning

In the CMS workflow system, the workflow is **fully defined before runtime** and then distributed across the dataset during job creation.

The WMBS subsystem performs the following operations:

1. **Dataset discovery**
2. **Dataset partitioning**
3. **Job generation**
4. **Runtime configuration generation**

Once these steps are completed, the runtime jobs are already **fully defined units of work**.

Each job receives:

```
\NormalTok{\bullet a well-defined subset of the dataset}
\NormalTok{\bullet runtime configuration parameters}
\NormalTok{\bullet a CMSSW execution configuration}
```

The runtime configuration is generated using **PSetTweaks**, which determine the **exact processing boundaries of each job over the dataset**.

Conceptually, the system performs the following transformation:

```
\NormalTok{workflow definition}  
\NormalTok{      +}  
\NormalTok{dataset description}  
\NormalTok{      ↓}  
\NormalTok{explicit job list}
```

The workflow management system therefore produces **all executable units of work ahead of execution**.

The runtime environment simply executes the predefined jobs.

This results in a **static workflow partitioning model**.

163 DIRAC Workflow Model: Dynamic Workload Expansion

DIRAC follows a different philosophy.

Instead of producing a fully materialized set of jobs ahead of execution, DIRAC uses **workflow templates combined with transformation rules**.

A transformation describes:

```
\NormalTok{• job template}  
\NormalTok{• input data selection criteria}  
\NormalTok{• job splitting policy}
```

The transformation system then **generates jobs dynamically as data becomes available**.

Conceptually, the transformation system performs:

```
\NormalTok{workflow template}  
\NormalTok{      +}  
\NormalTok{incoming data}  
\NormalTok{      ↓}  
\NormalTok{job generation}
```

Jobs are therefore not necessarily enumerated in advance.

Instead, they are **spawned dynamically based on data discovery and transformation rules**.

This produces a **dynamic workload expansion model**.

164 Comparison of the Two Models

The two approaches differ primarily in **when the workflow is resolved into executable jobs**.

Aspect	CMS WMCore	DIRAC
Workflow definition	full workflow graph defined	workflow template defined
Job generation moment	before execution	during execution
Dataset interaction	workflow distributed over dataset	jobs generated from data
Runtime configuration	embedded in job	externalized in environment
Execution model	static job set	dynamic job spawning

This difference significantly influences how each system manages data distribution and load balancing.

165 Analogy Between Quantum Pictures and Workflow Management Models

165.1 Context

When comparing the CMS Workflow Management System (WMCore) with the DIRAC workflow model, an analogy with different formulations of quantum mechanics can help explain the conceptual difference in how workflows are represented and executed.

166 Schrödinger Picture

In the **Schrödinger picture**, the **state evolves with time**, while the operators remain fixed.

Conceptually:

```
\NormalTok{state(t) evolves}
\NormalTok{operators remain fixed}
```

In this formulation, the entire evolution of the system is encoded in the **explicit time evolution of the state vector**.

166.1 Analogy with CMS WMCore

The CMS workflow system behaves similarly to the Schrödinger picture.

In CMS:

- workflows are **fully defined in advance**
- job boundaries are **explicitly constructed**
- the entire evolution of the workflow is **pre-determined**

Conceptually:

```
\NormalTok{workflow definition}  
\NormalTok{      ↓}  
\NormalTok{explicit job splitting}  
\NormalTok{      ↓}  
\NormalTok{predefined jobs}  
\NormalTok{      ↓}  
\NormalTok{execution}
```

Thus, the **state of the workflow evolves**, while the structure of the execution model remains fixed.

167 Heisenberg Picture

In the **Heisenberg picture**, the situation is reversed:

```
\NormalTok{operators evolve}  
\NormalTok{state remains fixed}
```

The system state is fixed, while the **observables (operators)** evolve with time.

This formulation focuses on **transformations acting on the system rather than explicit evolution of the state**.

167.1 Analogy with DIRAC

The DIRAC workflow model resembles the Heisenberg picture.

In DIRAC:

- the workflow definition acts as a **static template**
- jobs are generated dynamically as data becomes available
- the execution evolves through **operations applied to data**

Conceptually:

```
\NormalTok{workflow template}  
\NormalTok{      ↓}
```

```

\NormalTok{data appears}
\NormalTok{      ↓}
\NormalTok{transformation generates tasks}
\NormalTok{      ↓}
\NormalTok{jobs executed dynamically}

```

Thus, the **execution operators evolve**, while the workflow description remains largely static.

168 Conceptual Summary

```

\NormalTok{CMS WMCORE}
\NormalTok{Schrödinger{-}like model}

```

```

\NormalTok{workflow evolves explicitly}
\NormalTok{jobs defined in advance}

```

```

\NormalTok{DIRAC}
\NormalTok{Heisenberg{-}like model}

```

```

\NormalTok{workflow template fixed}
\NormalTok{execution generated dynamically}

```

This analogy highlights the fundamental philosophical difference between the systems:

- **CMS workflows encode the full evolution of computation explicitly.**
- **DIRAC workflows encode transformations applied dynamically to available data.**

This conceptual distinction helps explain why an intermediate abstraction layer such as the **Translation IR** is required to bridge the two workflow models.

169 Validity of the Analogy

The analogy is conceptually useful but should be treated carefully.

The systems do not literally implement physical time evolution models. However, the analogy captures an important architectural difference.

The essential point is the **location of dynamism in the system**.

System	Dynamic component
CMS WMCORE	workflow state over dataset

System	Dynamic component
DIRAC	job generation from workflow template

Thus:

```
\NormalTok{CMS → dynamic workflow over static job definitions}
\NormalTok{DIRAC → static workflow template generating dynamic jobs}
```

This analogy serves as a **useful conceptual aid**, but it should be interpreted as an illustration rather than a strict formal equivalence.

170 Implications for Workflow Translation

This difference has direct implications for interoperability.

Because the CMS system resolves workflows **before execution**, while DIRAC resolves workflows **during execution**, a translation between the two systems cannot simply map objects one-to-one.

Instead, a translation layer must represent:

```
\NormalTok{• workflow structure}
\NormalTok{• dataset interaction model}
\NormalTok{• job generation semantics}
```

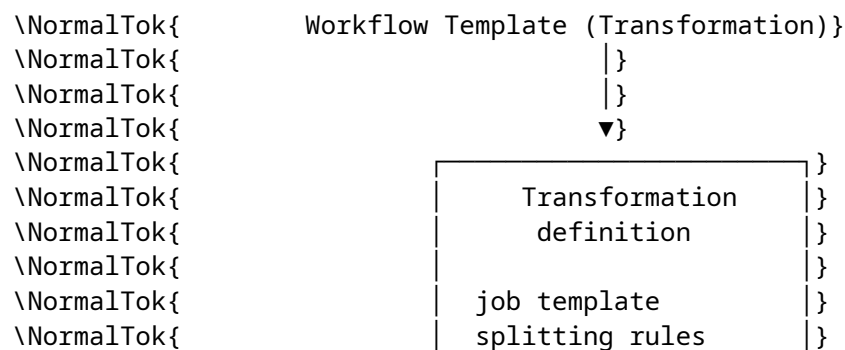
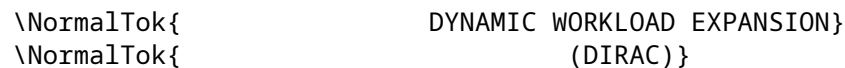
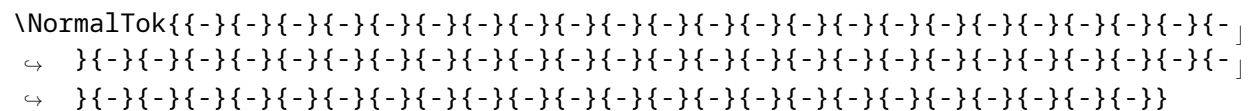
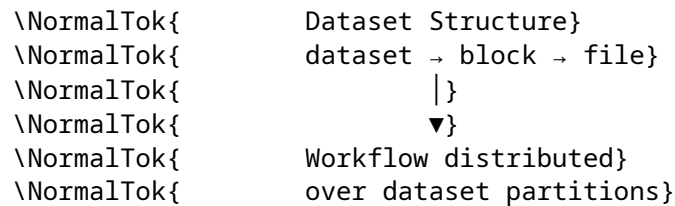
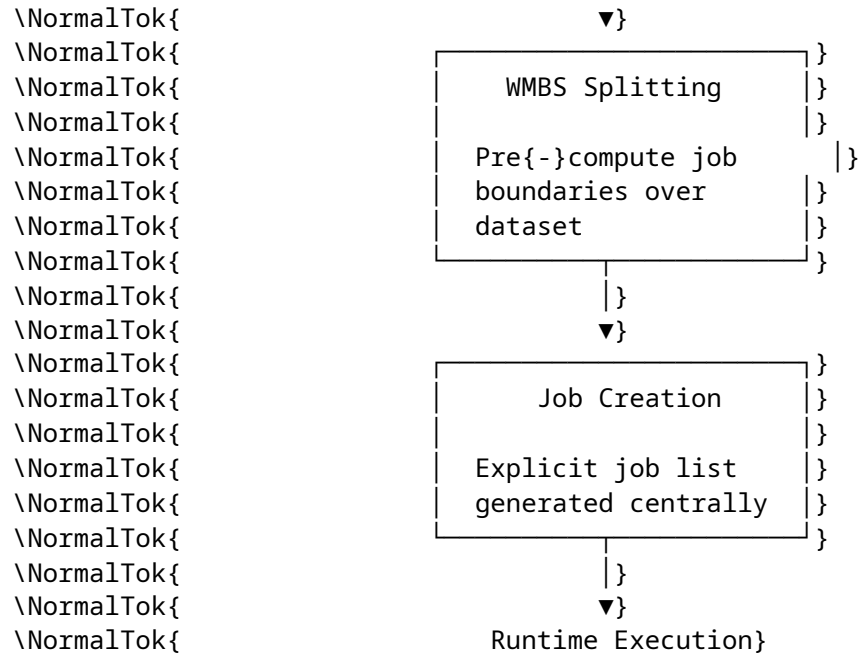
independently of the execution framework.

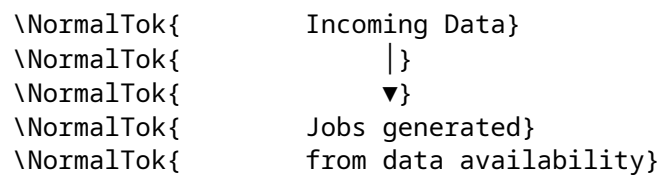
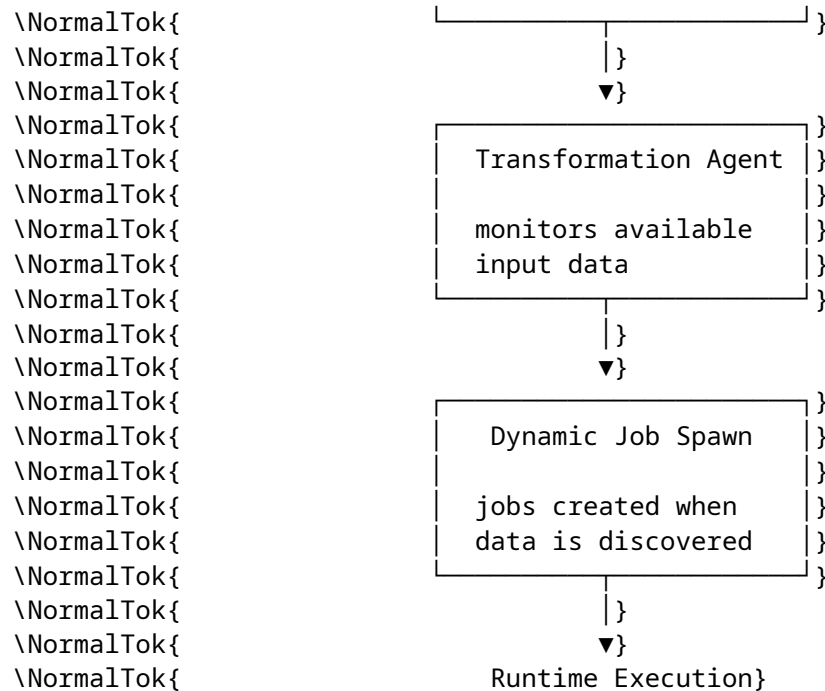
This requirement is one of the primary motivations for the **Translation IR** introduced in the CMSDiracAux project.

171 Static vs Dynamic Workflow-Data Interaction Models

```
\NormalTok{                                STATIC WORKFLOW PARTITIONING}
\NormalTok{                                (CMS WMCore / WMBS)}

\NormalTok{                                Workflow Definition (WMWorkload / TaskChain)}
\NormalTok{                                |}
\NormalTok{                                |}
\NormalTok{                                ▼}
\NormalTok{                                }
\NormalTok{                                Workflow Graph }
\NormalTok{                                fully defined }
\NormalTok{                                }
\NormalTok{                                |}
\NormalTok{                                |}
```





171.0.1 CMS model

\NormalTok{workflow → job set → runtime}

The workflow is **expanded into explicit jobs before execution.**

Data is treated as a **space over which the workflow is distributed.**

171.0.2 DIRAC model

```
\NormalTok{workflow template → dynamic job generation}
```

The workflow remains a **template**, and jobs are **spawned dynamically from data**.

172 WMBS necessity argument

Here is yet another place to mention why WMBS Exists, even though already explained in other sections of the report.

The CMS workflow system requires job splitting at a much finer granularity than the storage hierarchy used for dataset management.

While datasets and files represent the storage organization of CMS data, job boundaries are often defined using run, luminosity section, or event counts.

This requirement necessitates an additional bookkeeping layer that tracks the association between jobs and data units.

The Workload Management Bookkeeping System (WMBS) fulfills this role.

173 DIRACX outlook

DIRACX Perspective

The DIRACX architecture introduces explicit workflow and task abstractions that are conceptually closer to the CMS workflow model.

Classic DIRAC represents workflows using productions and transformations, while DIRACX moves toward a workflow $\rightarrow$ task $\rightarrow$ job hierarchy.

This convergence reduces the conceptual distance between the CMS workflow model and DIRAC execution infrastructure, which strengthens the relevance of the Translation IR approach proposed in this work.

174 Conceptual Interpretation

The diagram highlights where **dynamism resides in each system**.

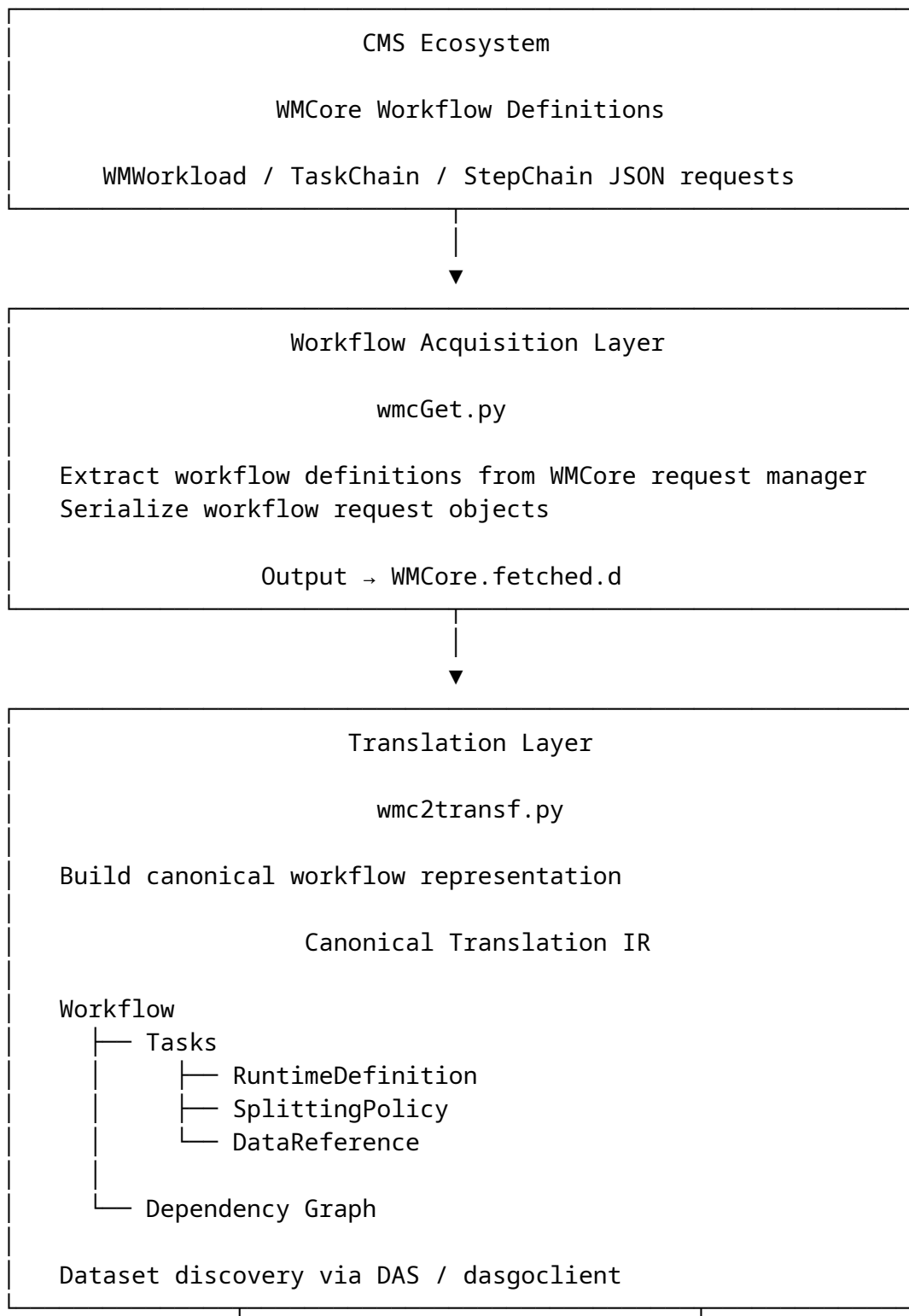
System	Dynamic element
CMS WMCore	workflow state over dataset
DIRAC	job generation from workflow template

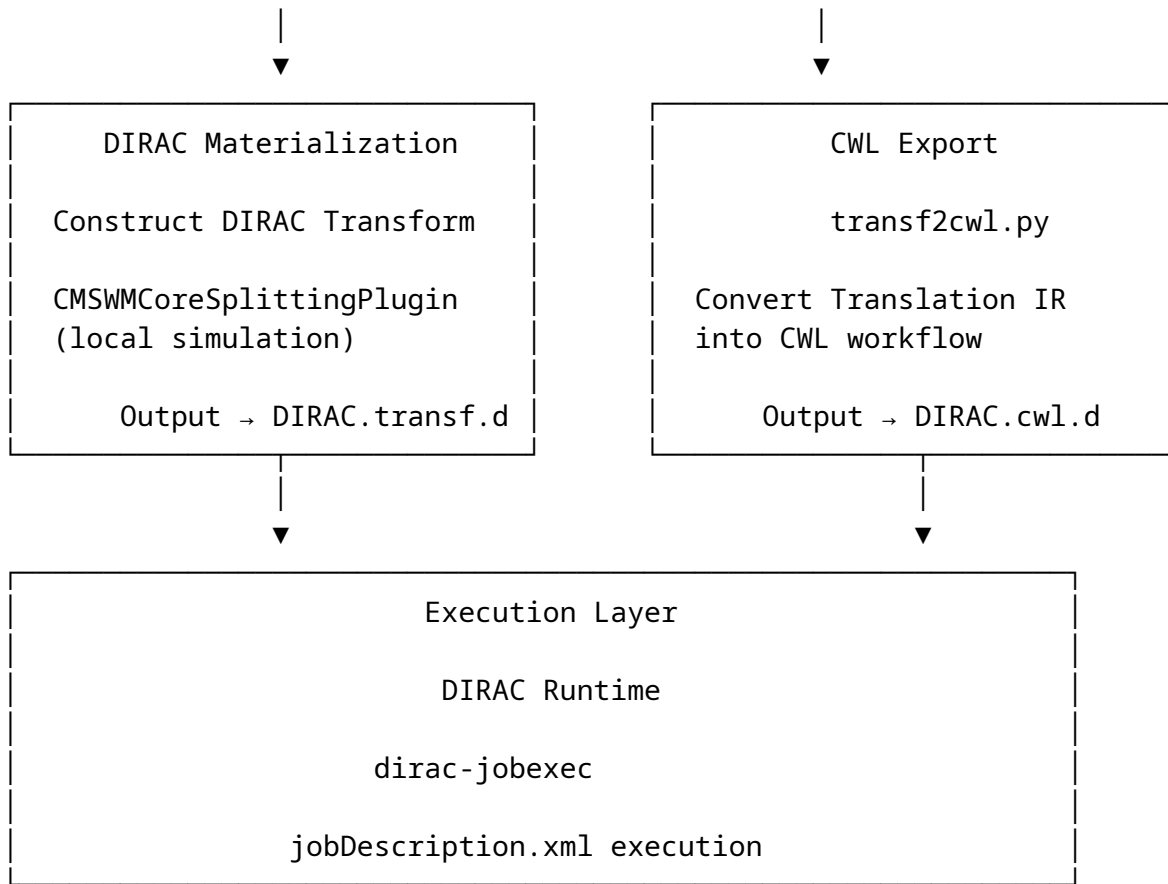
This explains why **direct translation between the systems is difficult**.

The CMSDiracAux project therefore introduces a **Translation IR** that separates:

```
\NormalTok{workflow semantics}  
\NormalTok{data distribution semantics}  
\NormalTok{execution system semantics}
```

175 IR Layer placement





\newpage

176 WMBS, DIRAC, and the Granularity Gap

A core architectural issue in this project is the mismatch between the semantic granularity used by CMS workflow management and the granularity naturally used by lower-level workload systems.

176.1 CMS workflow-management granularity

In CMS workflow management, the effective atomic unit is often a luminosity section. The CMS system must guarantee data uniqueness: no two jobs should reprocess the same luminosity section.

This requirement pushes CMS workflow management toward finer bookkeeping and assignment semantics than those naturally supported by most lower-level execution systems.

176.2 Lower-level scheduling granularity

Systems such as HTCondor and DIRAC naturally operate closer to file/job granularity.

They can associate jobs with files well, but they do not naturally express multiple jobs per file or lumi-level assignment semantics as first-class objects.

176.3 Why WMBS exists

One role of WMBS (Workflow Management Bookkeeping System) is to bridge this semantic mismatch.

WMBS performs a second-level bookkeeping and job/data association stage. It exists in the WMAgents layer and supports:

- static early splitting,
- bookkeeping of job-to-data associations,
- finer-grained mapping needed by CMS processing semantics,
- insulation of higher-level workflow logic from lower-level scheduler details.

Historically, this also helped preserve flexibility with respect to the underlying scheduling system.

176.4 Relevance for the current PoC

This is one of the reasons why the current proof of concept cannot simply map WM-Core semantics directly to plain DIRAC file-level transformations and claim full equivalence.

For the current stage, the project preserves and carries forward WMBS-style job parameters where available, but does not yet resolve the deeper architectural question of whether WMBS semantics should:

- remain present in some form,
- be emulated in DIRAC,
- be translated into a future workflow language,
- or be partially redesigned away.

\newpage

177 Translation IR Rationale

177.1 Purpose

The CMSDiracAux project introduces a **Translation Intermediate Representation (Translation IR)** to bridge the architectural and conceptual differences between the CMS Workflow Management System (WMCore) and the DIRAC workflow execution stack.

The Translation IR serves as a **canonical workflow description layer** that allows workflows defined in WMCore to be represented in a system-agnostic form before being materialized into DIRAC execution structures.

This section explains:

- why the Translation IR is necessary
 - how the IR abstracts workflow definitions
 - how workflow parameters and data structures are mapped
 - how IR objects translate into DIRAC constructs
-

178 Motivation

CMS workflows and DIRAC workflows are based on fundamentally different execution philosophies.

CMS workflows define the **entire job structure before execution**, whereas DIRAC workflows generate jobs dynamically as data becomes available.

```
\NormalTok{CMS Workflow Model}

\NormalTok{Workflow}
\NormalTok{  ↓}
\NormalTok{Tasks}
\NormalTok{  ↓}
\NormalTok{WMBS splitting}
\NormalTok{  ↓}
\NormalTok{Jobs}

\NormalTok{DIRAC Workflow Model}

\NormalTok{Production}
\NormalTok{  ↓}
\NormalTok{Transformation}
\NormalTok{  ↓}
\NormalTok{Tasks}
\NormalTok{  ↓}
\NormalTok{Jobs}
```

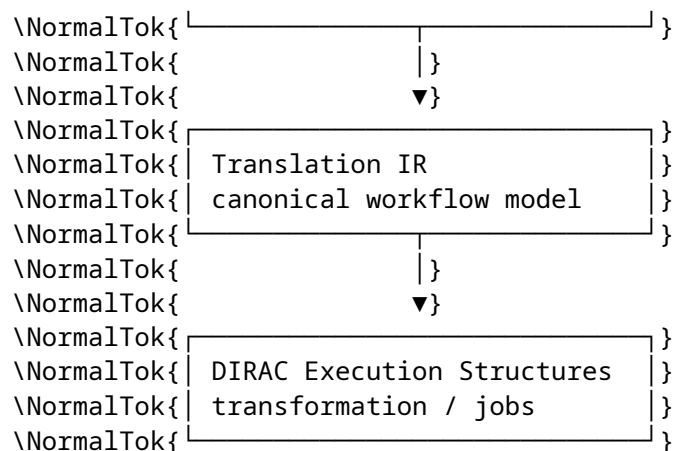
These models cannot be mapped directly because they operate at different abstraction layers.

The Translation IR provides a stable intermediate layer between the two systems.

179 Position of the Translation IR

The IR sits between the CMS workflow description and the DIRAC execution model.

```
\NormalTok{ }
\NormalTok{ CMS Workflow (WMCORE) }
\NormalTok{ workflow / task structure }
```



The Translation IR therefore performs two roles:

1. **semantic normalization**
2. **execution preparation**

180 Why Direct Field Mapping Is Insufficient

A naïve approach might attempt to translate workflow definitions by mapping field names from WMCore objects directly to DIRAC job definitions.

Rationale behind using a direct mapping would be the following rule of thumb: Direct field-name mapping is valid when all three are true:

1. the meaning is the same,
2. the cardinality is the same,
3. the lifecycle role is the same.

However this approach fails because:

1. **workflow abstractions differ**
2. **splitting models differ**
3. **job definitions occur at different layers**

```

\NormalTok{WMCore workflow}
\NormalTok{ }
\NormalTok{task definitions}
\NormalTok{ }
\NormalTok{splitting policies}
\NormalTok{ }
\NormalTok{job definitions}

```

versus

```

\NormalTok{DIRAC workflow}
\NormalTok{ }
\NormalTok{transformation}

```

```
\NormalTok{      ↓}  
\NormalTok{file discovery}  
\NormalTok{      ↓}  
\NormalTok{job generation}
```

180.0.1 Different abstraction boundaries

WMCore:

- request
- workload
- task
- step
- splitting

DIRAC:

- production
- transformation
- transformation plugin
- tasks/jobs
- workflow/job body

Those are similar, but not identical. Some WMCore fields map:

- directly,
- indirectly,
- after aggregation,
- after splitting normalization,
- or after inheritance resolution.

So many mappings are actually **semantic transforms**, not simple field copies.

180.0.2 One WMCore field may affect several DIRAC fields

Example pattern:

A WMCore step's runtime/resource configuration may influence:

- body executable arguments,
- CPU requirement,
- memory requirement,
- tags,
- plugin annotations.

That is not “copy one field to one field.”

180.0.3 Several WMCore fields may collapse into one normalized concept

For example, splitting information may come from several places in the WMCore material, but in DIRAC there should be one normalized plugin payload such as:

```
\FunctionTok{\{
  \DataTypeTok{"Mode"}\FunctionTok{:}
  ↪ \StringTok{"EventAwareLumiBased"}\FunctionTok{,,}
  \DataTypeTok{"EventsPerJob"}\FunctionTok{:} \DecValTok{5000}\FunctionTok{,,}
  \DataTypeTok{"LumisPerJob"}\FunctionTok{:} \DecValTok{10}\FunctionTok{,,}
  \DataTypeTok{"RespectRunBoundaries"}\FunctionTok{:} \KeywordTok{true}
\FunctionTok{\}}
```

The IR is where those multiple inputs collapse into one stable representation.

180.0.4 IR gives a reusable contract for future phases

A canonical translation object should be looked as a compiler layer:

```
\NormalTok{WMCore JSON = source language}
\NormalTok{Translation IR = abstract syntax / normalized program form}
\NormalTok{DIRAC objects = target language}
```

A compiler that directly rewrites source tokens into machine instructions is usually brittle.

A compiler with an IR is usually much more maintainable.

This is the same architectural reason.

180.1 Field-level schema with more concrete names

WMCore object	WMCore field name	Canonical IR field
WMRequest.json	RequestName	ProductionName
WMRequest.json	RequestType	ProductionType
WMRequest.json	Campaign	CampaignName
WMRequest.json	AcquisitionEra	AcquisitionEra
WMRequest.json	ProcessingString	ProcessingString
WMRequest.json	PrepID	PrepId
WMRequest.json	Priority	Priority
WMWorkload.json	RequestName or workload name	ProductionName
WMWorkload.json	task graph / task list	TaskGraph
WMWorkload.json	global policy values	GlobalPolicy
WMTask.json	TaskName	TaskName
WMTask.json	task path, often hierarchical	TaskPath
WMTask.json	task type / step type	TransformationType
WMTask.json	input dataset refs	InputDataset

WMCore object	WMCore field name	Canonical IR field
WMTask.json	output dataset intent	OutputDataset
WMTask.json	parent task refs	ParentTasks
WMTask.json	site whitelist / blacklist	SitePolicy
WMTask.json	task-level priority	Priority
WMTask.json	splitting section or ref	SplittingPolicy
WMStep.json	step name	StepName
WMStep.json	CMSSWVersion	SoftwareVersion
WMStep.json	ScramArch	SoftwareArchitecture
WMStep.json	step config / ConfigCacheID / cfg ref	StepConfiguration
WMStep.json	executable semantics, usually cmsRun	Executable
WMStep.json	runtime args	Arguments
WMStep.json	memory requirement	MemoryMB
WMStep.json	cores / threads	CpuCores
WMStep.json	estimated wallclock / time	CpuTime
WMStep.json	GPU requirement flag	GpuRequired
WMStep.json	input files/modules	InputArtifacts
WMStep.json	output modules/files	OutputArtifacts
WMSplitting.json	algorithm name, e.g. FileBased	SplitMode
WMSplitting.json	files_per_job	FilesPerJob
WMSplitting.json	events_per_job	EventsPerJob
WMSplitting.json	lumis_per_job	LumisPerJob
WMSplitting.json	max_events_per_lumi or similar	MaxEventsPerLumi
WMSplitting.json	halt_job_on_file_boundaries / similar policy	RespectFileBoundaries
WMSplitting.json	run-boundary policy	RespectRunBoundaries
WMSplitting.json	lumi-boundary policy	RespectLumiBoundaries
WMSplitting.json	runtime/resource-aware hints	SplitResourceHints
any WMCore source object	source object path/id	SourceRef
any WMCore source object	original JSON fragment	SourcePayload

180.2 What this table is really saying

The important pattern is:

- some fields are **copied directly**
- some are **renamed**
- some are **normalized**
- some are **constructed**
- some are **carried only for provenance**

That is exactly why a canonical translation layer is useful.

180.3 A more realistic mini-example

Instead of this brittle direct mapping:

```
\NormalTok{WMTask["TaskName"] {-}\textgreater{}{}}
↪ Transformation["TransformationName"]}
\NormalTok{WMSplitting["events\_per\_job"] {-}\textgreater{}{}}
↪ Transformation["EventsPerJob"]}
\NormalTok{WMStep["CMSSWVersion"] {-}\textgreater{}{}} Job["CMSSWVersion"]}
```

The IR should do this:

```
\NormalTok{WMCore JSON}
\NormalTok{{ {-}\textgreater{}{}} CanonicalTask \{}}
\NormalTok{{ TaskName,}}
\NormalTok{{ TransformationType,}}
\NormalTok{{ Executable,}}
\NormalTok{{ Arguments,}}
\NormalTok{{ SoftwareVersion,}}
\NormalTok{{ MemoryMB,}}
\NormalTok{{ SplitMode,}}
\NormalTok{{ EventsPerJob,}}
\NormalTok{{ SourceRef}}
\NormalTok{{ \}}
\NormalTok{{ {-}\textgreater{}{}} DIRAC objects}
```

That gives one place to resolve:

- inheritance,
- defaults,
- naming cleanup,
- semantic conversion,
- unsupported cases.

Therefore the translation process must introduce a *canonical intermediate model*.

181 Translation IR Architecture

The Translation IR defines the canonical objects used to represent workflows.

```
\NormalTok{{ IRWorkflow }}
\NormalTok{{ workflow metadata }}
\NormalTok{{ }}
\NormalTok{{ | }}
\NormalTok{{ ▼ }}
\NormalTok{{ }}
```


WMCore Parameter	Translation IR	Description
RequestName	workflow_name	workflow identifier
InputDataset	dataset	input dataset
ProcessingString	processing_tag	processing stage identifier
CMSSWVersion	software_release	CMSSW environment
GlobalTag	conditions_tag	detector conditions
ConfigCacheID	configuration_ref	job configuration
SplittingAlgo	splitting_policy	splitting strategy
SplittingArguments	splitting_parameters	parameters of splitting

184 Dataset Representation Mapping

WMCore Object	Translation IR	Description
Dataset	dataset_name	dataset identifier
Block	block_name	dataset block
File	file_lfn	logical file name
Run	run_number	run identifier
Lumi	lumi_section	luminosity section

CMS datasets follow the hierarchy:

```

\NormalTok{dataset}
\NormalTok{  ↓}
\NormalTok{block}
\NormalTok{  ↓}
\NormalTok{file}
\NormalTok{  ↓}
\NormalTok{run}
\NormalTok{  ↓}
\NormalTok{lumi}

```

The Translation IR stores these relationships in a normalized form.

185 Task Definition Mapping

The following table maps task definitions between WMCore and the Translation IR.

WMCore Field	IR Field	Description
TaskName	task_name	name of task

WMCore Field	IR Field	Description
InputDataset	input_dataset	dataset used
SplittingAlgo	splitting_algorithm	splitting strategy
SplittingArguments	splitting_parameters	splitting configuration
ConfigCacheID	configuration_reference	CMSSW configuration
OutputDataset	output_dataset	produced dataset

186 Job Definition Mapping

The Translation IR job template captures the information required to generate DIRAC jobs.

IR Field	DIRAC Equivalent	Description
executable	job executable	application
arguments	job arguments	runtime parameters
input_files	InputSandbox	input files
output_files	OutputSandbox	produced files
software_release	environment	runtime software

187 Splitting Policy Representation

CMS workflows define explicit job partitioning policies.

```
\NormalTok{dataset}
\NormalTok{      ↓}
\NormalTok{splitting algorithm}
\NormalTok{      ↓}
\NormalTok{job definitions}
```

Example splitting modes:

- FileBased
- LumiBased
- RunBased
- EventAware

The Translation IR represents these policies using the IRSplittingPolicy object.

Splitting Mode	IR Representation
FileBased	files_per_job

Splitting Mode	IR Representation
LumiBased	lumis_per_job
RunBased	runs_per_job
EventAware	events_per_job

188 DIRAC Materialization

Once the Translation IR is constructed, it can be converted into DIRAC workflow structures.

```
\NormalTok{IRWorkflow}
\NormalTok{\downarrow}
\NormalTok{IRTask}
\NormalTok{\downarrow}
\NormalTok{DIRAC Transformation}
\NormalTok{\downarrow}
\NormalTok{DIRAC Jobs}
```

This allows the CMS workflow to be executed through the DIRAC workload infrastructure.

189 Role of the Translation IR in CMSDiracAux

The Translation IR is the **core abstraction layer** in the CMSDiracAux architecture.

```
\NormalTok{WMCore workflow}
\NormalTok{\downarrow}
\NormalTok{Translation IR}
\NormalTok{\downarrow}
\NormalTok{DIRAC execution}
```

The IR therefore enables:

- workflow portability
- architecture decoupling
- execution model translation

190 Summary

The Translation IR provides a canonical representation of workflows that separates **workflow semantics from execution infrastructure**.

This abstraction allows CMS workflows to be expressed in a form compatible with DIRAC while preserving the workflow structure and data processing semantics.

The Translation IR is therefore the key architectural component that enables interoperability between the CMS workflow management system and DIRAC execution environments.

\newpage

191 Translation IR Design

191.1 Overview

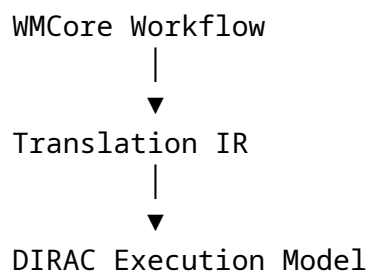
The CMSDiracAux project introduces a **Translation Intermediate Representation (Translation IR)** that acts as the semantic bridge between the CMS workflow management system (WMCore) and the DIRAC distributed workload management system.

The purpose of this layer is to decouple the **workflow description semantics** from the **execution infrastructure**.

Without such an abstraction layer, the system would require a direct mapping between WMCore workflow objects and DIRAC execution objects. Because the two systems were designed under very different architectural assumptions, such a direct mapping would be brittle, difficult to maintain, and tightly coupled to the internal structure of both systems.

The Translation IR therefore provides a **canonical workflow description** that captures the essential semantics of a CMS workflow in a form that can later be materialized into different execution environments.

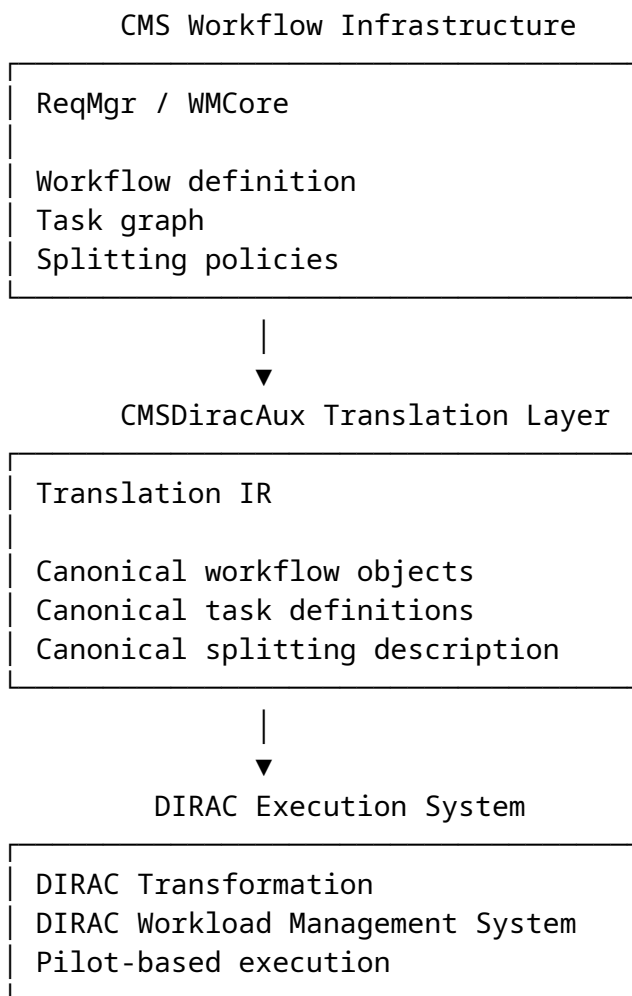
Conceptually the architecture becomes:



The Translation IR is therefore the **core abstraction layer of the CMSDiracAux architecture**.

192 Architectural Role of the Translation IR

The role of the Translation IR can be understood by examining the architectural layers involved in workflow execution.



The Translation IR acts as the **contract between the workflow domain and the execution domain**.

193 Motivation for an Intermediate Representation

The need for a canonical translation layer arises from several fundamental differences between WMCore and DIRAC.

193.1 Workflow definition philosophy

CMS workflows are defined as **explicit workflow graphs** where tasks and splitting rules are determined before execution begins.

DIRAC workflows are defined as **execution templates** where jobs are generated dynamically as data becomes available and resources are scheduled.

This difference can be summarized as:

CMS

workflow → jobs → execution

DIRAC

workflow template → runtime job generation

A direct translation between these models is difficult because the semantic structures differ significantly.

193.2 Data-driven splitting differences

CMS workflows define splitting rules at the level of **data content**, often reaching the granularity of:

```
dataset
|
▼
block
|
▼
file
|
▼
run
|
▼
luminosity section
```

DIRAC transformations generally operate at the level of **files or datasets**, not at run or luminosity section boundaries.

Because of this difference, the bookkeeping logic originally implemented in **WMBS** must be preserved during translation.

194 Design Goals

The Translation IR was designed with the following objectives.

194.0.1 1 System independence

The representation must not depend on:

```
\NormalTok{WMCore internal structures}  
\NormalTok{DIRAC transformation internals}
```

This ensures the workflow semantics can be reused across multiple execution systems.

194.0.2 2 Explicit workflow semantics

The IR must explicitly represent:

```
\NormalTok{workflow structure}  
\NormalTok{task dependencies}  
\NormalTok{runtime configuration}  
\NormalTok{data interaction}  
\NormalTok{job splitting policies}
```

These elements are implicit or distributed across multiple components in the original systems.

194.0.3 3 Compatibility with multiple execution backends

The IR must support translation into:

```
\NormalTok{DIRAC transformations}  
\NormalTok{CWL workflows}
```

Future workflow engines could also consume the same representation.

195 Translation IR Architecture

The Translation IR organizes workflow information into a small set of canonical entities.

```
\NormalTok{Workflow}  
\NormalTok{|}  
\NormalTok{| — Task}  
\NormalTok{| — RuntimeDefinition}
```

```

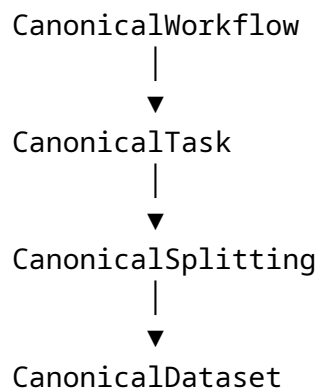
\NormalTok{      |      |— SplittingPolicy}
\NormalTok{      |      |— DataReference}
\NormalTok{      |    }
\NormalTok{      |— Dependency Graph}

```

Each entity captures a distinct aspect of the workflow semantics.

196 Conceptual Structure of the Translation IR

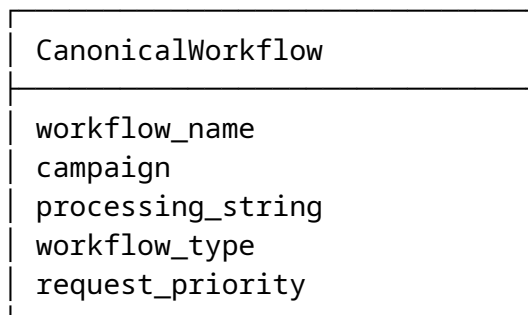
The Translation IR is composed of several canonical objects that represent the structure of a workflow.



Each object represents a layer of workflow semantics independent of the execution system.

197 Canonical Workflow Object

The CanonicalWorkflow object represents the top-level workflow definition.



This object corresponds to the high-level workflow definition stored in WMCore. It provides metadata describing the processing campaign and workflow context.

198 Canonical Task Object

Each workflow contains one or more tasks describing processing steps.

CanonicalTask
task_name
input_dataset
output_dataset
processing_step
software_version

Tasks represent the logical units of workflow processing.

In CMS workflows these correspond to WMCore task objects that define processing stages.

199 Canonical Splitting Description

The splitting object describes how data should be partitioned into jobs.

CanonicalSplitting
splitting_algorithm
files_per_job
lumis_per_job
events_per_job

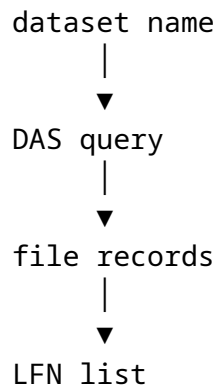
This object captures the logic required to generate job boundaries.

The information in this object later drives the **CMS-specific splitting plugin inside DIRAC transformations**.

200 Dataset Resolution

CMS workflows typically reference datasets rather than individual files.

Dataset resolution therefore occurs during the translation phase.

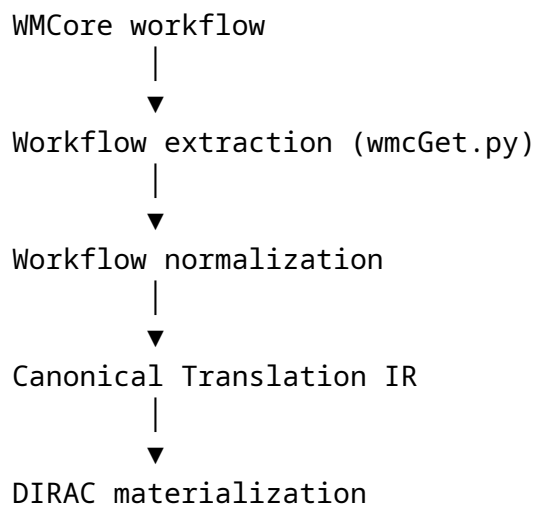


This resolution is performed using the CMS Data Aggregation System (DAS).

The resulting list of logical file names (LFNs) becomes the input for transformation splitting.

201 Translation Pipeline

The full translation process implemented in CMSDiracAux can be summarized as:



The Translation IR therefore sits at the **center of the translation pipeline**.

202 Mapping Between WMCore and Translation IR

The Translation IR is constructed by mapping WMCore workflow fields into canonical objects.

WMCore Field	Translation IR Field	Description
RequestName	workflow_name	Workflow identifier
TaskName	task_name	Task identifier
InputDataset	input_dataset	Primary dataset
SplittingAlgo	splitting_algorithm	Splitting method
FilesPerJob	files_per_job	File partition size
LumisPerJob	lumis_per_job	Luminosity section partition size

This mapping captures the workflow semantics while removing WMCore-specific implementation details.

203 Mapping to DIRAC

When exporting the IR to DIRAC the entities are mapped as follows.

Translation IR	DIRAC
Workflow	Production
Task	Transformation
RuntimeDefinition	WorkflowStep
SplittingPolicy	Transformation plugin configuration
DataReference.files	LFN list

The IR therefore provides all information required to construct a DIRAC transformation.

204 Mapping to CWL

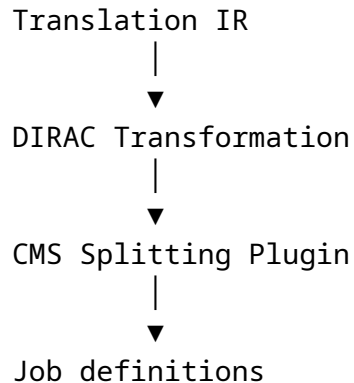
When exporting workflows to CWL the entities are mapped differently.

Translation IR	CWL
Workflow	Workflow
Task	CommandLineTool
RuntimeDefinition	baseCommand
DataReference	File inputs
DependencyGraph	Workflow DAG

This enables portable workflow descriptions compatible with CWL execution engines.

205 Translation IR and DIRAC Materialization

Once the workflow has been translated into the canonical IR, the execution structures required by DIRAC can be generated.



The CMS splitting plugin reconstructs the job boundaries defined by the original CMS workflow splitting rules.

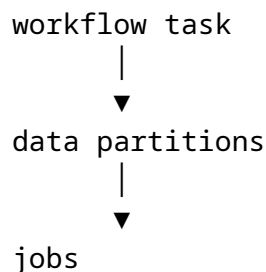
This ensures that CMS workflow semantics are preserved.

206 Relationship to WMBS

An important architectural observation arises from the design of the Translation IR.

The CMS workflow system originally relied on the **Workload Management Book-keeping System (WMBS)** to maintain the mapping between workflow tasks, data partitions, and jobs.

Even when workflows are executed in DIRAC, the system must still maintain this relationship.



Therefore the Translation IR and the DIRAC splitting plugin effectively reproduce the functionality originally implemented by WMBS.

207 Role in the CMSDiracAux Architecture

The Translation IR sits at the center of the CMSDiracAux architecture.

```
\NormalTok{WMCore workflow}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{Translation IR}
\NormalTok{|}
\NormalTok{| — DIRAC transformation}
\NormalTok{|}
\NormalTok{| — CWL workflow}
```

This design isolates the **workflow semantics** from the **execution system**, allowing workflows to be translated and reused across different distributed computing frameworks.

208 Importance of the Translation IR

The Translation IR solves the three main interoperability problems identified earlier in the report.

```
\NormalTok{workflow semantic mismatch}
\NormalTok{workflow-data interaction mismatch}
\NormalTok{runtime environment mismatch}
```

By capturing workflow semantics in a system-independent form, the IR enables deterministic translation between heterogeneous workflow management systems.

209 Design Advantages

The Translation IR provides several advantages:

209.0.1 Decoupling

Workflow semantics are separated from execution infrastructure.

209.0.2 Portability

Workflows can potentially be materialized into different execution systems.

209.0.3 Stability

Changes in either WMCore or DIRAC implementations do not require rewriting the translation logic.

209.0.4 Extensibility

Additional workflow systems could be integrated by implementing new translators.

210 Summary

The Translation IR represents the core architectural concept of CMSDiracAux.

It provides a canonical representation of CMS workflows that preserves the semantics of workflow definitions, task structures, and data splitting policies while allowing the workflows to be executed within the DIRAC distributed computing infrastructure.

By introducing this abstraction layer, CMSDiracAux demonstrates that workflows defined within WMCore can be systematically translated into execution structures compatible with DIRAC without losing the fine-grained workflow semantics required by CMS computing workflows.

\newpage

211 Dataset Resolution Model

211.1 Purpose

CMS workflows operate primarily on **datasets**, while execution infrastructures ultimately process **files**.

The CMSDiracAux translation layer must therefore resolve dataset-level workflow definitions into **file-level execution units** before jobs can be created.

This section describes the dataset resolution model used in the CMS workflow management system and the mechanism implemented in the CMSDiracAux proof-of-concept to translate dataset references into executable workload inputs.

212 CMS Data Hierarchy

CMS data are organized according to a hierarchical structure.

```
\NormalTok{dataset}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{block}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{file}
```

```

\NormalTok{    |}
\NormalTok{    ▼}
\NormalTok{run}
\NormalTok{    |}
\NormalTok{    ▼}
\NormalTok{lumi}
\NormalTok{    |}
\NormalTok{    ▼}
\NormalTok{events}

```

Each level represents a progressively finer partition of the data.

Level	Description
Dataset	logical collection of data files
Block	dataset subdivision used for storage management
File	physical storage unit
Run	data-taking period identifier
Lumi	luminosity section within a run
Event	individual recorded event

In most workflow definitions the **dataset is the primary input abstraction**.

213 Dataset References in Workflow Definitions

In CMS workflows the input data are typically specified using a dataset identifier.

Example:

```
\NormalTok{/PrimaryDataset/ProcessingString/DataTier}
```

This identifier represents a logical dataset registered in the CMS data bookkeeping systems.

Workflow tasks therefore define input data at the dataset level.

```

\NormalTok{
\NormalTok{ Workflow Task
\NormalTok{
\NormalTok{ InputDataset = dataset\_id
\NormalTok{
\NormalTok{
\NormalTok{
\NormalTok{ dataset resolution}

```

Before jobs can be created, the dataset must be resolved into the underlying files.

214 Dataset Discovery Infrastructure

Dataset resolution relies on the CMS data discovery infrastructure.

Two primary services are used:

Service	Purpose
DBS (Data Bookkeeping System)	stores dataset metadata
DAS (Data Aggregation System)	query interface for dataset discovery

These services provide information about:

- dataset existence
 - file membership
 - run/lumi metadata
 - storage locations
-

215 Data Representation in DIRAC

DIRAC transformations operate differently.

Jobs are typically generated using **explicit file-level inputs**.

These files are identified using their Logical File Names stored in the **DIRAC File Catalog**.

A transformation therefore defines:

input data query

which returns a set of LFNs.

The transformation system then generates jobs based on this file list.

Typical job splitting occurs at the level of:

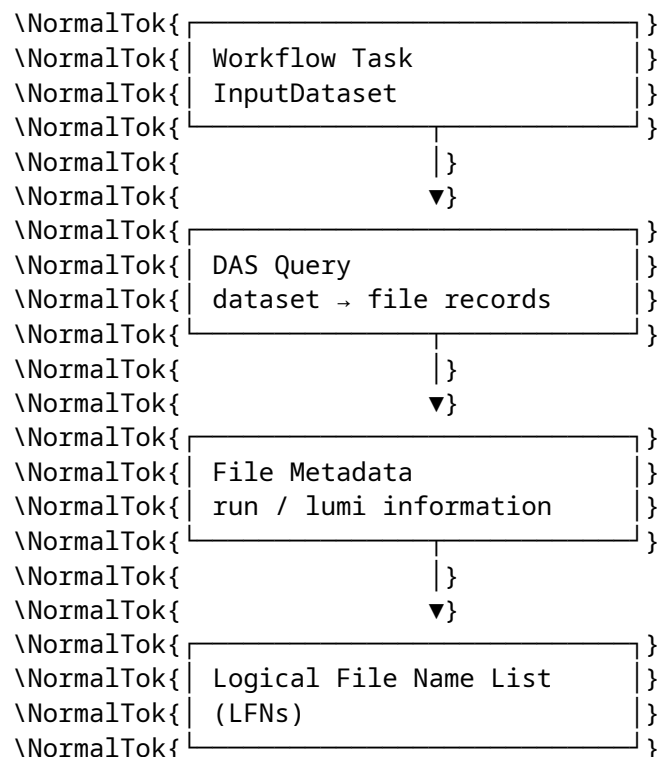
file

or groups of files.

Unlike CMS workflows, DIRAC transformations generally do not operate directly on dataset abstractions.

216 Dataset Resolution Pipeline

The dataset resolution process converts a dataset identifier into a list of logical file names (LFNs).



The resulting file list becomes the input to the job generation stage.

217 Example DAS Query

Dataset file discovery is typically performed using `dasgoclient`.

Example:

```
\NormalTok{dasgoclient {-}{-}query="file  
↪ dataset=/ExampleDataset/Processing/DataTier"}
```

The query returns metadata describing the files belonging to the dataset.

Example output fields include:

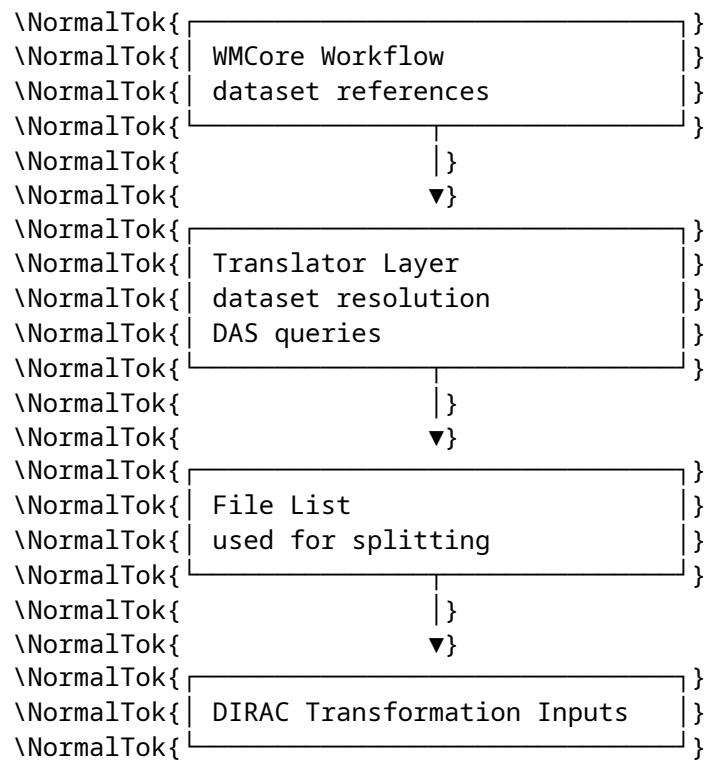
Field	Description
file.name	logical file name
file.size	file size
file.nevents	number of events
run	run identifier

Field	Description
lumi	luminosity sections

These metadata fields are later used for job partitioning.

218 Dataset Resolution in CMSDiracAux

The CMSDiracAux proof-of-concept implements dataset resolution as part of the **translator stage**.



Dataset resolution therefore occurs **before DIRAC job generation**.

219 Handling Large Datasets

CMS datasets often contain very large numbers of files.

Typical dataset sizes may include:

Dataset	Approximate file count
small analysis dataset	tens of files
typical production dataset	thousands of files
large reprocessing dataset	tens of thousands of files

For development purposes the CMSDiracAux proof-of-concept applies a temporary limit.

```
\NormalTok{maximum\_files\_per\_dataset = 20}
```

This limit prevents the generation of excessively large local artifacts during testing.

220 Integration with Splitting

Once files are resolved, job generation can proceed using splitting policies.

```
\NormalTok{dataset}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{file discovery}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{file list}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{splitting policy}
\NormalTok{|}
\NormalTok{▼}
\NormalTok{job definitions}
```

Splitting policies determine how files are partitioned into jobs.

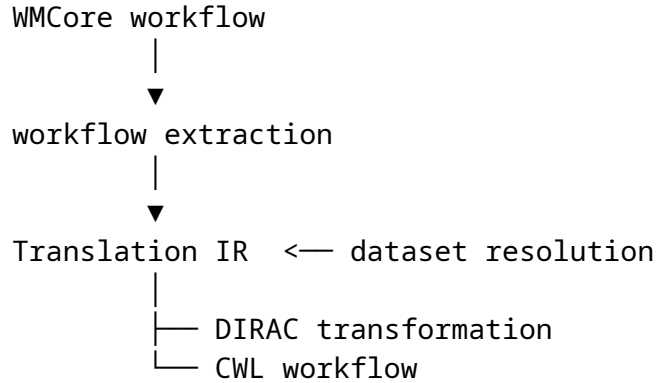
Examples include:

- file-based splitting
- run-based splitting
- lumi-based splitting
- event-aware splitting

These policies are implemented within the job generation stage.

221 Architectural Role in the CMSDiracAux System

Dataset resolution is located in the **translation layer of the CMSDiracAux architecture**.



By resolving datasets at this stage, the Translation IR becomes independent of the CMS metadata infrastructure.

This ensures that downstream execution systems only receive the **concrete file-level inputs required for execution**.

222 Implications for DIRAC Execution

DIRAC transformations typically operate on file-level inputs.

Therefore the dataset resolution stage produces the input structure required by DIRAC workflows.

```

\NormalTok{dataset}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{file list}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{DIRAC transformation tasks}
\NormalTok{  |}
\NormalTok{  ▼}
\NormalTok{DIRAC jobs}
  
```

This conversion bridges the difference between CMS dataset abstractions and DIRAC execution units.

223 Importance for Workflow Interoperability

The dataset resolution model is essential for interoperability because it bridges two fundamentally different data abstractions.

```

CMS workflow → dataset abstraction
DIRAC workflow → file-level inputs
  
```

224 Summary

CMS workflows define input data at the **dataset level**, while execution infrastructures require **file-level processing units**.

The dataset resolution model therefore performs the following steps:

1. resolve dataset identifiers using DAS queries
2. obtain file-level metadata
3. construct logical file name lists
4. provide these lists to the job generation stage

This process enables CMS workflows defined in WMCore to be translated into executable jobs compatible with DIRAC workflow infrastructures.

\newpage

225 Job Description Translation

225.1 Purpose of this section

The final stage of the workflow translation process is the generation of **executable job descriptions** that can be submitted to the DIRAC workload management system.

While the Translation IR provides a system-independent representation of the workflow semantics, DIRAC jobs must ultimately be expressed using the **DIRAC job execution model**.

This model relies on two key components:

JDL job description
+
jobDescription.xml workflow definition

This section describes how the Translation IR is transformed into these structures.

226 DIRAC Job Execution Model

In DIRAC, a job is executed through a generic runtime wrapper.

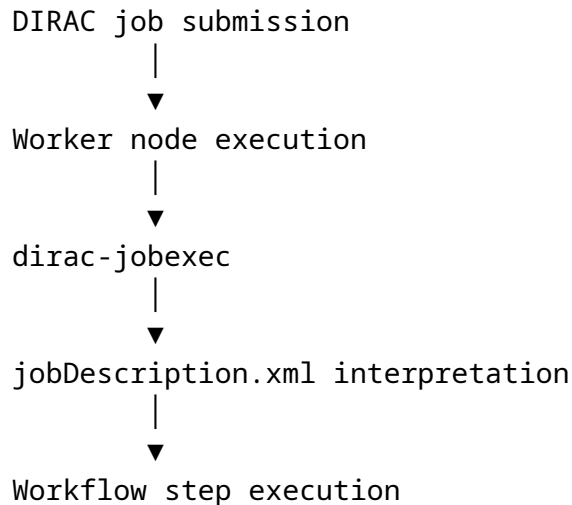
The job definition submitted to the system typically contains:

Executable
Arguments
InputSandbox
OutputSandbox
Job parameters

In most cases the executable is the DIRAC runtime entry point:

`dirac-jobexec`

The job execution flow is shown below.



The actual workflow logic is therefore contained inside the **XML workflow description** rather than in the JDL itself.

227 JDL Job Definition

DIRAC jobs are submitted using a **Job Description Language (JDL)** file.

A simplified JDL example is shown below.

```
Executable = "dirac-jobexec";
Arguments = "jobDescription.xml -o LogLevel=INFO";

InputSandbox =
{
    jobDescription.xml
};

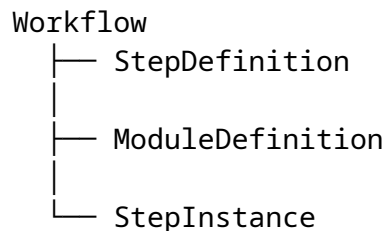
StdOutput = "std.out";
StdError = "std.err";
```

This JDL defines the runtime environment in which the workflow described in the XML file will execute.

228 Role of jobDescription.xml

The jobDescription.xml file defines the **workflow steps executed by the job**.

The structure typically includes:



Each step corresponds to a specific operation executed during the job runtime.

For example, a simple job may execute a script step defined as:

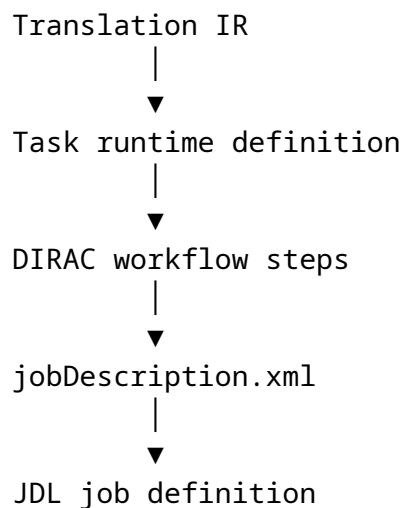
Script module

In more complex workflows, multiple steps may be chained together.

229 Job Description Translation Pipeline

The CMSDiracAux translation process converts the Translation IR into the DIRAC job structures shown above.

The pipeline can be summarized as:



This translation stage produces the executable job description required by the DIRAC runtime.

230 Mapping Translation IR to DIRAC Job Structures

The mapping between Translation IR entities and DIRAC job components is summarized below.

Translation IR	DIRAC Representation
Workflow	Production
Task	Transformation
RuntimeDefinition	Workflow step
SplittingPolicy	Transformation plugin configuration
DataReference.files	Input data list
JobTemplate	JDL + jobDescription.xml

This mapping ensures that the semantic information captured in the Translation IR is preserved during execution.

231 Runtime Entry Point

During execution, the worker node runs the following command:

```
dirac-jobexec jobDescription.xml
```

The `dirac-jobexec` program performs the following tasks:

1. Initializes the DIRAC runtime environment
2. Parses the `jobDescription.xml` file
3. Executes the defined workflow steps
4. Collects job output and logs

This architecture separates **workflow execution logic** from the **job submission description**.

232 Runtime Environment Considerations

One important difference between CMS and DIRAC execution models concerns **runtime environment distribution**.

In CMS workflows:

runtime environment distributed with job sandbox

In DIRAC:

runtime environment provided via CVMFS

This difference requires the translation layer to ensure that the required runtime environment is available to the job at execution time.

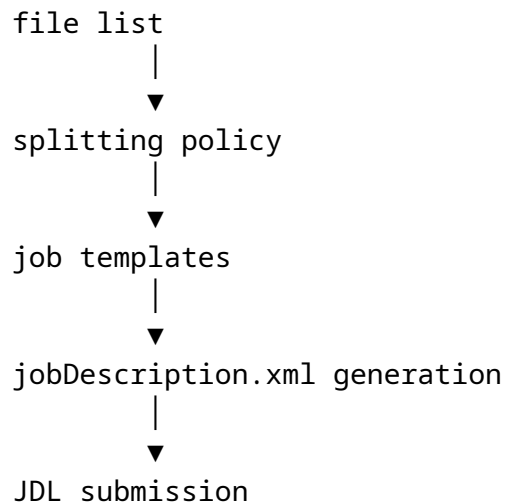
In practice, this may involve:

- pre-installed experiment software
- container execution environments
- runtime bootstrap scripts

233 Interaction with Job Splitting

Once datasets have been resolved into file lists, the job translation stage can construct job definitions according to the splitting policy defined in the Translation IR.

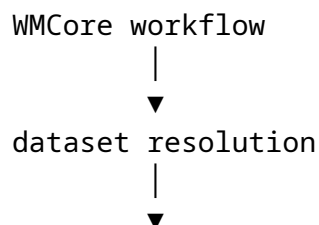
The process becomes:

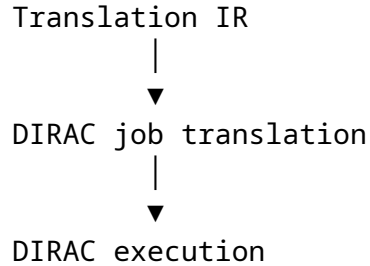


Each generated job therefore corresponds to a specific subset of input files.

234 Role in the CMSDiracAux Architecture

Job description translation represents the **final stage before execution** in the CMS-DiracAux architecture.





This stage converts the abstract workflow representation into concrete executable jobs.

235 Limitations of the Prototype

The CMSDiracAux prototype implements a simplified job translation process.

Several aspects of full CMS workflows are not yet fully represented.

These include:

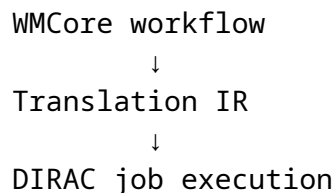
- complex runtime configuration
- multi-step task chains
- advanced job dependency management

However, the prototype demonstrates that the Translation IR contains sufficient information to construct valid DIRAC job descriptions.

236 Importance for Workflow Interoperability

The job description translation stage demonstrates that workflows defined in the CMS workflow management system can be transformed into executable jobs for the DIRAC runtime.

This stage therefore represents the **final step in the deterministic translation pipeline** implemented by the CMSDiracAux project.



The success of this stage confirms that the Translation IR provides a sufficiently expressive representation of workflow semantics to enable interoperability between the two systems.

\newpage

237 Job Runtime Differences Between CMS and DIRAC

This document captures the **runtime execution differences between CMS workflows and DIRAC**, the role of **cmsRun**, **PSet tweaks**, **sandbox limitations**, and how the **Translation IR and runtime metadata mechanisms interact**.

237.1 Purpose

While the CMSDiracAux project primarily focuses on translating workflow descriptions from WMCore to DIRAC-compatible execution structures, a significant portion of the interoperability challenge lies at the **runtime execution layer**.

CMS and DIRAC jobs differ not only in how workflows are defined, but also in how runtime environments are constructed and how job semantics are transported to worker nodes.

This document analyzes:

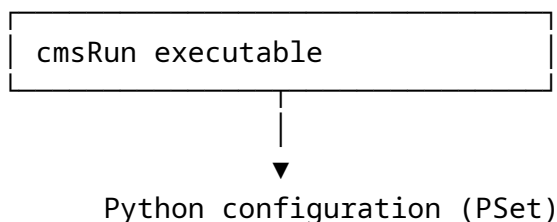
- runtime requirements of the CMS **cmsRun** process
- the role of **PSet configuration and runtime tweaks**
- runtime environment dependencies on **WMCore infrastructure** and **CMSSW**
- the constraints imposed by **DIRAC job sandbox distribution**
- the relationship between the **Translation IR layer** and legacy `__CMSJobParameter__` job metadata
- potential architectural improvements to runtime code distribution.

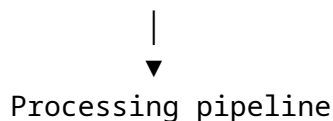
238 CMS Runtime Execution Model

In the CMS workflow system, the executable component of a job is typically the **CMSSW framework executable cmsRun**.

However, **cmsRun** itself contains very little intrinsic logic about the specific job being executed. Instead, the behavior of the job is entirely determined by a **Python configuration file (PSet)**.

Conceptually:





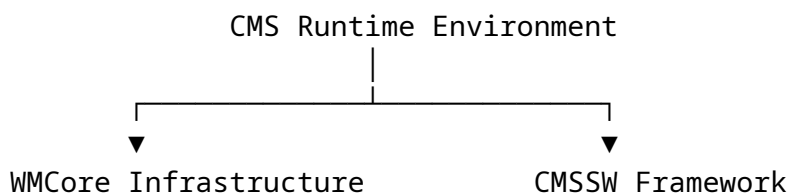
The configuration defines:

- modules to be executed
- execution order
- input sources
- output modules
- physics parameters.

Thus, **the runtime semantics of a CMS job reside in its configuration**, not in the executable itself.

239 Runtime Dependencies of CMS Jobs

A CMS job environment has two primary dependency domains.



239.0.1 WMCORE Infrastructure

WMCORE provides the workflow-level runtime context:

- workflow definition
- job package
- splitting metadata
- job-specific parameters
- PSet tweaks.

These components determine:

which unit of work the job represents

239.0.2 CMSSW Framework

CMSSW provides the physics execution environment:

- framework libraries
- modules and algorithms
- data processing pipelines
- runtime configuration interpreter.

These components determine:

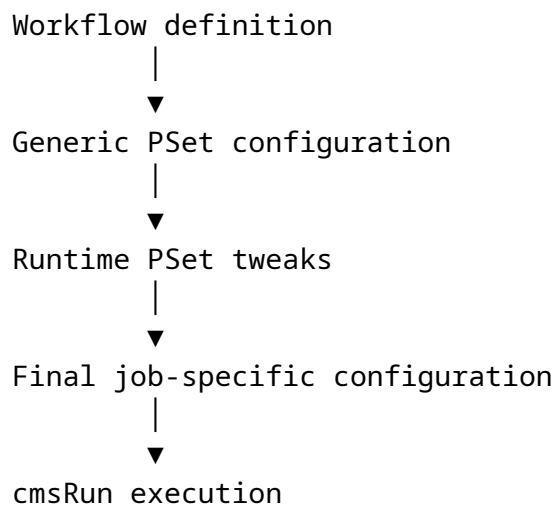
how the job processes the assigned data

Both sides must be satisfied for a CMS job to execute correctly.

240 Job Identity and PSet Tweaks

A CMS job becomes a **unique runtime instance** only after PSet modifications are applied.

Conceptually:



PSet tweaks typically encode:

- input file lists
- run/lumi boundaries
- event limits
- output dataset metadata
- runtime parameters derived from workflow splitting.

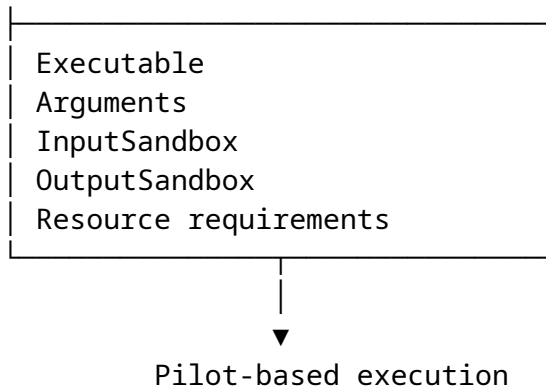
Thus the final runtime configuration of each job is constructed dynamically.

241 DIRAC Runtime Execution Model

DIRAC jobs follow a significantly different runtime philosophy.

A DIRAC job is defined primarily through a **Job Description Language (JDL)**.

DIRAC Job Description (JDL)



The pilot job retrieves the payload from the central system and executes it on the worker node.

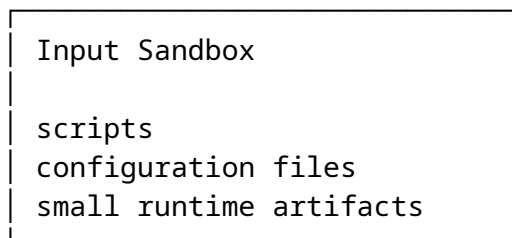
DIRAC therefore assumes:

- small job descriptions
- limited sandbox payloads
- runtime software accessible externally (e.g. via CVMFS).

242 Sandbox Distribution Constraints

One of the key constraints encountered by CMSDiracAux arises from the limitations of the **DIRAC sandbox model**.

The input sandbox is designed to transport small files required for job execution.



Large software bundles are not expected to be transported through this mechanism.

This constraint becomes problematic for CMS workflows because the runtime environment may include:

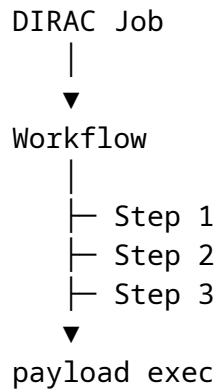
- workflow management scripts
- job package artifacts
- PSet configurations
- metadata files.

Transporting these artifacts through the sandbox is inefficient and potentially incompatible with the DIRAC execution model.

243 DIRAC Workflow Construction

In DIRAC a **Job** can contain a **Workflow**, which is a sequence of **Steps** executed on the worker node.

Conceptually:



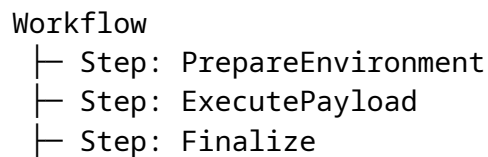
At runtime this is represented by:

jobDescription.xml

which defines:

- step order
- executable per step
- environment
- parameter passing

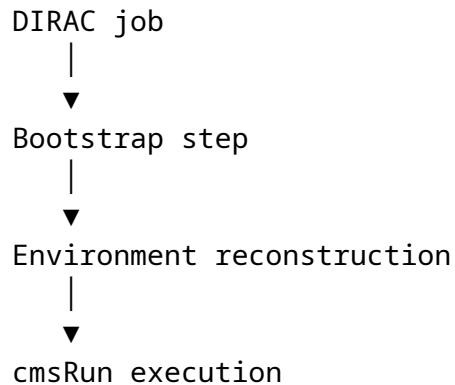
Example conceptual structure:



244 Bootstrap Execution Strategy

To address sandbox constraints, CMSDiracAux employs a **bootstrap execution model**.

A DIRAC job initially runs a lightweight bootstrap script which reconstructs the CMS runtime environment.



Example conceptual sequence:

- Step 1 prepare runtime environment
 - Step 2 retrieve workflow artifacts
 - Step 3 reconstruct job configuration
 - Step 4 run cmsRun
-

245 Role of `__CMSJobParameter__`

In early CMSDiracAux prototypes, CMS job metadata was transported into the DIRAC workflow object using the special parameter prefix:

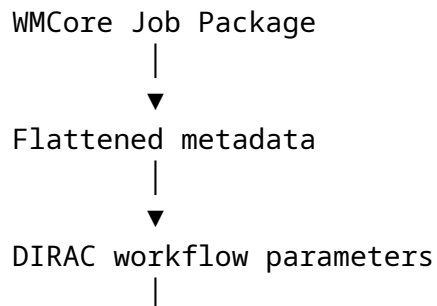
`__CMSJobParameter__`

These parameters preserved job-specific information extracted from the WMCore job package.

Examples include:

- workflow identifiers
- task names
- dataset partitions
- run/lumi ranges
- configuration references.

Conceptually:

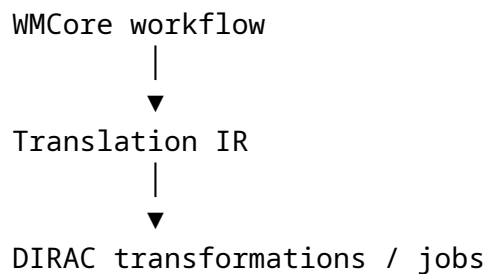


▼
Runtime bootstrap logic

This mechanism provided a simple way to preserve CMS job semantics within DIRAC job descriptions.

246 Translation IR as a Replacement Layer

CMSDiracAux introduces a **Translation Intermediate Representation (IR)** that formalizes the translation between WMCore workflows and DIRAC execution structures.



The IR captures structured information such as:

- workflow steps
- splitting rules
- input datasets
- resource hints
- executable definitions.

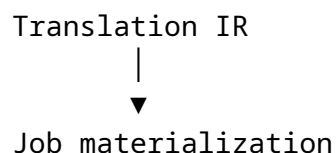
This allows the translation layer to preserve semantics without relying on flat meta-data structures.

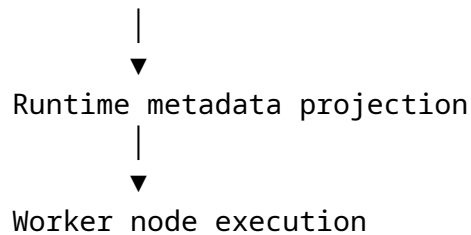
247 Relationship Between IR and Runtime Parameters

The Translation IR replaces the architectural role previously played by `__CMSJobParameter__`.

However, runtime jobs still require **job-specific metadata**.

Therefore the relationship becomes:





In this architecture:

- the IR acts as the **canonical semantic representation**
 - runtime parameters become a **projection of the IR**, not a primary source of truth.
-

248 How This Relates to CMSDiracAux

The `createCMSJob()` method implicitly tries to construct such a **DIRAC workflow**.

In the prototype the steps look like:

```
Step 1  clone CMSDiracAux repo
Step 2  source env.sh
Step 3  run Startup.py
```

So the DIRAC job acts as a **bootstrap wrapper**, while the actual CMS logic happens inside that wrapper.

249 Why This Is Needed

CMS runtime is fundamentally different from DIRAC runtime.

249.0.1 DIRAC runtime model

DIRAC assumes:

```
Executable
Arguments
InputSandbox
OutputSandbox
```

Everything else is expected to be:

- small
- pre-installed
- or accessible through CVMFS.

249.0.2 CMS runtime model

CMS jobs expect a **rich runtime payload** containing:

```
step_cfg.py
WMWorkload.pkl
JobPackage.pkl
CMSSW configuration
runtime metadata
```

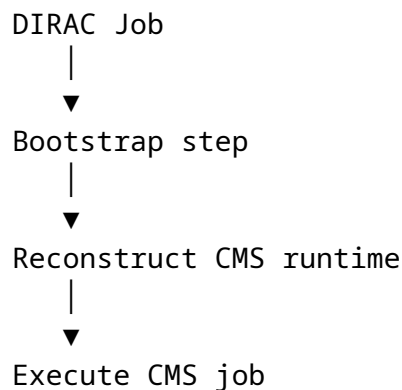
These objects define the **job semantics**, not just the executable.

This is one of the central architectural constraints for CMSDiracAux.

250 The Resulting Runtime Strategy

Because of this constraint the runtime must be **reconstructed on the worker node** rather than shipped entirely through the sandbox.

The strategy becomes:



Example conceptual pipeline:

```
Step 1   prepare environment
Step 2   retrieve job package
Step 3   execute CMSSW job
```

This matches exactly the idea of the **DIRAC workflow step chain**.

251 How the `__CMSJobParameter__` Fields Fit

Those parameters provide the **minimal metadata needed to reconstruct the CMS job context**.

They typically encode things like:

- task name
- dataset slice
- run/lumi partition
- workflow identifiers
- configuration references

At runtime the bootstrap script can use them to:

- recover WMCore job semantics
- load `JobPackage.pkl`
- configure CMSSW job

So they effectively carry **CMS job identity across the DIRAC boundary**.

252 Architectural Interpretation

This leads to the following layered runtime model.

DIRAC layer

- job
- workflow
- steps
- pilot execution

CMS layer

- WMWorkload
- JobPackage
- CMSSW runtime
- dataset partition

The bootstrap workflow bridges these two layers.

253 Why This Matters for CMSDiracAux

The real difficulty is not translating workflows.

The real difficulty is **translating runtime semantics**.

CMS jobs assume:

rich runtime state

DIRAC assumes:

minimal job description

The CMSDiracAux approach therefore relies on:

DIRAC workflow bootstrap

+

runtime reconstruction

+

metadata injection

rather than trying to directly map CMS jobs to plain DIRAC jobs.

254 Key Architectural Constraint

DIRAC cannot freely distribute arbitrary runtime payloads via job sandboxes.

This forces the architecture to use:

runtime reconstruction

or external retrieval

instead of direct sandbox shipping.

This constraint strongly influences the **design of the translation layer and runtime bootstrap logic**.

It also relates to the need of reinventing WMBS. The key point is that **WMBS is not just a historical artifact of CMS workflow management**. It exists because CMS workflows require a **level of job-data association that the storage hierarchy alone cannot provide**. When one moves execution to DIRAC, that requirement does not disappear — it simply moves somewhere else in the architecture.

Below is the reasoning step by step.

255 1. What WMBS Actually Does

WMBS (Workload Management Bookkeeping System) is often described as a job book-keeping database, but its **real architectural role** is deeper.

It maintains the mapping:

```
\NormalTok{workflow task}  
\NormalTok{      |}  
\NormalTok{      ▼}  
\NormalTok{data units}  
\NormalTok{(run / lumi / events)}  
\NormalTok{      |}  
\NormalTok{      ▼}  
\NormalTok{jobs}
```

Conceptually:

```
dataset  
  |  
  ▼  
files discovered  
  |  
  ▼  
runs/lumis extracted  
  |  
  ▼  
splitting rules  
  |  
  ▼  
job definitions
```

The important point is that the **job boundaries are not tied to storage objects**.

A job might process:

```
lumi 101-120  
from file A and B
```

That information **cannot be recovered from the dataset structure alone**.

Therefore WMBS must explicitly store:

job ↔ data content mapping

256 2. DIRAC's Native Model

DIRAC assumes a much simpler relationship between data and jobs.

Conceptually:

```
file
|
▼
job
```

In other words:

```
\NormalTok{job input = file list}
```

DIRAC transformations typically operate at the **file level**.

Example:

```
fileA.root → job1
fileB.root → job2
fileC.root → job3
```

No additional bookkeeping layer is required because **the storage unit equals the work unit**.

257 3. The Core Mismatch

CMS workloads require splitting **below the storage unit**.

DIRAC workloads assume splitting **at the storage unit**.

This mismatch looks like this:

CMS

```
dataset
|
▼
file
|
▼
run
|
▼
lumi
|
▼
job
```

versus

DIRAC

file

|



job

258 4. What Happens If WMBS is Removed

If one simply translates CMS workflows to file-level jobs and remove run/lumi splitting:

1. files contain **very uneven event counts**
2. job runtimes become unpredictable
3. resource scheduling becomes unstable
4. long jobs block resource pools
5. short jobs cause fragmentation

In other words:

runtime predictability collapses

This is precisely the reason CMS introduced WMBS originally.

259 5. What Happens in CMSDiracAux

When CMS workflows are executed through DIRAC, the **same problem reappears**.

DIRAC does not know:

run

lumi

event boundaries

Therefore it cannot perform the required splitting natively.

So the system must introduce a **bookkeeping mechanism again**.

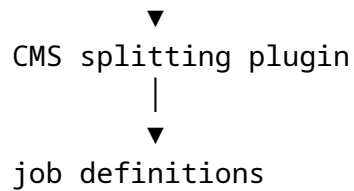
But this time it cannot live inside WMBS.

Instead it must live inside the **DIRAC splitting layer**.

Conceptually:

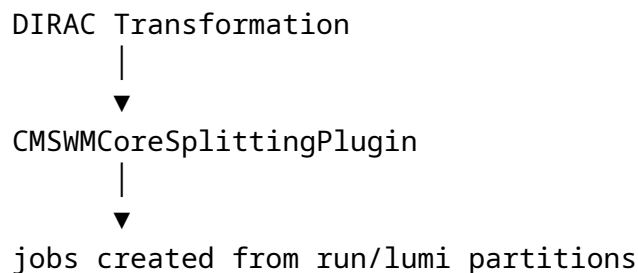
DIRAC Transformation

|



260 6. Where the WMBS Logic Moves

In CMSDiracAux the WMBS functionality effectively migrates to the **DIRAC transformation plugin**.



This plugin must perform the same functions that WMBS originally did:

- dataset resolution
- run/lumi extraction
- splitting policy application
- job-data bookkeeping

In other words:

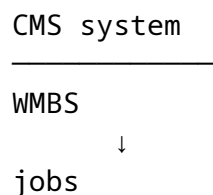
WMBS logic → DIRAC plugin

261 7. Architectural Consequence

This leads to a very important architectural conclusion.

Even if the **central scheduling system changes**, the need for WMBS-like functionality **does not disappear**.

It simply moves layers.



CMSDiracAux system

DIRAC Transformation Plugin



jobs

So the architecture becomes:

WMCore workflow



Translation IR



DIRAC transformation



CMS splitting plugin
(WMBS logic reborn)



jobs

262 8. Impact on Scheduling

This also explains why the splitting granularity matters for resource scheduling.

If job lengths are predictable:

scheduler efficiency ↑

resource utilization ↑

If jobs correspond to arbitrary file sizes:

runtime variance ↑

resource fragmentation ↑

scheduler predictability ↓

Therefore the **fine-grained splitting logic must remain part of the system.**

263 9. Final Architectural Insight

And to be more precise we can safely state that:

WMBS exists because CMS workflows require job splitting below the storage abstraction level. Any alternative execution infrastructure must therefore reintroduce equivalent bookkeeping if the same splitting granularity is preserved.

This is one of the **most important architectural constraints**.

264 How things change with DiracX

With **DIRACX**, the need for **WMBS-like logic does not disappear** if CMS keeps run/lumi/event-level splitting. What changes is **where that logic would plug in** and how naturally it fits the surrounding architecture. While we can be reasonably confident about the high-level direction, some details are still blurry because DIRACX is still evolving and some workflow/task pieces have been discussed only internally as issues/prototypes, etc., rather than frozen as long-stable user-facing architecture. (GitHub)

In classic DIRAC, the path is roughly:

```
\NormalTok{Production}  
\NormalTok{    ↓}  
\NormalTok{Transformation}  
\NormalTok{    ↓}  
\NormalTok{WMS / Matcher / Task Queues}  
\NormalTok{    ↓}  
\NormalTok{Pilots}
```

That stack is centered on **data-driven transformations** and **pilot-based pull scheduling**, with task queues and matcher logic inside the WMS. DIRAC's own docs describe the WMS in terms of pilot jobs, task queues, matching, and the core WMS databases such as JobDB, TaskQueueDB, and PilotAgentsDB. (DIRAC Documentation)

DIRACX is moving toward a more explicit **workflow / task / service API** model. Public DIRACX discussions and the `dirac-cwl` plan point toward first-class submission endpoints like `POST /jobs` and `POST /productions`, a workflow/task database, and a later transition where transformations and optimizers are handled through DIRACX task-aware services rather than the older tightly coupled classic stack. That means the conceptual shape becomes closer to:

```
\NormalTok{Workflow}  
\NormalTok{    ↓}  
\NormalTok{Tasks}  
\NormalTok{    ↓}  
\NormalTok{Scheduling services}
```

```
\NormalTok{    ↓}  
\NormalTok{Pilots / execution}
```

rather than classic DIRAC's Production → Transformation → Jobs emphasis. This is exactly why DIRACX looks architecturally closer to CMS than classic DIRAC does. (GitHub)

So if CMSDiracAux were retargeted from classic DIRAC to DIRACX, the **core requirement** would stay the same:

```
\NormalTok{CMS needs:}  
\NormalTok{dataset → file → run/lumi/event{-}aware partitioning}
```

and therefore it would still need a component that performs:

- dataset/file discovery
- run/lumi-aware partitioning
- job-data-content bookkeeping
- predictable runtime-oriented workload shaping.

That requirement comes from the CMS side, not from DIRAC's implementation details. DIRACX would not remove that need; it would only provide a cleaner architectural place to host it, likely as part of a workflow/task service layer or a task-generation component instead of burying it inside classic transformation-plugin mechanics. That is an inference from the currently visible DIRACX direction, not a guaranteed finalized product feature. (GitHub)

In other words, with classic DIRAC:

```
\NormalTok{WMBS logic must be reborn inside DIRAC transformation plugins}
```

With DIRACX:

```
\NormalTok{WMBS logic must be reborn inside DIRACX workflow/task generation  
↔ services}
```

or whatever the final DIRACX task abstraction stabilizes into. The essential logic does not go away; it just migrates into a more natural abstraction layer because DIRACX explicitly talks in terms of workflows and tasks rather than only productions and transformations. (GitHub)

The **sandbox/runtime-distribution constraint** also does not fundamentally disappear. Classic DIRAC sandbox handling is still designed around relatively small input/output sandboxes, with the WMS and pilot infrastructure expecting that large software payloads live elsewhere. DIRAC pilot bootstrapping remains centered on downloading pilot bootstrap material and then retrieving work, not on arbitrarily shipping large experiment-specific runtime bundles as a general solution. DIRACX discussions around CWL and new job wrappers suggest more flexible ways to describe where executables and inputs come from, but that is not the same as "DIRACX natively solves CMS-style random runtime bundle distribution." This should be treated as **partially improved in principle, but not solved by default**. (DIRAC Documentation)

So for CMSDiracAux, the runtime story under DIRACX would likely shift from:

```
\NormalTok{classic DIRAC bootstrap job}
\NormalTok{    ↓}
\NormalTok{reconstruct CMS runtime on worker node}
```

to something more declarative, possibly closer to:

```
\NormalTok{workflow/task description}
\NormalTok{    ↓}
\NormalTok{explicit description of executable + inputs + metadata source}
\NormalTok{    ↓}
\NormalTok{scheduler / wrapper materializes runtime}
```

especially if CWL-backed workflows become the dominant path. But even then, CMS still has the same hard problem: its runtime semantics and job partitioning are not naturally file-level or minimal-sandbox-level. So the CMS-specific metadata and book-keeping layer is still required. (GitHub)

The cleanest summary is this:

```
\NormalTok{Classic DIRAC:}
\NormalTok{WMBS{-}like logic would have to be reborn mainly in transformation}
↔ \NormalTok{plugins.}
```

```
\NormalTok{DIRACX:}
\NormalTok{WMBS{-}like logic would still have to exist,}
\NormalTok{but could likely live in a more natural workflow/task layer.}
```

That is why DIRACX is promising for future inter-operational architecture for CMS integration: it reduces the **abstraction mismatch** between CMS and DIRAC, but it does **not** remove the deep CMS requirement for fine-grained data-content bookkeeping and runtime shaping. That part remains a CMS-driven constraint regardless of the execution backend.

265 Runtime constraints for CMS processes

To the picture discussed so far, one must not miss the runtime enforced requirements on the `cmsRun` process and the additional information which must be provided by its Parameter Set (PSet) files and the PSet tweaks application mechanisms.

Putting `cmsRun`, the runtime PSet tweaks, and the two code-distribution methods for the two different runtime code bundles into the picture makes the architectural constraint even more complex.

At runtime, a CMS job is not just “an executable plus some files.” `cmsRun` is a **single executable whose behavior is fully determined by a Python configuration file**; that configuration defines which modules are loaded, in what order they run, and with which parameters, and it is fixed at the beginning of the job. In other words,

the runtime semantics of the job live in the configuration, not in the executable alone. (TWiki)

That immediately creates a two-sided dependency for the CMS runtime environment:

- On one side, the job depends on **WMCore-side workflow/runtime artifacts**: the workflow description, job package, and the per-job information produced by splitting and packaging.
- On the other side, the job depends on **CMSSW-side physics software**: the actual framework, modules, release environment, and the Python configuration that cmsRun will execute. (GitHub)

That is why the CMS runtime environment is structurally constrained by **both** WMCore and CMSSW. WMCore determines *which exact unit of work this job is*, while CMSSW determines *how that unit of work is executed physically and logically*.

265.1 Where PSet tweaks fit

The PSet layer is especially important because it is the point where a generic workflow/job package becomes a **specific job instance**. The CMS configuration model is Python-based and is meant to be modified/configured before execution; SetupCMSSWPset.py in WMCore explicitly exists to load the shipped PSet and mock or adjust values that depend on runtime context. That is exactly the kind of place where job-specific tweaks are applied. (GitHub)

So in practice, the runtime chain is conceptually:

```
\NormalTok{WMCore job identity}
\NormalTok{    |}
\NormalTok{    ▼}
\NormalTok{PSet / PSetTweaks}
\NormalTok{    |}
\NormalTok{    ▼}
\NormalTok{cmsRun executes specific job instance}
```

The important consequence is that **PSet distribution is not just configuration convenience**. It is part of the job identity and part of the semantics of the split.

265.2 How this interacts with DIRAC / DIRACX

Classic DIRAC is comfortable with:

- a relatively small input sandbox
- a job description
- a runtime that is otherwise already available on the worker node or accessible through standard mechanisms. (DIRAC Documentation)

Classic DIRAC is **not naturally built around shipping large arbitrary experiment-specific runtime bundles with every job**, and even in the newer

DIRACX/CWL discussions, sandbox/input handling is still being clarified and standardized rather than already solved as a stable, final model. That means this whole area is still somewhat blurry on the DIRACX side. (GitHub)

So the core problem for CMSDiracAux remains:

```
\NormalTok{cmsRun needs:}
\NormalTok{ CMSSW environment}
\NormalTok{ + job{-}specific PSet semantics}
\NormalTok{ + WMCore{-}derived job identity}

\NormalTok{DIRAC expects:}
\NormalTok{ small job description}
\NormalTok{ + modest sandbox}
\NormalTok{ + runtime available externally}
```

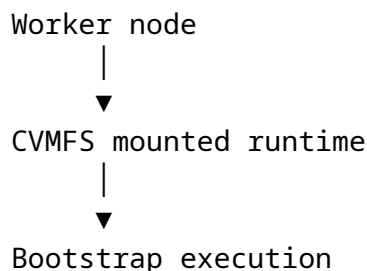
That mismatch does not disappear just because the execution backend changes.

266 Paths for change of the CMS runtime environment

Two architectural improvements can significantly reduce runtime friction.

266.1 1. Distributing the Core Runtime Bundle via CVMFS

Instead of shipping the workflow runtime environment with each job sandbox, the common runtime bundle could be distributed via CVMFS.



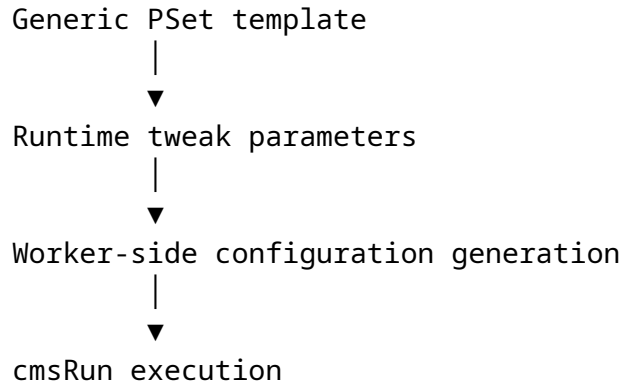
Advantages include:

- smaller job sandboxes
- consistent runtime environment
- centralized version management.

266.2 2. Reconstructing Per-Job PSet Configurations at Runtime

Currently, each job often receives a fully materialized PSet configuration.

A more scalable approach is to distribute **PSet templates and tweak parameters**, allowing the final configuration to be generated on the worker node.

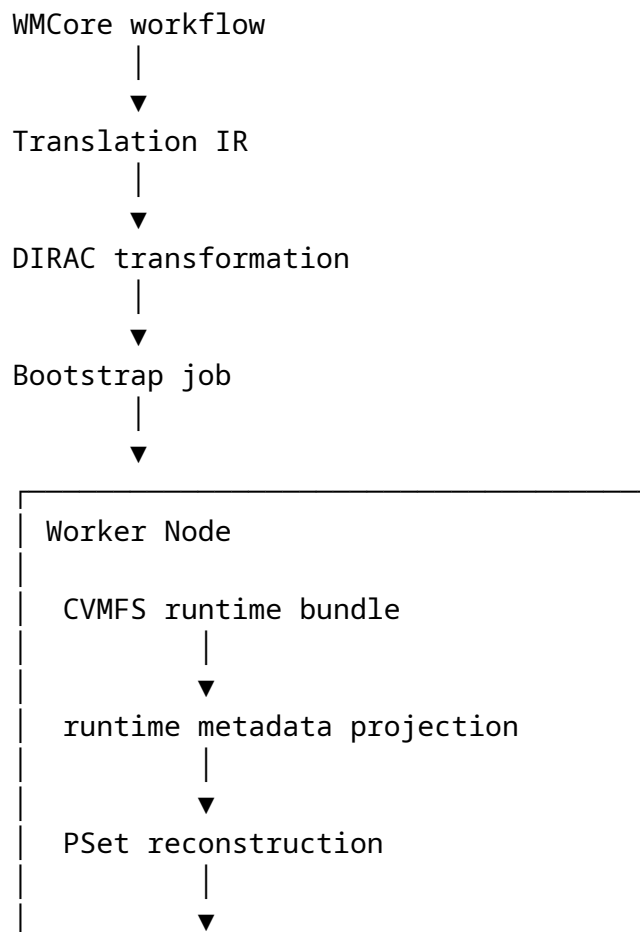


This reduces sandbox payload size and aligns better with DIRAC's execution model.

267 Impact on the CMSDiracAux Architecture

Combining the above improvements results in the following runtime architecture.

CMSDiracAux Runtime Model





Changing the CMS jobs runtime environment would help, but this must be a complex effort with different focus areas. The first being a change of the CMS Runtime code distribution methods. Lets elaborate on the two already mentioned directions.

- A change of how we distribute the main Workflow Management System dependent core bundle, by uploading it cvmfs instead distributing it with the job sandboxes.
- A change of the PSet configuration per job distribution

267.1 Direction 1: move the WMCore-dependent core bundle to CVMFS

This would help **significantly**, but only partially.

If the main workflow-management-dependent bundle — the generic WMCore runtime/bootstrap layer, helper scripts, unpackers, startup logic, maybe common workflow handling code — were moved to CVMFS instead of being shipped through sandboxes, then one major pressure point would be removed:

- the sandbox gets smaller
- the runtime becomes more reproducible
- common code is versioned once centrally
- jobs stop redundantly shipping the same management code repeatedly.

This is exactly aligned with the DIRAC expectation that most runtime software should already be available in shared infrastructure rather than being pushed with each job. (DIRAC Documentation)

Architecturally, that would transform the runtime problem from:

```
\NormalTok{ship WMCore logic + ship job config + run cmsRun}
```

to:

```
\NormalTok{mount WMCore logic from CVMFS + inject job config + run cmsRun}
```

That is a real improvement, because it removes the “generic runtime bundle” from the per-job transport problem.

But it does **not** eliminate the need for per-job semantics. The worker node would still need to know:

- which exact split this job corresponds to
- which job package / workflow slice it represents
- which runtime parameters or PSet modifications make it unique.

So moving the generic bundle to CVMFS helps with **distribution overhead and reproducibility**, but it does not solve the CMS-specific **job individuation problem**.

267.2 Direction 2: rethink per-job PSet distribution

Solving this problem is a difficult task. Because `cmsRun` behavior is driven by configuration, the PSet is not just “small metadata”; it is often the actual expression of the job instance. If CMS continue shipping a distinct per-job PSet through the sandbox, then even after moving the generic WMCore code to CVMFS, the remaining per-job transport burden is still semantically essential.

There are two broad possibilities in addressing this problem:

267.2.1 A. Keep shipping the per-job PSet

This preserves exact semantics and is operationally straightforward, but it keeps the job tied to sandbox-style distribution. That means the architecture still depends on per-job transport of the final runtime description.

267.2.2 B. Generate or reconstruct the per-job PSet on the worker node

This would help much more architecturally, because then the sandbox would no longer need to carry the final per-job configuration. Instead, the job would carry only:

- stable configuration references
- job-specific parameter values
- split metadata
- maybe a small templating/tweak description.

Then the worker-side bootstrap could materialize the actual final PSet just before `cmsRun`.

This would change the runtime model from:

```
\NormalTok{ship final PSet}
```

to:

```
\NormalTok{ship PSet recipe / tweak inputs}
\NormalTok{      ↓}
\NormalTok{reconstruct final PSet at runtime}
```

For CMSDiracAux, that is the more strategic direction, because it aligns much better with both classic DIRAC and the likely DIRACX direction: keep large/common/runtime-stable things externalized, and keep only the truly job-specific state flowing per job.

267.3 How much do these two changes help?

A fair qualitative assessment is:

267.3.1 Move generic WMCore-dependent runtime bundle to CVMFS

Helps a lot operationally. It reduces sandbox dependency substantially and aligns the CMS runtime better with the DIRAC model. But it does **not** remove the need for WMBS-like bookkeeping or job-specific runtime construction.

267.3.2 Move from per-job shipped PSet to per-job reconstructed PSet

Helps even more architecturally. It reduces per-job payload coupling and moves the system closer to a declarative execution model. But it requires a robust way to encode and reconstruct the job-specific semantics correctly.

So if ranked by effect:

```
\NormalTok{1. CVMFS for generic WMCore bundle}
\NormalTok{    = strong operational improvement}

\NormalTok{2. runtime reconstruction of per{-}job PSet}
\NormalTok{    = stronger architectural improvement}
```

The second one is more transformative because it attacks the most CMS-specific piece of the runtime.

267.4 What does not change, even after both improvements

Even if both changes are adopted, one thing remains true:

CMS still needs a component that determines **which exact data-content slice** a job processes and how that becomes runtime configuration. That requirement comes from CMS splitting granularity, not from the transport mechanism. So even with:

- generic bundle on CVMFS
- per-job PSet reconstructed at runtime

the new system would still need a WMBS-like layer — whether in classic DIRAC transformation plugins or, more naturally, in a DIRACX workflow/task-generation layer — to preserve the mapping:

```
\NormalTok{workflow/task}
\NormalTok{    ↓}
\NormalTok{run/lumi/event partition}
\NormalTok{    ↓}
\NormalTok{job{-}specific runtime semantics}
```

That is the real invariant.

267.5 DIRACX improvement

The future DIRACX system will help in this situation mainly by providing a **cleaner architectural home** for this logic. Public DIRACX and `dirac-cwl` discussions point toward explicit workflow/task services, job/production endpoints, and more modern

handling of workflow metadata, but sandbox/input-data behavior is still under discussion. (GitHub)

- **DIRACX does not remove the CMS runtime problem**

- but it may make it easier to express the solution as:

- workflow/task metadata
- declarative input descriptions
- runtime-side materialization of job specifics.

That is a meaningful improvement in architecture, but not a magic removal of the CMS-specific constraints.

267.6 Bottom line

The CMS runtime environment is constrained simultaneously by:

- **WMCore infrastructure semantics** because the job must know what exact split-produced workload instance it is

and

- **CMSSW framework semantics** because cmsRun needs a concrete, job-specific Python configuration to define the processing graph. (TWiki)

Changing distribution in those two directions would help, but asymmetrically:

- putting the **generic WMCore-dependent core bundle on CVMFS** removes a large operational incompatibility with DIRAC-style execution
- changing **per-job PSet handling** is the deeper architectural move, because that is where CMS job uniqueness actually lives.

The strongest long-term shape is:

```
\NormalTok{common runtime logic}
\NormalTok{    → CVMFS}

\NormalTok{job{-}specific semantics}
\NormalTok{    → lightweight metadata / tweak inputs}

\NormalTok{final runtime PSet}
\NormalTok{    → materialized on worker node}
```

That would make CMSDiracAux much more compatible with both classic DIRAC and future DIRACX-style workflow execution, while preserving the essential CMS semantics that WMBS originally existed to protect.

268 CMSDiracAux

In an early attempt to address this problem in the CMSDiracAux project was implemented a mechanism of taking any WMBS defined job parameter and attach them to the DIRAC job as external attribute and mark them accordingly with a label as `__CMSJobParameter`

The main method for creating the classical DIRAC jobs called `createCMSJob()` is using `__CMSJobParameter__` to take a flat WMCore job dictionary from the CMS Job-Package (A bundle of CMS jobs predefined at WMBS) and inject each entry into the DIRAC workflow object as a workflow/job parameter. In the code, every CMS job field except name is added through `job._addParameter(job.workflow, parName, value, f"__CMSJobParameter__: {parName}")`. That means the function is not merely building a runnable DIRAC job shell; it is also trying to preserve the WMCore job identity and per-job runtime metadata inside the DIRAC-side workflow description. (GitHub)

Architecturally, those parameters are needed because the CMS job model carries far more job-specific state than a normal DIRAC file-driven job. In CMS, the executable is not enough: the job also needs the information that tells it which exact workload slice it is responsible for, how that slice was produced by splitting, and how it relates back to the workflow/task context. In the PoC, the parameters are the simplest way to carry that WMCore job state across the WMCore → DIRAC boundary without losing it during translation. They are effectively a compatibility payload attached to the DIRAC workflow object. (GitHub)

Within the DIRAC runtime model, this fits at the level of the job description rather than at the pure JDL level. DIRAC JDL describes the executable, arguments, input sandbox, output sandbox, and resource requirements such as Executable, Arguments, InputSandbox, and OutputSandbox. That is enough for generic grid execution, but not enough by itself to encode the richer CMS job semantics. The extra `__CMSJobParameter__` values therefore belong to the workflow/job-description layer, where they can travel with the job definition and be available to the runtime logic that interprets that definition. This is consistent with DIRAC's model, where JDL is the submission envelope and the runtime behavior can depend on richer job metadata beyond those basic submission attributes. (DIRAC Documentation)

In the specific `createCMSJob()` prototype, those parameters coexist with a very CMS-specific three-step runtime chain: first clone the CMSDiracAux repo, then source `env.sh`, then call `Startup.py`. So the intention is clear: the DIRAC job is being used as a bootstrap shell for executing a CMS-style runtime environment, and the injected CMS job parameters are there so the CMS bootstrap layer can recover the WMCore job semantics from inside the DIRAC execution container. The code currently shows the bootstrap stage more clearly than the final parameter consumption stage, but the design intent is explicit. (GitHub)

That is also why these parameters matter in the broader CMSDiracAux concept. The project is not trying to submit ordinary DIRAC user jobs; it is trying to preserve

enough CMS semantics that a DIRAC-executed payload can still behave like a CMS job. The canonical code in the newer Interop layer shows the same overall direction: preserve CMS workflow/task/splitting information in translation objects and then materialize them into local DIRAC-like jobs and transformations. The CanonicalTask, CanonicalStep, and CanonicalSplitting dataclasses explicitly preserve request name, task path, input dataset, splitting mode, resource hints, executable, arguments, and related metadata. The older `__CMSJobParameter__` approach in `wm-cGet.py` is the direct, flat, early prototype version of that same preservation strategy. (GitHub)

So the purpose of the `__CMSJobParameter__` fields is not cosmetic and not redundant with the JDL. They are there to smuggle CMS-specific job semantics into a DIRAC workflow object, because the plain DIRAC submission description cannot by itself represent everything a CMS job needs. In the CMSDiracAux bundle and concept, they are needed in the use cases where the runtime must know not only “what executable to run” but also “which exact WMCore-generated unit of work this job represents.” That includes reconstructing job identity, reconnecting to `WMWorkload.pkl` and `JobPackage.pkl`, preserving splitting outcomes, and eventually allowing a DIRAC-side runtime or plugin layer to behave in a WMBS-like way rather than as a purely file-based DIRAC job. (GitHub)

The most important conceptual point is this: in classic DIRAC, job metadata usually supports execution; in CMSDiracAux, these `__CMSJobParameter__` entries are trying to preserve workflow semantics. That is why they are needed, and that is where they fit. (GitHub)

268.1 Relation to the IR layer of CMSDiracAux

Yet another layer of CMSDiracAux which relates to the problem of distributing job specific parameters to the runtime environment is the IR layer. The both methods, though, are **not strict alternatives**.

The best way to think about this is:

```
\NormalTok{Translation IR}
\NormalTok{      = canonical architecture{-}level representation}

\NormalTok{\_\_CMSJobParameter\_\_}
\NormalTok{      = runtime/job{-}level transport mechanism}
```

In principle:

- the **IR layer replaces the architectural role** that the flat `__CMSJobParameter__` bundle was playing in the early prototype
- but it does **not automatically eliminate** the need for some **runtime-carried per-job metadata**.

Shortly:

```
\NormalTok{IR layer ≠ direct drop{-}in replacement for \_\_CMSJobParameter\_\_}
```

More precisely:

```
\NormalTok{IR replaces their role as the main translation abstraction,}  
\NormalTok{but some job{-}level metadata still has to reach runtime.}
```

Unless runtime reconstruction is redesigned further, those two mechanisms will likely need to work in a **coupled or successor relationship**, not as mutually exclusive options.

The old `__CMSJobParameter__` mechanism is essentially:

```
\NormalTok{flat WMCore job metadata}  
\NormalTok{      ↓}  
\NormalTok{attached directly to DIRAC job/workflow}  
\NormalTok{      ↓}  
\NormalTok{available at runtime}
```

It is an **early prototype bridge**.

The IR layer does something more structured:

```
\NormalTok{WMCore workflow}  
\NormalTok{      ↓}  
\NormalTok{canonical normalized objects}  
\NormalTok{      ↓}  
\NormalTok{materialized DIRAC jobs / transformations}
```

The IR is a **better place to preserve meaning**, but the worker node still needs some of that meaning at runtime.

That means the question becomes:

```
\NormalTok{How does IR information get projected into runtime{-}consumable form?}
```

And there are only a few possibilities:

268.1.1 1. Flat job parameters

Equivalent to the old `__CMSJobParameter__` idea.

268.1.2 2. Structured sidecar artifact

For example, a job-local JSON/YAML/pkl emitted from the IR.

268.1.3 3. Runtime reconstruction from references

The job receives only IDs / minimal metadata and reconstructs the rest.

269 Current architectural interpretation

Given how CMSDiracAux is evolving, the most likely correct interpretation is:

```
\NormalTok{IR layer}
\NormalTok{      = source of truth}

\NormalTok{\_\_CMSJobParameter\_\_{-}like payload}
\NormalTok{      = one possible projection of IR into runtime}
```

So if kept, the parameter bundle will become:

```
\NormalTok{derived runtime view of the IR}
```

rather than an independent parallel truth source.

That is the important distinction.

270 In practice

If the project stays close to the current bootstrap model, then some coupling is still needed:

```
\NormalTok{IR}
\NormalTok{  ↓}
\NormalTok{job materialization}
\NormalTok{  ↓}
\NormalTok{runtime metadata injection}
\NormalTok{  ↓}
\NormalTok{bootstrap / Startup.py / cmsRun}
```

In that case, `__CMSJobParameter__` or its successor still exists, but only as a **runtime transport layer**.

If the project later moves toward:

- CVMFS-hosted common runtime
- runtime-side PSet reconstruction
- structured per-job metadata artifacts

then the old flat parameter bundle can likely be **reduced or replaced**.

271 In Conclusion

The Translation IR in CMSDiracAux should replace `__CMSJobParameter__` as the **primary semantic representation**, but it does not automatically replace the need for **runtime-delivered per-job metadata**. Therefore,

at the current conceptual stage, they should be viewed as working in a coupled manner, with `__CMSJobParameter__`-like data becoming a runtime projection of the IR rather than an alternative to it.

So:

```
\NormalTok{today: coupled}
```

```
\NormalTok{future ideal state: IR primary, runtime projection minimized}
```

272 Summary

CMS and DIRAC differ fundamentally in how jobs are defined and executed.

CMS jobs rely on runtime configuration and workflow semantics derived from WM-Core, while DIRAC jobs rely on minimal job descriptions executed through a pilot-based infrastructure.

CMSDiracAux bridges this gap by introducing a Translation IR and by reconstructing CMS runtime semantics inside DIRAC jobs through bootstrap mechanisms and metadata projection.

Moving common runtime components to CVMFS and reconstructing per-job PSet configurations dynamically can significantly improve compatibility between CMS runtime requirements and DIRAC execution constraints.

\newpage

273 CMS Runtime Construction Challenges

The CMS runtime model is substantially more complex than a plain DIRAC job template.

273.1 Inputs to runtime behavior

Runtime behavior depends on at least two major sources:

1. the actual data assigned to the current job,
2. the physics software configuration and parameter set used by the job.

In practice, the physics configuration often needs to be tweaked at runtime so that it reflects the exact boundaries inside the data that the current job is supposed to process.

This is necessary to preserve CMS data uniqueness guarantees.

273.2 Runtime dependency layers

The effective runtime environment is built from multiple dependency layers:

- WMCore runtime code distributed with the job sandbox,
- CMSSW runtime code mounted through CVMFS,
- worker-node architecture and hardware properties.

These layers are linked through workload/job description objects such as the serialized WMWorkload and WMBS job definitions.

273.3 Why this matters

DIRAC is not naturally built around this exact runtime construction model.

This is one of the reasons why removing static early splitting is difficult: the runtime boundary logic and the workload/job description objects are tightly connected.

273.4 Current PoC approach

The current stage materializes local DIRAC-like Job and Transformation objects that explicitly preserve:

- CMS runtime bootstrap steps,
- carried WMBS job parameters,
- the link back to serialized WMCore objects.

This is an intermediate executable representation rather than the final target architecture.

273.5 Future direction

A later stage may express these workflow objects in a more portable workflow language aligned with future DIRACX directions, while using the current DIRAC Python object model only as an intermediate representation.

\newpage

274 WMCore → DIRAC Parameter Mapping

This document describes the conceptual and practical parameter mapping between WMCore workflow objects and the corresponding DIRAC constructs used in the CMS-DiracAux interoperability proof of concept.

The mapping is implemented through an intermediate canonical representation referred to as the Translation IR.

1. Translation layers

The interoperability layer introduces three conceptual levels.

WMCore objects | v Canonical Translation IR | v DIRAC workflow objects

WMCore represents CMS workflow semantics.

DIRAC represents workload execution semantics.

The Translation IR bridges the conceptual differences between the two systems.

2. WMCore request → Production mapping

WMCore Request objects describe the global workflow context.

Example WMCore fields:

RequestName Campaign RequestPriority ProcessingString AcquisitionEra PrepID

Translation IR production fields:

ProductionName CampaignName Priority ProcessingString AcquisitionEra PrepID

DIRAC representation:

Transformation group Production metadata Transformation priority

Conceptual mapping:

WMCore Request | v IR Production | v DIRAC Transformation metadata

3. WMCore Task → DIRAC Transformation

In WMCore the Task represents the main execution unit.

Example WMCore Task parameters:

taskName pathName input.dataset.name splitting configuration

Example WMCore task path:

/pdmvserv_Run2024E_DisplacedJet_MINIv6NANOv15_260309_125412_4202/DataProcessing

Derived IR task parameters:

TaskName TaskPath InputDataset Splitting Step

DIRAC equivalent:

Transformation definition Transformation body Transformation parameters

Conceptual mapping:

4. WMCore Step → DIRAC Job workflow

A WMCore Step describes a runtime processing stage.

Typical WMCore Step properties:

stepName runtime configuration CMSSW version Scram architecture

Example WMCore step:

cmsRun

IR representation:

StepName Executable Arguments SoftwareVersion SoftwareArchitecture

DIRAC equivalent:

jobDescription.xml workflow step

Conceptual mapping:

WMCore Step | v IR Step | v DIRAC job workflow definition

5. WMCore Splitting → DIRAC Transformation Plugin

WMCore splitting defines how datasets are divided across jobs.

Example splitting parameters:

algorithm events_per_job files_per_job lumis_per_job

IR representation:

PluginName SplitMode FilesPerJob EventsPerJob LumisPerJob

DIRAC equivalent:

TransformationPlugin

Example plugin used in the PoC:

CMSWMCoreSplittingPlugin

Conceptual mapping:

WMCore Splitting | v IR Splitting | v DIRAC Transformation plugin

6. Dataset → LFN mapping

WMCore tasks usually reference datasets rather than explicit files.

Example dataset:

/DisplacedJet/Run2024E-2024CDEReprocessing-v1/AOD

The PoC resolves file lists through DAS.

dataset | v dasgoclient query | v file records | v LFN list

Example resolved LFN:

/store/data/Run2024E/DisplacedJet/AOD/2024CDEReprocessing-v1/2550000/7e4f9e3e-9757-4484-bc73-232921339a58.root

The resolved file metadata is propagated into the IR.

7. IR InputDataset structure

The canonical IR stores dataset information as:

DatasetHint DatasetsResolved ResolvedFileRecords ResolvedLFNs PlaceholderLFNs

Example IR dataset structure:

DatasetHint ResolvedFileRecords ResolvedLFNs

The IR preserves both:

dataset semantics file-level metadata

8. Plugin input generation

DIRAC transformations require explicit file input definitions.

The PoC generates plugin input datasets as:

PluginInput/TASKNAME.inputdata.json

Structure:

LFN events size dataset block

Example entry:

/store/data/...root events: 12889 size: 1427947644 dataset: /DisplacedJet/Run2024E-2024CDEReprocessing-v1/AOD

9. PoC scalability limitation

The current implementation intentionally caps file materialization.

Maximum files per dataset:

20

Reason:

Datasets may contain thousands of files.

Example observed dataset:

~7100 files

Generating a job definition per file would produce an extremely large local transformation structure.

Important:

This limit is temporary and exists only for the proof of concept.

10. CMS data hierarchy implications

CMS data follows a strict hierarchical structure.

dataset | v block | v file

Production workflows often operate across:

thousands of files hundreds of blocks

The current PoC operates only on a small subset of the file layer.

11. Future mapping extensions

Future work may extend the mapping to include:

run and lumi masks

block-level dataset partitioning

DBS metadata enrichment

Rucio integration for data management

CWL-native workflow definitions compatible with DIRACX

12. Summary

The Translation IR enables a clean conceptual separation between:

WMCore workflow semantics DIRAC execution semantics

The mapping can be summarized as:

WMCore Request | v IR Production | v DIRAC Transformation metadata

WMCore Task | v IR Task | v DIRAC Transformation

WMCore Step | v IR Step | v DIRAC job workflow

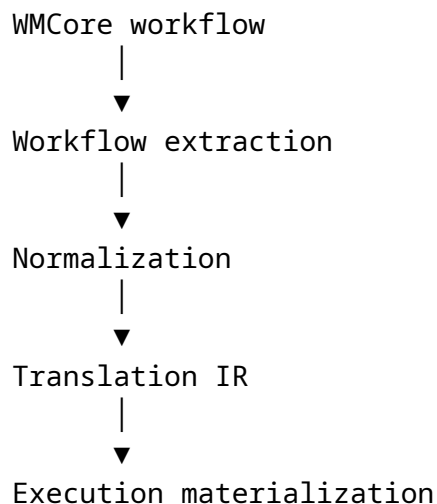
\newpage

275 Translator Design

275.1 Overview

The CMSDiracAux translator is responsible for converting workflow definitions originating from the CMS workflow management system (WMCore) into a canonical intermediate representation that can later be materialized into execution structures compatible with the DIRAC distributed computing framework.

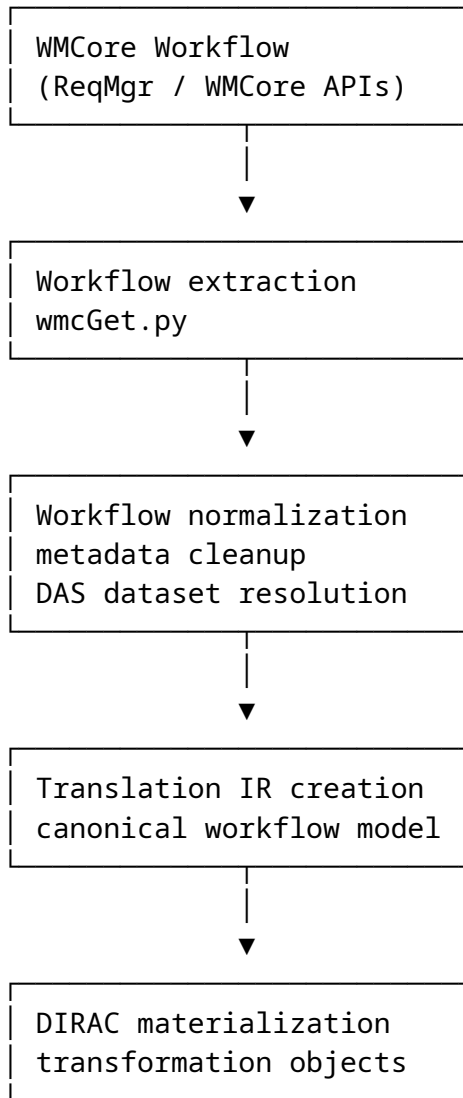
The translator operates as a multi-stage pipeline:



The translator isolates the semantic components of the workflow from execution details.

276 Translator Pipeline

The current proof-of-concept pipeline consists of the following stages:



Each stage performs a well-defined transformation.

277 Workflow Extraction

The workflow extraction stage retrieves workflow definitions from WMCore and serializes them into portable artifacts.

This stage is implemented by:

wmcGet.py

Its responsibilities include:

- retrieving workflow request information
- extracting task graphs
- collecting dataset references
- capturing splitting policies
- serializing workflow metadata

The output of this stage is a **serialized representation of the WMCore workflow structure**.

278 Workflow Normalization

After extraction the workflow description is normalized.

Normalization includes:

- resolving dataset references
- querying DAS for file lists
- converting metadata formats
- preparing splitting information

Dataset resolution pipeline:

```
dataset name
  |
  ▼
DAS query
  |
  ▼
file records
  |
  ▼
LFN list
```

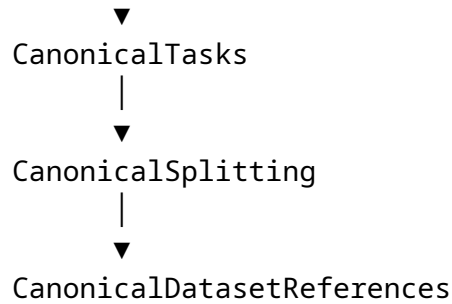
Normalization ensures the workflow description becomes deterministic and portable.

279 Translation IR Construction

The canonical workflow representation is built during this stage.

The Translation IR captures the following elements:

```
CanonicalWorkflow
  |
```



The IR intentionally removes WMCore-specific implementation details while preserving workflow semantics.

280 Translation Strategy

The translation logic follows a structured mapping approach.

WMCore Object	Translation IR Object
Workflow	CanonicalWorkflow
Task	CanonicalTask
Splitting policy	CanonicalSplitting
Dataset reference	CanonicalDataset

This mapping preserves the conceptual workflow structure while enabling independent execution backends.

281 Translation Design Principles

The translator is designed according to several principles.

281.0.1 Deterministic translation

Given the same WMCore workflow definition, the translator must produce identical Translation IR objects.

281.0.2 Execution independence

The IR must not depend on a specific execution system.

281.0.3 Minimal loss of semantics

Important workflow parameters such as splitting policies and runtime configuration must remain intact.

281.0.4 Extensibility

The translator should allow future execution targets to be supported without rewriting the extraction logic.

282 Relationship to Execution Systems

The translator intentionally stops before execution system specifics are introduced. Execution systems are handled in the subsequent stage:

Translation IR



DIRAC materialization

This separation ensures that workflow semantics remain stable even if the execution infrastructure evolves.

283 Summary

The CMSDiracAux translator forms the front end of the interoperability pipeline.

It extracts CMS workflows from WMCore, normalizes their metadata, and constructs a canonical representation that can later be used to generate execution structures for the DIRAC distributed computing framework.

\newpage

284 DIRAC Materialization

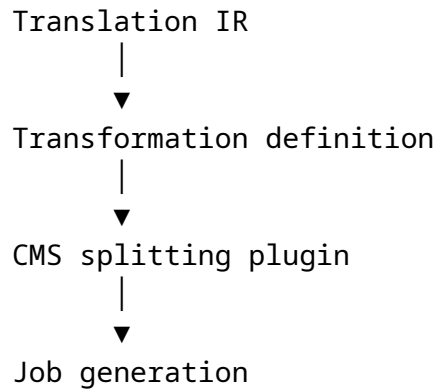
284.1 Overview

After the Translation IR has been constructed, the workflow must be materialized into execution structures compatible with the DIRAC distributed computing framework.

This stage converts canonical workflow descriptions into DIRAC execution constructs such as transformations and jobs.

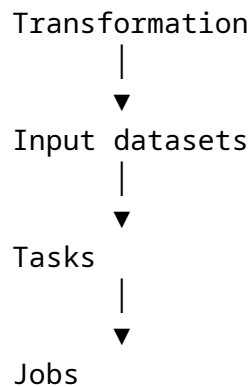
285 Materialization Pipeline

The materialization stage transforms canonical objects into DIRAC execution entities.



286 DIRAC Transformation System

The DIRAC Transformation System is responsible for generating jobs from data inputs and workflow definitions.

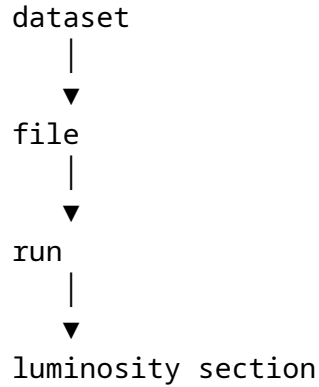


In CMSDiracAux, transformations are created using metadata derived from the Translation IR.

287 CMS Splitting Plugin

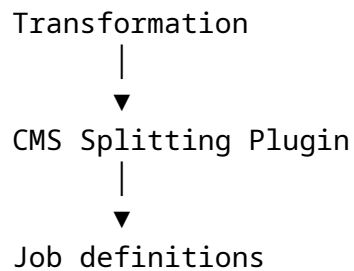
CMS workflows require splitting policies that operate at fine data granularity.

Typical splitting levels include:



DIRAC normally performs splitting at the file level.

To preserve CMS workflow semantics, a CMS-specific splitting plugin is introduced.



This plugin reproduces the logic originally implemented in WMBS.

288 Job Description Construction

Once jobs are generated, the job description must be constructed.

A typical DIRAC job description includes:

Executable
Arguments
InputSandbox
OutputSandbox
CPUTime

CMSDiracAux extends the job description to include CMS-specific runtime parameters.

These parameters correspond to the configuration required by the CMS runtime environment.

289 Runtime Parameter Injection

CMS jobs require runtime configuration artifacts.

Typical parameters include:

```
__CMSJobParameter__dataset  
__CMSJobParameter__runRange  
__CMSJobParameter__lumiMask  
__CMSJobParameter__psetTweaks
```

These parameters ensure that the job processes the correct portion of the dataset.

290 Worker Node Execution

Once jobs are scheduled they execute on worker nodes.

Runtime reconstruction pipeline:

```
DIRAC job starts  
|  
▼  
Bootstrap script  
|  
▼  
Runtime environment preparation  
|  
▼  
cmsRun execution
```

The bootstrap stage prepares the CMS runtime environment.

291 Summary

DIRAC materialization converts canonical workflow descriptions into executable job structures.

This stage preserves CMS workflow semantics while enabling execution through the DIRAC workload management infrastructure.

\newpage

292 CWL Export

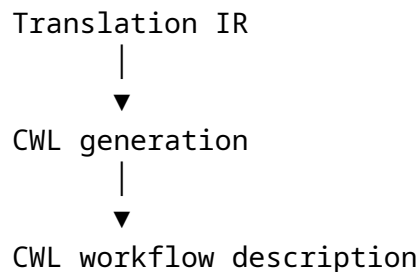
292.1 Motivation

One of the objectives of CMSDiracAux is to demonstrate that workflows defined in WMCore can be expressed in a portable workflow representation.

The Common Workflow Language (CWL) provides a standardized description format for computational workflows.

Exporting workflows to CWL enables experimentation with modern workflow execution architectures.

293 Export Pipeline



The Translation IR contains all information required to generate a CWL representation.

294 Workflow Representation

The CWL export stage generates:

`workflow.cwl`
`job.yaml`

These files describe:

- workflow steps
 - inputs and outputs
 - runtime commands
 - data dependencies
-

295 Mapping IR Objects to CWL

IR Object	CWL Representation
CanonicalWorkflow	CWL workflow
CanonicalTask	CWL step
CanonicalDataset	CWL input
CanonicalSplitting	CWL parameterization

296 Experimental Nature

The CWL export functionality is currently experimental.

Its main goal is to demonstrate that workflow descriptions can be expressed independently from both WMCore and DIRAC.

297 Summary

The CWL export stage illustrates the portability of the Translation IR by demonstrating that workflows defined in WMCore can be expressed in a generic workflow language.

\newpage

298 Proof-of-Concept Evaluation

298.1 Overview

The CMSDiracAux prototype demonstrates that workflows defined within the CMS workflow management system can be translated into execution structures compatible with the DIRAC distributed computing framework.

The prototype focuses on validating the architectural feasibility of such a translation rather than implementing a full production-grade system.

299 Translation Feasibility

The project successfully demonstrates that workflow descriptions originating from WMCore can be converted into a canonical intermediate representation.

This representation captures:

- workflow metadata
- task structure
- dataset references
- splitting policies

The resulting IR provides a stable abstraction for further processing.

300 Execution Model Compatibility

The translation pipeline successfully bridges the conceptual differences between CMS and DIRAC execution models.

CMS workflows follow an explicit workflow evolution model:

`workflow → tasks → jobs`

DIRAC workflows follow a dynamic execution model:

`transformation → tasks → jobs`

The Translation IR allows CMS workflows to be expressed in a form compatible with the DIRAC transformation system.

301 Preservation of Splitting Semantics

A key challenge in the translation process is preserving CMS splitting semantics.

CMS workflows often define splitting policies at the level of data content rather than storage containers.

The CMSDiracAux architecture demonstrates that this functionality can be reproduced inside DIRAC through a CMS-specific splitting plugin.

302 Runtime Reconstruction

The prototype demonstrates the feasibility of reconstructing CMS runtime environments on worker nodes using bootstrap scripts.

These scripts inject the runtime parameters required by CMS jobs and ensure that the correct dataset partitions are processed.

303 Architectural Insight

The most important architectural insight emerging from this work is that CMS workflow bookkeeping cannot be removed simply by changing the execution backend.

Even when workflows are executed through DIRAC, the system must maintain the mapping between:

workflow tasks
data partitions
jobs

This mapping was originally implemented by WMBS and must be reproduced within the DIRAC transformation system.

304 Summary

The CMSDiracAux proof-of-concept confirms that interoperability between the CMS workflow management system and the DIRAC distributed computing framework is achievable through a carefully designed translation architecture centered around a canonical intermediate representation.

305 Proof of Concept Evaluation

This document summarizes the results, limitations, and lessons learned from the WM-Core → DIRAC interoperability proof of concept implemented in CMSDiracAux.

The goal of the PoC was not to provide a production-ready integration, but to demonstrate that CMS workflows defined in WMCore can be translated into a form compatible with DIRAC-style execution models and portable workflow representations.

1. PoC goals
-

The proof of concept aimed to demonstrate the following capabilities.

1. Extract workflow information from WMCore.
2. Translate WMCore workflow structures into an intermediate representation independent from both WMCore and DIRAC.
3. Materialize a local representation of DIRAC transformations.
4. Simulate DIRAC transformation splitting behavior locally.
5. Resolve CMS datasets into file lists using DAS.

6. Export the resulting workflow structure into a CWL-compatible workflow bundle.

2. Successfully demonstrated capabilities

The PoC successfully demonstrated the following architectural concepts.

Workflow extraction

WMCore workflows can be serialized into portable JSON artifacts.

Translation layer

A canonical Translation IR can represent workflow semantics in a system-neutral way.

Dataset discovery

CMS datasets can be resolved to file lists using DAS queries.

Transformation simulation

DIRAC-style transformations and job definitions can be materialized locally without running DIRAC services.

Workflow export

The translated workflow can be exported into CWL, aligning with future workflow-language-based execution models.

3. Key architectural achievements

Translation IR abstraction

The introduction of a canonical intermediate representation provides a clean separation between:

workflow definition semantics

and

execution infrastructure semantics.

Request-scoped artifact layout

All artifacts produced during translation are grouped under a single request directory.

```
REQUEST_ROOT | |-- WMCore.fetched.d | |-- DIRAC.transf.d | '-- DIRAC.cwl.d
```

This structure makes the pipeline easy to inspect and debug.

Local transformation simulation

DIRAC transformations can be simulated locally using:

`runLocalTransformation.py`

This allows transformation logic to be tested without a full DIRAC server deployment.

CWL workflow export

The ability to export the workflow into CWL demonstrates that the workflow representation can be decoupled from the legacy DIRAC Python object model.

4. Current limitations

Several limitations remain in the current implementation.

Dataset materialization limit

The PoC currently limits dataset materialization to:

20 files per dataset.

Reason

Large CMS datasets often contain thousands of files.

Example dataset used during testing:

approximately 7100 files.

Without this limit the PoC would generate extremely large job structures.

Run and luminosity section handling

CMS workflows operate at run and luminosity section granularity.

The PoC currently operates only at file granularity.

DIRAC server integration

The PoC simulates DIRAC transformations locally.

Server-side DIRAC components such as the Transformation Agent are not currently integrated.

CMS runtime integration

CMS jobs require a complex runtime environment involving:

CMSSW software

WMCore runtime modules

runtime configuration artifacts.

The PoC currently focuses on workflow structure rather than full runtime execution.

5. Lessons learned

Different workflow abstractions

WMCore and DIRAC operate at fundamentally different conceptual levels.

WMCore describes physics workflows.

DIRAC schedules distributed workloads.

Importance of an intermediate representation

Attempting to directly map WMCore objects to DIRAC objects would introduce tight coupling between systems.

The Translation IR enables clean abstraction boundaries.

Dataset discovery is essential

WMCore workflows reference datasets rather than explicit files.

Therefore dataset discovery must occur before job definitions can be generated.

CMS data hierarchy matters

CMS data is organized as:

dataset → block → file

Understanding this hierarchy is essential when designing splitting and data discovery logic.

6. Future work

Several directions remain for future development.

Improved dataset handling

Block-level dataset partitioning

Richer DBS metadata usage

More scalable dataset materialization

DIRAC integration

Server-side deployment of CMS transformation plugins

Integration with the DIRAC Transformation Agent

Workflow representation

Native CWL workflows aligned with DIRACX
Workflow portability across execution infrastructures
CMS runtime support
Integration of CMSSW runtime environments
Support for CMS job sandbox semantics

7. PoC conclusion

The proof of concept demonstrates that CMS workflows defined in WMCore can be translated into an intermediate representation that can be materialized as DIRAC-style transformations and exported as portable workflow definitions.

The Translation IR serves as the central abstraction layer enabling this interoperability.

Although significant engineering work remains for production-scale integration, the PoC validates the architectural feasibility of this approach.

\newpage

306 Limitations

The CMSDiracAux project is currently a proof-of-concept implementation designed to explore the interoperability between the CMS workflow management system and the DIRAC distributed computing framework.

Several limitations exist in the current prototype.

307 Dataset Size Limitation

The prototype limits dataset materialization to a small number of files.

Typical CMS datasets may contain thousands of files.

In the current proof-of-concept implementation the number of files processed during testing is restricted.

This limitation exists to keep generated job structures manageable during experimentation.

308 Incomplete Run/Lumi Support

The prototype demonstrates run and luminosity section splitting concepts but does not fully implement all possible CMS splitting modes.

Some workflow configurations used in production CMS computing are therefore not yet supported.

309 Runtime Environment Distribution

CMS runtime environments depend on both:

WMCore runtime artifacts
CMSSW software environment

Reconstructing this environment within the DIRAC execution model remains an open challenge.

Future work will explore improved runtime environment distribution strategies.

310 DIRAC Server Integration

The current implementation performs transformation materialization locally.

Full integration with the DIRAC server infrastructure is not yet implemented.

This includes:

- server-side transformation agents
 - persistent task queues
 - production-scale scheduling.
-

311 Experimental CWL Export

The CWL export stage is currently experimental and intended for demonstration purposes.

It does not yet represent the full complexity of CMS workflows.

312 Summary

Despite these limitations, the CMSDiracAux prototype successfully demonstrates the feasibility of translating CMS workflows into execution structures compatible with the DIRAC distributed computing framework.

\newpage

313 Current Stage Limitations and Checkpoints

313.1 Server-side limitation

The current environment does not provide access to a CMS-specific DIRAC server-side extension deployment.

As a result, the current prototype can:

- translate serialized WMCore workflow artifacts,
- materialize local DIRAC-like Job and Transformation objects,
- execute the CMSWMCoreSplittingPlugin locally,

but cannot yet produce real server-side Transformation task creation.

313.2 Data visibility limitation

The current DIRAC test environment does not see CMS data or CMS storage definitions in the way needed for real CMS data discovery and runtime execution.

Because of that, the current stage uses placeholder LFNs and static plugin-input sidecars.

313.3 Postponed topics

The following topics are intentionally postponed:

- intra-file splitting,
- run/lumi masks,
- CMS DBS/DAS-based data discovery,
- integration or replacement of DIRAC data-management calls with Rucio,
- full CMS runtime and sandbox distribution redesign.

313.4 Current milestone

The current milestone is:

- represent the WMCore workflow structure faithfully,
- preserve key CMS/WMBS semantics locally,
- materialize DIRAC-style objects on disk,

- prepare the path toward later workflow-language translation and/or DIRACX-facing evolution.

\newpage

314 Checkpoint — Local Transformation Materialization

Date: 2026-03-12

314.1 Achieved

Implemented a complete local WMCore to DIRAC translation pipeline.

Components

1. wmcGet.py
Fetches workflows and serializes WMCore objects.
2. wmc2transf.py
Converts serialized objects into canonical translation objects and emits local DIRAC style artifacts.
3. runLocalTransformation.py
Runs CMSWMCoreSplittingPlugin locally and produces grouped tasks and per task job descriptions.

Result

A fully local transformation simulation independent of server side DIRAC.

314.2 Local Pipeline

WMCore request ↓ wmcGet.py ↓ WM JSON artifacts ↓ wmc2transf.py ↓ Transformations PluginInput Jobs ↓ runLocalTransformation.py ↓ Tasks TaskJobs

314.3 Important Decision

Server side DIRAC integration is postponed.

The available environment

- does not provide CMS DIRAC extensions
- cannot access CMS DBS or DAS
- cannot register transformation plugins

Therefore the PoC focuses on correct translation and local object materialization.

314.4 Known Gaps

Repository main branch currently contains partial merges in the emit and materialization layer.

Modules requiring synchronization

emit.py
io.py
task_materialize.py
runLocalTransformation.py

314.5 Next Step

Restore repository consistency across

io.py
model.py
normalize.py
materialize.py
emit.py
task_materialize.py
runLocalTransformation.py
wmc2transf.py

314.6 Future Branch

WMJob.json aware task job generation.

This will allow emitted jobs to inherit WMBS job parameters when available.

\newpage

315 Checkpoint — DAS LFN Resolution and PoC File Cap

Date 2026-03-13

315.1 Current milestone state

The WMCore → DIRAC PoC pipeline is now able to:

1. Fetch and serialize a CMS workflow from WMCore

2. Translate the workflow into a canonical translation IR
3. Materialize a local DIRAC-style transformation bundle
4. Resolve real CMS dataset LFNs using DAS (dasgoclient)
5. Propagate real file metadata into plugin input structures
6. Export the materialized transformation into a CWL-compatible bundle

End-to-end pipeline:

WMCore request | v wmcGet.py | v serialized WM JSON artifacts | v wmc2transf.py
 | v canonical translation objects | v local DIRAC-style artifacts | v runLocalTransformation.py | v CMSWMCoreSplittingPlugin | v Tasks and TaskJobs | v transf2cwl.py |
 v CWL bundle

315.2 Output directory structure

Each request is materialized under a request-scoped root directory.

REQUEST_ROOT | |-- WMCore.fetched.d | |-- DIRAC.transf.d | '-- DIRAC.cwl.d

Meaning:

WMCore.fetched.d Serialized WMCore objects such as:

- WMTask.json
- WMStep.json
- WMSplitting.json

DIRAC.transf.d Local DIRAC-style materialization artifacts:

- transformation definitions
- job descriptions
- plugin input data
- task objects

DIRAC.cwl.d CWL-export bundle representing the same workflow.

315.3 DAS-based LFN resolution

Dataset LFNs are now resolved using:

dasgoclient

The call is executed through an interactive shell so that environments where dasgoclient is provided through a shell alias (for example via CVMFS) work correctly.

Example dataset:

/DisplacedJet/Run2024E-2024CDEReprocessing-v1/AOD

Example resolved LFN:

/store/data/Run2024E/DisplacedJet/AOD/2024CDEReprocessing-v1/2550000/7e4f9e3e-9757-4484-bc73-232921339a58.root

The translator now propagates:

- full DAS file records
- per-file event counts
- per-file metadata

into the canonical IR and plugin input data.

315.4 PoC scalability limitation

The current PoC intentionally materializes only the first 20 files per dataset.

This cap is implemented during the canonical IR normalization stage.

Reason:

Generating a job definition per file becomes extremely heavy when datasets contain thousands of files.

Example dataset size in current testing:

~7100 files

Without the cap, the PoC would generate thousands of jobs and large plugin input artifacts.

This limit is strictly temporary and must be clearly documented in the report.

315.5 CMS data hierarchy relevance

This limitation is particularly important because CMS data follows a strict hierarchy.

dataset | v block | v file

Large CMS productions operate at:

- dataset scale
- block scale
- file scale
- sometimes run/lumi scale

The current PoC only operates at a small subset of the file layer.

Therefore performance and scalability conclusions cannot yet be drawn.

315.6 DIRAC runtime observations

During testing we confirmed:

- runtime parameters must ultimately live in jobDescription.xml

- .jdl files should not attempt to propagate large sandboxes
- DIRAC sandbox handling differs from CMS WMCore runtime assumptions

This topic is marked for a dedicated report section:

DIRAC InputSandbox vs jobDescription.xml runtime contract.

315.7 Next technical directions

Data discovery improvements:

- more robust DAS queries
- optional DBS-based metadata enrichment
- better block-level dataset handling

CWL export refinement:

Continue aligning the exported bundle with:

DIRACGrid/dirac-cwl

Goals:

- validate workflow with cwltool
- preserve DIRAC task metadata
- maintain compatibility with future DIRACX workflows

CMS runtime bootstrap analysis:

Further work needed on:

- CMSSW runtime distribution
- WMCore runtime sandbox structure
- interaction with CVMFS
- possible future Rucio integration

315.8 Report workstream reminder

The final report must include:

- architecture diagram of the full system
- parameter mapping tables
- comparison between WMCore and DIRAC execution models
- discussion of WMBS splitting vs DIRAC task generation
- the current PoC file cap limitation
- CMS dataset/block/file hierarchy explanation